
Concurrency Models in Python

Concurrency is about dealing with lots of things at once.

Parallelism is about doing lots of things at once.

Not the same, but related.

One is about structure, one is about execution.

Concurrency provides a way to structure a solution to solve a problem that may (but not necessarily) be parallelizable.

—Rob Pike, co-inventor of the Go language¹

This chapter is about how to make Python deal with “lots of things at once.” This may involve concurrent or parallel programming—even academics who are keen on jargon disagree on how to use those terms. I will adopt Rob Pike’s informal definitions in this chapter’s epigraph, but note that I’ve found papers and books that claim to be about parallel computing but are mostly about concurrency.²

Parallelism is a special case of concurrency, in Pike’s view. All parallel systems are concurrent, but not all concurrent systems are parallel. In the early 2000s we used single-core machines that handled 100 processes concurrently on GNU Linux. A modern laptop with 4 CPU cores is routinely running more than 200 processes at any given time under normal, casual use. To execute 200 tasks in parallel, you’d need 200 cores. So, in practice, most computing is concurrent and not parallel. The OS

¹ Slide 8 of the talk “[Concurrency Is Not Parallelism](#)”.

² I studied and worked with Prof. Imre Simon, who liked to say there are two major sins in science: using different words to mean the same thing and using one word to mean different things. Imre Simon (1943–2009) was a pioneer of computer science in Brazil who made seminal contributions to Automata Theory and started the field of Tropical Mathematics. He was also an advocate of free software and free culture.

manages hundreds of processes, making sure each has an opportunity to make progress, even if the CPU itself can't do more than four things at once.

This chapter assumes no prior knowledge of concurrent or parallel programming. After a brief conceptual introduction, we will study simple examples to introduce and compare Python's core packages for concurrent programming: `threading`, `multi` processing, and `asyncio`.

The last 30% of the chapter is a high-level overview of third-party tools, libraries, application servers, and distributed task queues—all of which can enhance the performance and scalability of Python applications. These are all important topics, but beyond the scope of a book focused on core Python language features. Nevertheless, I felt it was important to address these themes in this second edition of *Fluent Python*, because Python's fitness for concurrent and parallel computing is not limited to what the standard library provides. That's why YouTube, DropBox, Instagram, Reddit, and others were able to achieve web scale when they started, using Python as their primary language—despite persistent claims that “Python doesn't scale.”

What's New in This Chapter

This chapter is new in the second edition of *Fluent Python*. The spinner examples in “A Concurrent Hello World” on page 701 previously were in the chapter about *asyncio*. Here they are improved, and provide the first illustration of Python's three approaches to concurrency: threads, processes, and native coroutines.

The remaining content is new, except for a few paragraphs that originally appeared in the chapters on `concurrent.futures` and *asyncio*.

“Python in the Multicore World” on page 725 is different from the rest of the book: there are no code examples. The goal is to mention important tools that you may want to study to achieve high-performance concurrency and parallelism beyond what's possible with Python's standard library.

The Big Picture

There are many factors that make concurrent programming hard, but I want to touch on the most basic factor: starting threads or processes is easy enough, but how do you keep track of them?³

When you call a function, the calling code is blocked until the function returns. So you know when the function is done, and you can easily get the value it returned. If

³ This section was suggested by my friend Bruce Eckel—author of books about Kotlin, Scala, Java, and C++.

the function raises an exception, the calling code can surround the call site with `try/except` to catch the error.

Those familiar options are not available when you start a thread or process: you don't automatically know when it's done, and getting back results or errors requires setting up some communication channel, such as a message queue.

Additionally, starting a thread or a process is not cheap, so you don't want to start one of them just to perform a single computation and quit. Often you want to amortize the startup cost by making each thread or process into a “worker” that enters a loop and stands by for inputs to work on. This further complicates communications and introduces more questions. How do you make a worker quit when you don't need it anymore? And how do you make it quit without interrupting a job partway, leaving half-baked data and unreleased resources—like open files? Again the usual answers involve messages and queues.

A coroutine is cheap to start. If you start a coroutine using the `await` keyword, it's easy to get a value returned by it, it can be safely cancelled, and you have a clear site to catch exceptions. But coroutines are often started by the asynchronous framework, and that can make them as hard to monitor as threads or processes.

Finally, Python coroutines and threads are not suitable for CPU-intensive tasks, as we'll see.

That's why concurrent programming requires learning new concepts and coding patterns. Let's first make sure we are on the same page regarding some core concepts.

A Bit of Jargon

Here are some terms I will use for the rest of this chapter and the next two:

Concurrency

The ability to handle multiple pending tasks, making progress one at a time or in parallel (if possible) so that each of them eventually succeeds or fails. A single-core CPU is capable of concurrency if it runs an OS scheduler that interleaves the execution of the pending tasks. Also known as multitasking.

Parallelism

The ability to execute multiple computations at the same time. This requires a multicore CPU, multiple CPUs, a GPU, or multiple computers in a cluster.

Execution unit

General term for objects that execute code concurrently, each with independent state and call stack. Python natively supports three kinds of execution units: *processes*, *threads*, and *coroutines*.

Process

An instance of a computer program while it is running, using memory and a slice of the CPU time. Modern desktop operating systems routinely manage hundreds of processes concurrently, with each process isolated in its own private memory space. Processes communicate via pipes, sockets, or memory mapped files—all of which can only carry raw bytes. Python objects must be serialized (converted) into raw bytes to pass from one process to another. This is costly, and not all Python objects are serializable. A process can spawn subprocesses, each called a child process. These are also isolated from each other and from the parent. Processes allow *preemptive multitasking*: the OS scheduler *preempts*—i.e., suspends—each running process periodically to allow other processes to run. This means that a frozen process can't freeze the whole system—in theory.

Thread

An execution unit within a single process. When a process starts, it uses a single thread: the main thread. A process can create more threads to operate concurrently by calling operating system APIs. Threads within a process share the same memory space, which holds live Python objects. This allows easy data sharing between threads, but can also lead to corrupted data when more than one thread updates the same object concurrently. Like processes, threads also enable *preemptive multitasking* under the supervision of the OS scheduler. A thread consumes less resources than a process doing the same job.

Coroutine

A function that can suspend itself and resume later. In Python, *classic coroutines* are built from generator functions, and *native coroutines* are defined with `async def`. “[Classic Coroutines](#)” on page 641 introduced the concept, and [Chapter 21](#) covers the use of native coroutines. Python coroutines usually run within a single thread under the supervision of an *event loop*, also in the same thread. Asynchronous programming frameworks such as *asyncio*, *Curio*, or *Trio* provide an event loop and supporting libraries for nonblocking, coroutine-based I/O. Coroutines support *cooperative multitasking*: each coroutine must explicitly cede control with the `yield` or `await` keyword, so that another may proceed concurrently (but not in parallel). This means that any blocking code in a coroutine blocks the execution of the event loop and all other coroutines—in contrast with the *preemptive multitasking* supported by processes and threads. On the other hand, each coroutine consumes less resources than a thread or process doing the same job.

Queue

A data structure that lets us put and get items, usually in FIFO order: first in, first out. Queues allow separate execution units to exchange application data and control messages, such as error codes and signals to terminate. The implementation of a queue varies according to the underlying concurrency model: the queue

package in Python’s standard library provides queue classes to support threads, while the `multiprocessing` and `asyncio` packages implement their own queue classes. The `queue` and `asyncio` packages also include queues that are not FIFO: `LifoQueue` and `PriorityQueue`.

Lock

An object that execution units can use to synchronize their actions and avoid corrupting data. While updating a shared data structure, the running code should hold an associated lock. This signals other parts of the program to wait until the lock is released before accessing the same data structure. The simplest type of lock is also known as a mutex (for mutual exclusion). The implementation of a lock depends on the underlying concurrency model.

Contention

Dispute over a limited asset. Resource contention happens when multiple execution units try to access a shared resource—such as a lock or storage. There’s also CPU contention, when compute-intensive processes or threads must wait for the OS scheduler to give them a share of the CPU time.

Now let’s use some of that jargon to understand concurrency support in Python.

Processes, Threads, and Python’s Infamous GIL

Here is how the concepts we just saw apply to Python programming, in 10 points:

1. Each instance of the Python interpreter is a process. You can start additional Python processes using the *`multiprocessing`* or *`concurrent.futures`* libraries. Python’s *`subprocess`* library is designed to launch processes to run external programs, regardless of the languages used to write them.
2. The Python interpreter uses a single thread to run the user’s program and the memory garbage collector. You can start additional Python threads using the *`threading`* or *`concurrent.futures`* libraries.
3. Access to object reference counts and other internal interpreter state is controlled by a lock, the Global Interpreter Lock (GIL). Only one Python thread can hold the GIL at any time. This means that only one thread can execute Python code at any time, regardless of the number of CPU cores.
4. To prevent a Python thread from holding the GIL indefinitely, Python’s bytecode interpreter pauses the current Python thread every 5ms by default,⁴ releasing the GIL. The thread can then try to reacquire the GIL, but if there are other threads waiting for it, the OS scheduler may pick one of them to proceed.

⁴ Call `sys.getswitchinterval()` to get the interval; change it with `sys.setswitchinterval(s)`.

5. When we write Python code, we have no control over the GIL. But a built-in function or an extension written in C—or any language that interfaces at the Python/C API level—can release the GIL while running time-consuming tasks.
6. Every Python standard library function that makes a syscall⁵ releases the GIL. This includes all functions that perform disk I/O, network I/O, and `time.sleep()`. Many CPU-intensive functions in the NumPy/SciPy libraries, as well as the compressing/decompressing functions from the `zlib` and `bz2` modules, also release the GIL.⁶
7. Extensions that integrate at the Python/C API level can also launch other non-Python threads that are not affected by the GIL. Such GIL-free threads generally cannot change Python objects, but they can read from and write to the memory underlying objects that support the **buffer protocol**, such as `bytearray`, `array.array`, and *NumPy* arrays.
8. The effect of the GIL on network programming with Python threads is relatively small, because the I/O functions release the GIL, and reading or writing to the network always implies high latency—compared to reading and writing to memory. Consequently, each individual thread spends a lot of time waiting anyway, so their execution can be interleaved without major impact on the overall throughput. That’s why David Beazley says: “Python threads are great at doing nothing.”⁷
9. Contention over the GIL slows down compute-intensive Python threads. Sequential, single-threaded code is simpler and faster for such tasks.
10. To run CPU-intensive Python code on multiple cores, you must use multiple Python processes.

Here is a good summary from the `threading` module documentation:⁸

CPython implementation detail: In CPython, due to the Global Interpreter Lock, only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation). If you want your application to make better use of the computational resources of multicore machines, you are advised to use `multiprocessing` or `concurrent.futures.ProcessPoolExecutor`. However,

5 A syscall is a call from user code to a function of the operating system kernel. I/O, timers, and locks are some of the kernel services available through syscalls. To learn more, read the Wikipedia “**System call**” article.

6 The `zlib` and `bz2` modules are specifically mentioned in a **python-dev message** by Antoine Pitrou, who contributed the time-slicing GIL logic to Python 3.2.

7 Source: slide 106 of Beazley’s “**Generators: The Final Frontier**” tutorial.

8 Source: last paragraph of the “**Thread objects**” section.

threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.

The previous paragraph starts with “CPython implementation detail” because the GIL is not part of the Python language definition. The Jython and IronPython implementations don’t have a GIL. Unfortunately, both are lagging behind—still tracking Python 2.7. The highly performant **PyPy interpreter** also has a GIL in its 2.7 and 3.7 versions—the latest as of June 2021.



This section did not mention coroutines, because by default they share the same Python thread among themselves and with the supervising event loop provided by an asynchronous framework, therefore the GIL does not affect them. It is possible to use multiple threads in an asynchronous program, but the best practice is that one thread runs the event loop and all coroutines, while additional threads carry out specific tasks. This will be explained in “**Delegating Tasks to Executors**” on page 797.

Enough concepts for now. Let’s see some code.

A Concurrent Hello World

During a discussion about threads and how to avoid the GIL, Python contributor Michele Simionato **posted an example** that is like a concurrent “Hello World”: the simplest program to show how Python can “walk and chew gum.”

Simionato’s program uses multiprocessing, but I adapted it to introduce threading and `asyncio` as well. Let’s start with the threading version, which may look familiar if you’ve studied threads in Java or C.

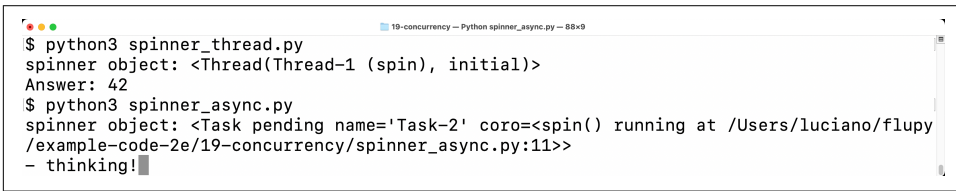
Spinner with Threads

The idea of the next few examples is simple: start a function that blocks for 3 seconds while animating characters in the terminal to let the user know that the program is “thinking” and not stalled.

The script makes an animated spinner displaying each character in the string “\|/ - ” in the same screen position.⁹ When the slow computation finishes, the line with the spinner is cleared and the result is shown: `Answer: 42`.

⁹ Unicode has lots of characters useful for simple animations, like the **Braille patterns** for example. I used the ASCII characters “\|/ - ” to keep the examples simple.

Figure 19-1 shows the output of two versions of the spinning example: first with threads, then with coroutines. If you're away from the computer, imagine the \ in the last line is spinning.



```
$ python3 spinner_thread.py
spinner object: <Thread(Thread-1 (spin), initial)>
Answer: 42
$ python3 spinner_async.py
spinner object: <Task pending name='Task-2' coro=<spin() running at /Users/luciano/flupy
/example-code-2e/19-concurrency/spinner_async.py:11>>
- thinking!
```

Figure 19-1. The scripts `spinner_thread.py` and `spinner_async.py` produce similar output: the repr of a spinner object and the text “Answer: 42”. In the screenshot, `spinner_async.py` is still running, and the animated message “/ thinking!” is shown; that line will be replaced by “Answer: 42” after 3 seconds.

Let’s review the `spinner_thread.py` script first. Example 19-1 lists the first two functions in the script, and Example 19-2 shows the rest.

Example 19-1. `spinner_thread.py`: the `spin` and `slow` functions

```
import itertools
import time
from threading import Thread, Event

def spin(msg: str, done: Event) -> None: ❶
    for char in itertools.cycle(r'\|/-'): ❷
        status = f'\r{char} {msg}' ❸
        print(status, end='', flush=True)
        if done.wait(.1): ❹
            break ❺
    blanks = ' ' * len(status)
    print(f'\r{blanks}\r', end='') ❻

def slow() -> int:
    time.sleep(3) ❼
    return 42
```

- ❶ This function will run in a separate thread. The `done` argument is an instance of `threading.Event`, a simple object to synchronize threads.
- ❷ This is an infinite loop because `itertools.cycle` yields one character at a time, cycling through the string forever.
- ❸ The trick for text-mode animation: move the cursor back to the start of the line with the carriage return ASCII control character (`'\r'`).

- ④ The `Event.wait(timeout=None)` method returns `True` when the event is set by another thread; if the `timeout` elapses, it returns `False`. The `.1s` `timeout` sets the “frame rate” of the animation to 10 FPS. If you want the spinner to go faster, use a smaller `timeout`.
- ⑤ Exit the infinite loop.
- ⑥ Clear the status line by overwriting with spaces and moving the cursor back to the beginning.
- ⑦ `slow()` will be called by the main thread. Imagine this is a slow API call over the network. Calling `sleep` blocks the main thread, but the GIL is released so the spinner thread can proceed.



The first important insight of this example is that `time.sleep()` blocks the calling thread but releases the GIL, allowing other Python threads to run.

The `spin` and `slow` functions will execute concurrently. The main thread—the only thread when the program starts—will start a new thread to run `spin` and then call `slow`. By design, there is no API for terminating a thread in Python. You must send it a message to shut down.

The `threading.Event` class is Python’s simplest signalling mechanism to coordinate threads. An `Event` instance has an internal boolean flag that starts as `False`. Calling `Event.set()` sets the flag to `True`. While the flag is false, if a thread calls `Event.wait()`, it is blocked until another thread calls `Event.set()`, at which time `Event.wait()` returns `True`. If a `timeout` in seconds is given to `Event.wait(s)`, this call returns `False` when the `timeout` elapses, or returns `True` as soon as `Event.set()` is called by another thread.

The `supervisor` function, listed in [Example 19-2](#), uses an `Event` to signal the `spin` function to exit.

Example 19-2. `spinner_thread.py`: the supervisor and main functions

```
def supervisor() -> int: ①
    done = Event() ②
    spinner = Thread(target=spin, args=('thinking!', done)) ③
    print(f'spinner object: {spinner}') ④
    spinner.start() ⑤
    result = slow() ⑥
    done.set() ⑦
```

```

    spinner.join() ❸
    return result

def main() -> None:
    result = supervisor() ❹
    print(f'Answer: {result}')

if __name__ == '__main__':
    main()

```

- ❶ supervisor will return the result of slow.
- ❷ The `threading.Event` instance is the key to coordinate the activities of the `main` thread and the `spinner` thread, as explained further down.
- ❸ To create a new `Thread`, provide a function as the `target` keyword argument, and positional arguments to the `target` as a tuple passed via `args`.
- ❹ Display the `spinner` object. The output is `<Thread(Thread-1, initial)>`, where `initial` is the state of the thread—meaning it has not started.
- ❺ Start the `spinner` thread.
- ❻ Call `slow`, which blocks the `main` thread. Meanwhile, the secondary thread is running the `spinner` animation.
- ❼ Set the `Event` flag to `True`; this will terminate the `for` loop inside the `spin` function.
- ❽ Wait until the `spinner` thread finishes.
- ❾ Run the `supervisor` function. I wrote separate `main` and `supervisor` functions to make this example look more like the `asyncio` version in [Example 19-4](#).

When the `main` thread sets the `done` event, the `spinner` thread will eventually notice and exit cleanly.

Now let's take a look at a similar example using the `multiprocessing` package.

Spinner with Processes

The `multiprocessing` package supports running concurrent tasks in separate Python processes instead of threads. When you create a `multiprocessing.Process` instance, a whole new Python interpreter is started as a child process in the background. Since each Python process has its own GIL, this allows your program to use all available

CPU cores—but that ultimately depends on the operating system scheduler. We’ll see practical effects in “A Homegrown Process Pool” on page 716, but for this simple program it makes no real difference.

The point of this section is to introduce multiprocessing and show that its API emulates the threading API, making it easy to convert simple programs from threads to processes, as shown in *spinner_proc.py* (Example 19-3).

Example 19-3. spinner_proc.py: only the changed parts are shown; everything else is the same as spinner_thread.py

```
import itertools
import time
from multiprocessing import Process, Event ❶
from multiprocessing import synchronize ❷

def spin(msg: str, done: synchronize.Event) -> None: ❸

# [snip] the rest of spin and slow functions are unchanged from spinner_thread.py

def supervisor() -> int:
    done = Event()
    spinner = Process(target=spin, ❹
                      args=('thinking!', done))
    print(f'spinner object: {spinner}') ❺
    spinner.start()
    result = slow()
    done.set()
    spinner.join()
    return result

# [snip] main function is unchanged as well
```

- ❶ The basic multiprocessing API imitates the threading API, but type hints and Mypy expose this difference: `multiprocessing.Event` is a function (not a class like `threading.Event`) which returns a `synchronize.Event` instance...
- ❷ ...forcing us to import `multiprocessing.synchronize`...
- ❸ ...to write this type hint.
- ❹ Basic usage of the `Process` class is similar to `Thread`.
- ❺ The `spinner` object is displayed as `<Process name='Process-1' parent=14868 initial>`, where 14868 is the process ID of the Python instance running *spinner_proc.py*.

The basic API of threading and multiprocessing are similar, but their implementation is very different, and multiprocessing has a much larger API to handle the added complexity of multiprocess programming. For example, one challenge when converting from threads to processes is how to communicate between processes that are isolated by the operating system and can't share Python objects. This means that objects crossing process boundaries have to be serialized and deserialized, which creates overhead. In [Example 19-3](#), the only data that crosses the process boundary is the Event state, which is implemented with a low-level OS semaphore in the C code underlying the multiprocessing module.¹⁰



Since Python 3.8, there's a `multiprocessing.shared_memory` package in the standard library, but it does not support instances of user-defined classes. Besides raw bytes, the package allows processes to share a `ShareableList`, a mutable sequence type that can hold a fixed number of items of types `int`, `float`, `bool`, and `None`, as well as `str` and `bytes` up to 10 MB per item. See the [ShareableList](#) documentation for more.

Now let's see how the same behavior can be achieved with coroutines instead of threads or processes.

Spinner with Coroutines



[Chapter 21](#) is entirely devoted to asynchronous programming with coroutines. This is just a high-level introduction to contrast this approach with the threads and processes concurrency models. As such, we will overlook many details.

It is the job of OS schedulers to allocate CPU time to drive threads and processes. In contrast, coroutines are driven by an application-level event loop that manages a queue of pending coroutines, drives them one by one, monitors events triggered by I/O operations initiated by coroutines, and passes control back to the corresponding coroutine when each event happens. The event loop and the library coroutines and the user coroutines all execute in a single thread. Therefore, any time spent in a coroutine slows down the event loop—and all other coroutines.

¹⁰ The semaphore is a fundamental building block that can be used to implement other synchronization mechanisms. Python provides different semaphore classes for use with threads, processes, and coroutines. We'll see `asyncio.Semaphore` in [“Using `asyncio.as\_completed` and a Thread” on page 788 \(Chapter 21\)](#).

The coroutine version of the spinner program is easier to understand if we start from the main function, then study the supervisor. That's what [Example 19-4](#) shows.

Example 19-4. spinner_async.py: the main function and supervisor coroutine

```
def main() -> None: ❶
    result = asyncio.run(supervisor()) ❷
    print(f'Answer: {result}')

async def supervisor() -> int: ❸
    spinner = asyncio.create_task(spin('thinking!')) ❹
    print(f'spinner object: {spinner}') ❺
    result = await slow() ❻
    spinner.cancel() ❼
    return result

if __name__ == '__main__':
    main()
```

- ❶ main is the only regular function defined in this program—the others are coroutines.
- ❷ The `asyncio.run` function starts the event loop to drive the coroutine that will eventually set the other coroutines in motion. The main function will stay blocked until supervisor returns. The return value of supervisor will be the return value of `asyncio.run`.
- ❸ Native coroutines are defined with `async def`.
- ❹ `asyncio.create_task` schedules the eventual execution of `spin`, immediately returning an instance of `asyncio.Task`.
- ❺ The repr of the spinner object looks like `<Task pending name='Task-2' coro=<spin() running at /path/to/spinner_async.py:11>>`.
- ❻ The `await` keyword calls `slow`, blocking supervisor until `slow` returns. The return value of `slow` will be assigned to `result`.
- ❼ The `Task.cancel` method raises a `CancelledError` exception inside the `spin` coroutine, as we'll see in [Example 19-5](#).

[Example 19-4](#) demonstrates the three main ways of running a coroutine:

`asyncio.run(coro())`

Called from a regular function to drive a coroutine object that usually is the entry point for all the asynchronous code in the program, like the supervisor in this example. This call blocks until the body of `coro` returns. The return value of the `run()` call is whatever the body of `coro` returns.

`asyncio.create_task(coro())`

Called from a coroutine to schedule another coroutine to execute eventually. This call does not suspend the current coroutine. It returns a `Task` instance, an object that wraps the coroutine object and provides methods to control and query its state.

`await coro()`

Called from a coroutine to transfer control to the coroutine object returned by `coro()`. This suspends the current coroutine until the body of `coro` returns. The value of the `await` expression is whatever the body of `coro` returns.



Remember: invoking a coroutine as `coro()` immediately returns a coroutine object, but does not run the body of the `coro` function. Driving the body of coroutines is the job of the event loop.

Now let's study the `spin` and `slow` coroutines in [Example 19-5](#).

Example 19-5. `spinner_async.py`: the `spin` and `slow` coroutines

```
import asyncio
import itertools

async def spin(msg: str) -> None: ❶
    for char in itertools.cycle(r'\|/-'):
        status = f'\r{char} {msg}'
        print(status, flush=True, end='')
        try:
            await asyncio.sleep(.1) ❷
        except asyncio.CancelledError: ❸
            break
    blanks = ' ' * len(status)
    print(f'\r{blanks}\r', end='')

async def slow() -> int:
    await asyncio.sleep(3) ❹
    return 42
```

- ❶ We don't need the `Event` argument that was used to signal that `slow` had completed its job in `spinner_thread.py` (Example 19-1).
- ❷ Use `await asyncio.sleep(.1)` instead of `time.sleep(.1)`, to pause without blocking other coroutines. See the experiment after this example.
- ❸ `asyncio.CancelledError` is raised when the `cancel` method is called on the `Task` controlling this coroutine. Time to exit the loop.
- ❹ The `slow` coroutine also uses `await asyncio.sleep` instead of `time.sleep`.

Experiment: Break the spinner for an insight

Here is an experiment I recommend to understand how `spinner_async.py` works. Import the `time` module, then go to the `slow` coroutine and replace the line `await asyncio.sleep(3)` with a call to `time.sleep(3)`, like in Example 19-6.

Example 19-6. `spinner_async.py`: replacing `await asyncio.sleep(3)` with `time.sleep(3)`

```
async def slow() -> int:
    time.sleep(3)
    return 42
```

Watching the behavior is more memorable than reading about it. Go ahead, I'll wait.

When you run the experiment, this is what you see:

1. The spinner object is shown, similar to this: `<Task pending name='Task-2' coro=<spin() running at /path/to/spinner_async.py:12>>`.
2. The spinner never appears. The program hangs for 3 seconds.
3. Answer: 42 is displayed and the program ends.

To understand what is happening, recall that Python code using `asyncio` has only one flow of execution, unless you've explicitly started additional threads or processes. That means only one coroutine executes at any point in time. Concurrency is achieved by control passing from one coroutine to another. In Example 19-7, let's focus on what happens in the supervisor and `slow` coroutines during the proposed experiment.

Example 19-7. `spinner_async_experiment.py`: the supervisor and `slow` coroutines

```
async def slow() -> int:
    time.sleep(3) ❹
```

```
return 42
```

```
async def supervisor() -> int:
    spinner = asyncio.create_task(spin('thinking!')) ❶
    print(f'spinner object: {spinner}') ❷
    result = await slow() ❸
    spinner.cancel() ❺
    return result
```

- ❶ The `spinner` task is created, to eventually drive the execution of `spin`.
- ❷ The display shows the Task is “pending.”
- ❸ The `await` expression transfers control to the `slow` coroutine.
- ❹ `time.sleep(3)` blocks for 3 seconds; nothing else can happen in the program, because the main thread is blocked—and it is the only thread. The operating system will continue with other activities. After 3 seconds, `sleep` unblocks, and `slow` returns.
- ❺ Right after `slow` returns, the `spinner` task is cancelled. The flow of control never reached the body of the `spin` coroutine.

The *spinner_async_experiment.py* teaches an important lesson, as explained in the following warning.



Never use `time.sleep(...)` in `asyncio` coroutines unless you want to pause your whole program. If a coroutine needs to spend some time doing nothing, it should await `asyncio.sleep(DELAY)`. This yields control back to the `asyncio` event loop, which can drive other pending coroutines.

Greenlet and gevent

As we discuss concurrency with coroutines, it’s important to mention the *greenlet* package, which has been around for many years and is used at scale.¹¹ The package supports cooperative multitasking through lightweight coroutines—named *greenlets*—that don’t require any special syntax such as `yield` or `await`, therefore are easier to integrate into existing, sequential codebases. *SQLAlchemy 1.4 ORM* uses *greenlets* internally to implement its new *asynchronous API* compatible with *asyncio*.

¹¹ Thanks to tech reviewers Caleb Hattingh and Jürgen Gmach who did not let me overlook *greenlet* and *gevent*.

The *gevent* networking library monkey patches Python’s standard socket module making it nonblocking by replacing some of its code with greenlets. To a large extent, *gevent* is transparent to the surrounding code, making it easier to adapt sequential applications and libraries—such as database drivers—to perform concurrent network I/O. Numerous open source projects use *gevent*, including the widely deployed *Gunicorn*—mentioned in “WSGI Application Servers” on page 730.

Supervisors Side-by-Side

The line count of *spinner_thread.py* and *spinner_async.py* is nearly the same. The supervisor functions are the heart of these examples. Let’s compare them in detail.

Example 19-8 lists only the supervisor from Example 19-2.

Example 19-8. spinner_thread.py: the threaded supervisor function

```
def supervisor() -> int:
    done = Event()
    spinner = Thread(target=spin,
                     args=('thinking!', done))
    print('spinner object:', spinner)
    spinner.start()
    result = slow()
    done.set()
    spinner.join()
    return result
```

For comparison, Example 19-9 shows the supervisor coroutine from Example 19-4.

Example 19-9. spinner_async.py: the asynchronous supervisor coroutine

```
async def supervisor() -> int:
    spinner = asyncio.create_task(spin('thinking!'))
    print('spinner object:', spinner)
    result = await slow()
    spinner.cancel()
    return result
```

Here is a summary of the differences and similarities to note between the two supervisor implementations:

- An `asyncio.Task` is roughly the equivalent of a `threading.Thread`.
- A `Task` drives a coroutine object, and a `Thread` invokes a callable.
- A coroutine yields control explicitly with the `await` keyword.

- You don't instantiate Task objects yourself, you get them by passing a coroutine to `asyncio.create_task(...)`.
- When `asyncio.create_task(...)` returns a Task object, it is already scheduled to run, but a Thread instance must be explicitly told to run by calling its `start` method.
- In the threaded supervisor, `slow` is a plain function and is directly invoked by the main thread. In the asynchronous supervisor, `slow` is a coroutine driven by `await`.
- There's no API to terminate a thread from the outside; instead, you must send a signal—like setting the `done` Event object. For tasks, there is the `Task.cancel()` instance method, which raises `CancelledError` at the `await` expression where the coroutine body is currently suspended.
- The supervisor coroutine must be started with `asyncio.run` in the main function.

This comparison should help you understand how concurrent jobs are orchestrated with *asyncio*, in contrast to how it's done with the `Threading` module, which may be more familiar to you.

One final point related to threads versus coroutines: if you've done any nontrivial programming with threads, you know how challenging it is to reason about the program because the scheduler can interrupt a thread at any time. You must remember to hold locks to protect the critical sections of your program, to avoid getting interrupted in the middle of a multistep operation—which could leave data in an invalid state.

With coroutines, your code is protected against interruption by default. You must explicitly `await` to let the rest of the program run. Instead of holding locks to synchronize the operations of multiple threads, coroutines are “synchronized” by definition: only one of them is running at any time. When you want to give up control, you use `await` to yield control back to the scheduler. That's why it is possible to safely cancel a coroutine: by definition, a coroutine can only be cancelled when it's suspended at an `await` expression, so you can perform cleanup by handling the `CancelledError` exception.

The `time.sleep()` call blocks but does nothing. Now we'll experiment with a CPU-intensive call to get a better understanding of the GIL, as well as the effect of CPU-intensive functions in asynchronous code.

The Real Impact of the GIL

In the threading code (Example 19-1), you can replace the `time.sleep(3)` call in the `slow` function with an HTTP client request from your favorite library, and the spinner will keep spinning. That's because a well-designed network library will release the GIL while waiting for the network.

You can also replace the `asyncio.sleep(3)` expression in the `slow` coroutine to `await` for a response from a well-designed asynchronous network library, because such libraries provide coroutines that yield control back to the event loop while waiting for the network. Meanwhile, the spinner will keep spinning.

With CPU-intensive code, the story is different. Consider the function `is_prime` in Example 19-10, which returns `True` if the argument is a prime number, `False` if it's not.

Example 19-10. `primes.py`: an easy to read primality check, from Python's `ProcessPoolExecutor` example

```
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    root = math.isqrt(n)
    for i in range(3, root + 1, 2):
        if n % i == 0:
            return False
    return True
```

The call `is_prime(5_000_111_000_222_021)` takes about 3.3s on the company laptop I am using now.¹²

Quick Quiz

Given what we've seen so far, please take the time to consider the following three-part question. One part of the answer is tricky (at least it was for me).

What would happen to the spinner animation if you made the following changes, assuming that `n = 5_000_111_000_222_021`—that prime which my machine takes 3.3s to verify:

¹² It's a 15" MacBook Pro 2018 with a 6-core, 2.2 GHz Intel Core i7 CPU.

1. In *spinner_proc.py*, replace `time.sleep(3)` with a call to `is_prime(n)`?
2. In *spinner_thread.py*, replace `time.sleep(3)` with a call to `is_prime(n)`?
3. In *spinner_async.py*, replace `await asyncio.sleep(3)` with a call to `is_prime(n)`?

Before you run the code or read on, I recommend figuring out the answers on your own. Then, you may want to copy and modify the *spinner_*.py* examples as suggested.

Now the answers, from easier to hardest.

1. Answer for multiprocessing

The spinner is controlled by a child process, so it continues spinning while the primality test is computed by the parent process.¹³

2. Answer for threading

The spinner is controlled by a secondary thread, so it continues spinning while the primality test is computed by the main thread.

I did not get this answer right at first: I was expecting the spinner to freeze because I overestimated the impact of the GIL.

In this particular example, the spinner keeps spinning because Python suspends the running thread every 5ms (by default), making the GIL available to other pending threads. Therefore, the main thread running `is_prime` is interrupted every 5ms, allowing the secondary thread to wake up and iterate once through the for loop, until it calls the `wait` method of the done event, at which time it will release the GIL. The main thread will then grab the GIL, and the `is_prime` computation will proceed for another 5ms.

This does not have a visible impact on the running time of this specific example, because the `spin` function quickly iterates once and releases the GIL as it waits for the done event, so there is not much contention for the GIL. The main thread running `is_prime` will have the GIL most of the time.

We got away with a compute-intensive task using threading in this simple experiment because there are only two threads: one hogging the CPU, and the other waking up only 10 times per second to update the spinner.

¹³ This is true today because you are probably using a modern OS with *preemptive multitasking*. Windows before the NT era and macOS before the OSX era were not “preemptive,” therefore any process could take over 100% of the CPU and freeze the whole system. We are not completely free of this kind of problem today but trust this graybeard: this troubled every user in the 1990s, and a hard reset was the only cure.

But if you have two or more threads vying for a lot of CPU time, your program will be slower than sequential code.

3. Answer for asyncio

If you call `is_prime(5_000_111_000_222_021)` in the slow coroutine of the *spinner_async.py* example, the spinner will never appear. The effect would be the same we had in [Example 19-6](#), when we replaced `await asyncio.sleep(3)` with `time.sleep(3)`: no spinning at all. The flow of control will pass from supervisor to slow, and then to `is_prime`. When `is_prime` returns, slow returns as well, and supervisor resumes, cancelling the spinner task before it is executed even once. The program appears frozen for about 3s, then shows the answer.

Power Napping with `sleep(0)`

One way to keep the spinner alive is to rewrite `is_prime` as a coroutine, and periodically call `asyncio.sleep(0)` in an `await` expression to yield control back to the event loop, like in [Example 19-11](#).

Example 19-11. `spinner_async_nap.py`: `is_prime` is now a coroutine

```
async def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    root = math.isqrt(n)
    for i in range(3, root + 1, 2):
        if n % i == 0:
            return False
        if i % 100_000 == 1:
            await asyncio.sleep(0) ❶
    return True
```

❶ Sleep once every 50,000 iterations (because the step in the range is 2).

[Issue #284](#) in the asyncio repository has an informative discussion about the use of `asyncio.sleep(0)`.

However, be aware this will slow down `is_prime`, and—more importantly—will still slow down the event loop and your whole program with it. When I used `await asyncio.sleep(0)` every 100,000 iterations, the spinner was smooth but the program ran in 4.9s on my machine, almost 50% longer than the original `primes.is_prime` function by itself with the same argument (5_000_111_000_222_021).

Using `await asyncio.sleep(0)` should be considered a stopgap measure before you refactor your asynchronous code to delegate CPU-intensive computations to another process. We'll see one way of doing that with `asyncio.loop.run_in_executor`, covered in [Chapter 21](#). Another option would be a task queue, which we'll briefly discuss in [“Distributed Task Queues” on page 732](#).

So far, we've only experimented with a single call to a CPU-intensive function. The next section presents concurrent execution of multiple CPU-intensive calls.

A Homegrown Process Pool



I wrote this section to show the use of multiple processes for CPU-intensive tasks, and the common pattern of using queues to distribute tasks and collect results. [Chapter 20](#) will show a simpler way of distributing tasks to processes: a `ProcessPoolExecutor` from the `concurrent.futures` package, which uses queues internally.

In this section we'll write programs to compute the primality of a sample of 20 integers, from 2 to 9,999,999,999,999,999—i.e., $10^{16} - 1$, or more than 2^{53} . The sample includes small and large primes, as well as composite numbers with small and large prime factors.

The *sequential.py* program provides the performance baseline. Here is a sample run:

```
$ python3 sequential.py
      2 P 0.000001s
142702110479723 P 0.568328s
299593572317531 P 0.796773s
3333333333333301 P 2.648625s
333333333333333 0.000007s
3333335652092209 2.672323s
444444444444423 P 3.052667s
4444444444444444 0.000001s
4444444488888889 3.061083s
5555553133149889 3.451833s
555555555555503 P 3.556867s
555555555555555 0.000007s
666666666666666 0.000001s
6666666666666719 P 3.781064s
6666667141414921 3.778166s
777777536340681 4.120069s
777777777777753 P 4.141530s
777777777777777 0.000007s
999999999999917 P 4.678164s
999999999999999 0.000007s
Total time: 40.31
```

The results are shown in three columns:

- The number to be checked.
- P if it's a prime number, blank if not.
- Elapsed time for checking the primality for that specific number.

In this example, the total time is approximately the sum of the times for each check, but it is computed separately, as you can see in [Example 19-12](#).

Example 19-12. sequential.py: sequential primality check for a small dataset

```
#!/usr/bin/env python3

"""
sequential.py: baseline for comparing sequential, multiprocessing,
and threading code for CPU-intensive work.
"""

from time import perf_counter
from typing import NamedTuple

from primes import is_prime, NUMBERS

class Result(NamedTuple): ❶
    prime: bool
    elapsed: float

def check(n: int) -> Result: ❷
    t0 = perf_counter()
    prime = is_prime(n)
    return Result(prime, perf_counter() - t0)

def main() -> None:
    print(f'Checking {len(NUMBERS)} numbers sequentially:')
    t0 = perf_counter()
    for n in NUMBERS: ❸
        prime, elapsed = check(n)
        label = 'P' if prime else ' '
        print(f'{n:16} {label} {elapsed:9.6f}s')

    elapsed = perf_counter() - t0 ❹
    print(f'Total time: {elapsed:.2f}s')

if __name__ == '__main__':
    main()
```

- ❶ The `check` function (in the next callout) returns a `Result` tuple with the boolean value of the `is_prime` call and the elapsed time.
- ❷ `check(n)` calls `is_prime(n)` and computes the elapsed time to return a `Result`.
- ❸ For each number in the sample, we call `check` and display the result.
- ❹ Compute and display the total elapsed time.

Process-Based Solution

The next example, *procs.py*, shows the use of multiple processes to distribute the primality checks across multiple CPU cores. These are the times I get with *procs.py*:

```
$ python3 procs.py
Checking 20 numbers with 12 processes:
 2 P 0.000002s
3333333333333333 0.000021s
4444444444444444 0.000002s
5555555555555555 0.000018s
6666666666666666 0.000002s
142702110479723 P 1.350982s
7777777777777777 0.000009s
299593572317531 P 1.981411s
9999999999999999 0.000008s
3333333333333301 P 6.328173s
3333335652092209 6.419249s
4444444488888889 7.051267s
4444444444444423 P 7.122004s
5555553133149889 7.412735s
5555555555555503 P 7.603327s
6666666666666719 P 7.934670s
6666667141414921 8.017599s
777777536340681 8.339623s
7777777777777753 P 8.38859s
9999999999999917 P 8.117313s
20 checks in 9.58s
```

The last line of the output shows that *procs.py* was 4.2 times faster than *sequential.py*.

Understanding the Elapsed Times

Note that the elapsed time in the first column is for checking that specific number. For example, `is_prime(7777777777777753)` took almost 8.4s to return `True`. Meanwhile, other processes were checking other numbers in parallel.

There were 20 numbers to check. I wrote *procs.py* to start a number of worker processes equal to the number of CPU cores, as determined by `multiprocessing.cpu_count()`.

The total time in this case is much less than the sum of the elapsed time for the individual checks. There is some overhead in spinning up processes and in inter-process communication, so the end result is that the multiprocessing version is only about 4.2 times faster than the sequential. That’s good, but a little disappointing considering the code launches 12 processes to use all cores on this laptop.



The `multiprocessing.cpu_count()` function returns 12 on the MacBook Pro I’m using to write this chapter. It’s actually a 6-CPU Core-i7, but the OS reports 12 CPUs because of hyperthreading, an Intel technology which executes 2 threads per core. However, hyperthreading works better when one of the threads is not working as hard as the other thread in the same core—perhaps the first is stalled waiting for data after a cache miss, and the other is crunching numbers. Anyway, there’s no free lunch: this laptop performs like a 6-CPU machine for compute-intensive work that doesn’t use a lot of memory—like that simple primality test.

Code for the Multicore Prime Checker

When we delegate computing to threads or processes, our code does not call the worker function directly, so we can’t simply get a return value. Instead, the worker is driven by the thread or process library, and it eventually produces a result that needs to be stored somewhere. Coordinating workers and collecting results are common uses of queues in concurrent programming—and also in distributed systems.

Much of the new code in *procs.py* has to do with setting up and using queues. The top of the file is in [Example 19-13](#).



`SimpleQueue` was added to `multiprocessing` in Python 3.9. If you’re using an earlier version of Python, you can replace `SimpleQueue` with `Queue` in [Example 19-13](#).

Example 19-13. procs.py: multiprocessing primality check; imports, types, and functions

```
import sys
from time import perf_counter
from typing import NamedTuple
from multiprocessing import Process, SimpleQueue, cpu_count ❶
from multiprocessing import queues ❷

from primes import is_prime, NUMBERS

class PrimeResult(NamedTuple): ❸
    n: int
```

```

    prime: bool
    elapsed: float

JobQueue = queues.SimpleQueue[int] ❷
ResultQueue = queues.SimpleQueue[PrimeResult] ❸

def check(n: int) -> PrimeResult: ❹
    t0 = perf_counter()
    res = is_prime(n)
    return PrimeResult(n, res, perf_counter() - t0)

def worker(jobs: JobQueue, results: ResultQueue) -> None: ❺
    while n := jobs.get(): ❻
        results.put(check(n)) ❼
        results.put(PrimeResult(0, False, 0.0)) ❽

def start_jobs(
    procs: int, jobs: JobQueue, results: ResultQueue ❾
) -> None:
    for n in NUMBERS:
        jobs.put(n) ❿
    for _ in range(procs):
        proc = Process(target=worker, args=(jobs, results)) ⓫
        proc.start() ⓬
        jobs.put(0) ⓭

```

- ❶ Trying to emulate threading, multiprocessing provides multiprocessing.SimpleQueue, but this is a method bound to a predefined instance of a lower-level BaseContext class. We must call this SimpleQueue to build a queue, we can't use it in type hints.
- ❷ multiprocessing.queues has the SimpleQueue class we need for type hints.
- ❸ PrimeResult includes the number checked for primality. Keeping n together with the other result fields simplifies displaying results later.
- ❹ This is a type alias for a SimpleQueue that the main function (Example 19-14) will use to send numbers to the processes that will do the work.
- ❺ Type alias for a second SimpleQueue that will collect the results in main. The values in the queue will be tuples made of the number to be tested for primality, and a Result tuple.
- ❻ This is similar to *sequential.py*.
- ❼ worker gets a queue with the numbers to be checked, and another to put results.

- ❸ In this code, I use the number 0 as a *poison pill*: a signal for the worker to finish. If `n` is not 0, proceed with the loop.¹⁴
- ❹ Invoke the primality check and enqueue `PrimeResult`.
- ❺ Send back a `PrimeResult(0, False, 0.0)` to let the main loop know that this worker is done.
- ❻ `procs` is the number of processes that will compute the prime checks in parallel.
- ❼ Enqueue the numbers to be checked in `jobs`.
- ❽ Fork a child process for each worker. Each child will run the loop inside its own instance of the `worker` function, until it fetches a 0 from the `jobs` queue.
- ❾ Start each child process.
- ❿ Enqueue one 0 for each process, to terminate them.

Loops, Sentinels, and Poison Pills

The worker function in [Example 19-13](#) follows a common pattern in concurrent programming: looping indefinitely while taking items from a queue and processing each with a function that does the actual work. The loop ends when the queue produces a sentinel value. In this pattern, the sentinel that shuts down the worker is often called a “poison pill.”

`None` is often used as a sentinel value, but it may be unsuitable if it can occur in the data stream. Calling `object()` is a common way to get a unique value to use as sentinel. However, that does not work across processes because Python objects must be serialized for inter-process communication, and when you `pickle.dump` and `pickle.load` an instance of `object`, the unpickled instance is distinct from the original: it doesn’t compare equal. A good alternative to `None` is the `Ellipsis` built-in object (a.k.a. `...`), which survives serialization without losing its identity.¹⁵

Python’s standard library uses [lots of different values](#) as sentinels. [PEP 661—Sentinel Values](#) proposes a standard sentinel type. As of September 2021, it’s only a draft.

¹⁴ In this example, 0 is a convenient sentinel. `None` is also commonly used for that. Using 0 simplifies the type hint for `PrimeResult` and the code for `worker`.

¹⁵ Surviving serialization without losing our identity is a pretty good life goal.

Now let's study the main function of *procs.py* in [Example 19-14](#).

Example 19-14. procs.py: multiprocessing primality check; main function

```
def main() -> None:
    if len(sys.argv) < 2: ❶
        procs = cpu_count()
    else:
        procs = int(sys.argv[1])

    print(f'Checking {len(NUMBERS)} numbers with {procs} processes:')
    t0 = perf_counter()
    jobs: JobQueue = SimpleQueue() ❷
    results: ResultQueue = SimpleQueue()
    start_jobs(procs, jobs, results) ❸
    checked = report(procs, results) ❹
    elapsed = perf_counter() - t0
    print(f'{checked} checks in {elapsed:.2f}s') ❺

def report(procs: int, results: ResultQueue) -> int: ❻
    checked = 0
    procs_done = 0
    while procs_done < procs: ❼
        n, prime, elapsed = results.get() ❽
        if n == 0: ❾
            procs_done += 1
        else:
            checked += 1 ❿
            label = 'P' if prime else ' '
            print(f'{n:16} {label} {elapsed:9.6f}s')
    return checked

if __name__ == '__main__':
    main()
```

- ❶ If no command-line argument is given, set the number of processes to the number of CPU cores; otherwise, create as many processes as given in the first argument.
- ❷ jobs and results are the queues described in [Example 19-13](#).
- ❸ Start proc processes to consume jobs and post results.
- ❹ Retrieve the results and display them; report is defined in ❻.
- ❺ Display how many numbers were checked and the total elapsed time.
- ❻ The arguments are the number of procs and the queue to post the results.

- ⑦ Loop until all processes are done.
- ⑧ Get one `PrimeResult`. Calling `.get()` on a queue block until there is an item in the queue. It's also possible to make this nonblocking, or set a timeout. See the [SimpleQueue.get](#) documentation for details.
- ⑨ If `n` is zero, then one process exited; increment the `procs_done` count.
- ⑩ Otherwise, increment the checked count (to keep track of the numbers checked) and display the results.

The results will not come back in the same order the jobs were submitted. That's why I had to put `n` in each `PrimeResult` tuple. Otherwise, I'd have no way to know which result belonged to each number.

If the main process exits before all subprocesses are done, you may see confusing tracebacks on `FileNotFoundError` exceptions caused by an internal lock in `multi` processing. Debugging concurrent code is always hard, and debugging `multiproc` essing is even harder because of all the complexity behind the thread-like façade. Fortunately, the `ProcessPoolExecutor` we'll meet in [Chapter 20](#) is easier to use and more robust.



Thanks to reader Michael Albert who noticed the code I published during the early release had a *race condition* in [Example 19-14](#). A race condition is a bug that may or may not occur depending on the order of actions performed by concurrent execution units. If “A” happens before “B,” all is fine; but if “B” happens first, something goes wrong. That's the race.

If you are curious, this [diff](#) shows the bug and how I fixed it: [example-code-2e/commit/2c123057](#)—but note that I later refactored the example to delegate parts of `main` to the `start_jobs` and `report` functions. There's a [README.md](#) file in the same directory explaining the problem and the solution.

Experimenting with More or Fewer Processes

You may want try running `procs.py`, passing arguments to set the number of worker processes. For example, this command...

```
$ python3 procs.py 2
```

...will launch two worker processes, producing results almost twice as fast as *sequential.py*—if your machine has at least two cores and is not too busy running other programs.

I ran *procs.py* 12 times with 1 to 20 processes, totaling 240 runs. Then I computed the median time for all runs with the same number of processes, and plotted [Figure 19-2](#).

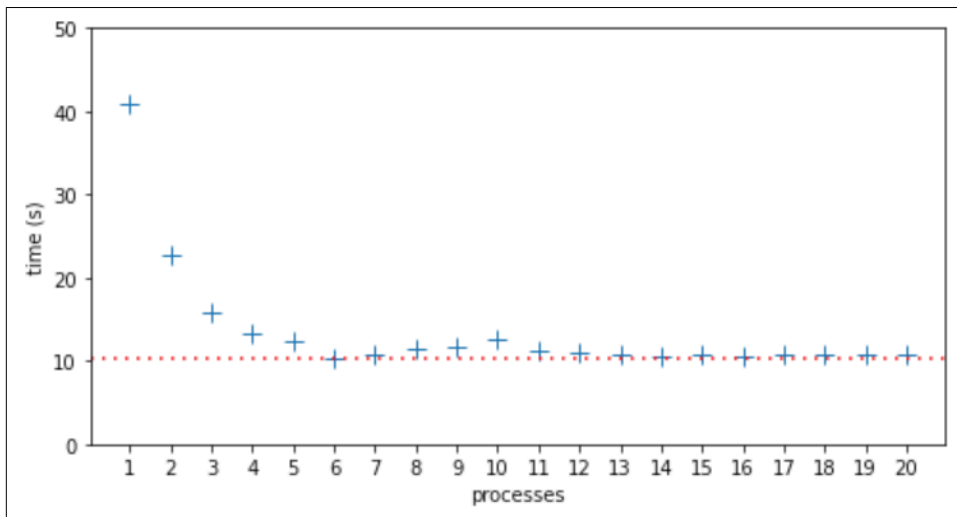


Figure 19-2. Median run times for each number of processes from 1 to 20. Highest median time was 40.81s, with 1 process. Lowest median time was 10.39s, with 6 processes, indicated by the dotted line.

In this 6-core laptop, the lowest median time was with 6 processes: 10.39s—marked by the dotted line in [Figure 19-2](#). I expected the run time to increase after 6 processes due to CPU contention, and it reached a local maximum of 12.51s at 10 processes. I did not expect and I can’t explain why the performance improved at 11 processes and remained almost flat from 13 to 20 processes, with median times only slightly higher than the lowest median time at 6 processes.

Thread-Based Nonsolution

I also wrote *threads.py*, a version of *procs.py* using threading instead of multiprocessing. The code is very similar—as is usually the case when converting simple examples between these two APIs.¹⁶ Due to the GIL and the compute-intensive nature of *is_prime*, the threaded version is slower than the sequential code in [Example 19-12](#), and it gets slower as the number of threads increase, because of CPU contention and the cost of context switching. To switch to a new thread, the OS needs to save CPU registers and update the program counter and stack pointer,

¹⁶ See [19-concurrency/primes/threads.py](#) in the *Fluent Python* code repository.

triggering expensive side effects like invalidating CPU caches and possibly even swapping memory pages.¹⁷

The next two chapters will cover more about concurrent programming in Python, using the high-level *concurrent.futures* library to manage threads and processes ([Chapter 20](#)) and the *asyncio* library for asynchronous programming ([Chapter 21](#)).

The remaining sections in this chapter aim to answer the question:

Given the limitations discussed so far, how is Python thriving in a multicore world?

Python in the Multicore World

Consider this citation from the widely quoted article “[The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software](#)” by Herb Sutter:

The major processor manufacturers and architectures, from Intel and AMD to Sparc and PowerPC, have run out of room with most of their traditional approaches to boosting CPU performance. Instead of driving clock speeds and straight-line instruction throughput ever higher, they are instead turning en masse to hyper-threading and multicore architectures. March 2005. [Available online].

What Sutter calls the “free lunch” was the trend of software getting faster with no additional developer effort because CPUs were executing sequential code faster, year after year. Since 2004, that is no longer true: clock speeds and execution optimizations reached a plateau, and now any significant increase in performance must come from leveraging multiple cores or hyperthreading, advances that only benefit code that is written for concurrent execution.

Python’s story started in the early 1990s, when CPUs were still getting exponentially faster at sequential code execution. There was no talk about multicore CPUs except in supercomputers back then. At the time, the decision to have a GIL was a no-brainer. The GIL makes the interpreter faster when running on a single core, and its implementation simpler.¹⁸ The GIL also makes it easier to write simple extensions through the Python/C API.

¹⁷ To learn more, see “[Context switch](#)” in the English Wikipedia.

¹⁸ These are probably the same reasons that prompted the creator of the Ruby language, Yukihiro Matsumoto, to use a GIL in his interpreter as well.



I just wrote “simple extensions” because an extension does not need to deal with the GIL at all. A function written in C or Fortran may be hundreds of times faster than the equivalent in Python.¹⁹ Therefore the added complexity of releasing the GIL to leverage multicore CPUs may not be needed in many cases. So we can thank the GIL for many extensions available for Python—and that is certainly one of the key reasons why the language is so popular today.

Despite the GIL, Python is thriving in applications that require concurrent or parallel execution, thanks to libraries and software architectures that work around the limitations of CPython.

Now let’s discuss how Python is used in system administration, data science, and server-side application development in the multicore, distributed computing world of 2021.

System Administration

Python is widely used to manage large fleets of servers, routers, load balancers, and network-attached storage (NAS). It’s also a leading option in software-defined networking (SDN) and ethical hacking. Major cloud service providers support Python through libraries and tutorials authored by the providers themselves or by their large communities of Python users.

In this domain, Python scripts automate configuration tasks by issuing commands to be carried out by the remote machines, so rarely there are CPU-bound operations to be done. Threads or coroutines are well suited for such jobs. In particular, the concurrent.futures package we’ll see in [Chapter 20](#) can be used to perform the same operations on many remote machines at the same time without a lot of complexity.

Beyond the standard library, there are popular Python-based projects to manage server clusters: tools like *Ansible* and *Salt*, as well as libraries like *Fabric*.

There is also a growing number of libraries for system administration supporting coroutines and `asyncio`. In 2016, Facebook’s [Production Engineering team reported](#): “We are increasingly relying on `AsyncIO`, which was introduced in Python 3.4, and seeing huge performance gains as we move codebases away from Python 2.”

¹⁹ As an exercise in college, I had to implement the LZW compression algorithm in C. But first I wrote it in Python, to check my understanding of the spec. The C version was about 900× faster.

Data Science

Data science—including artificial intelligence—and scientific computing are very well served by Python. Applications in these fields are compute-intensive, but Python users benefit from a vast ecosystem of numeric computing libraries written in C, C++, Fortran, Cython, etc.—many of which are able to leverage multicore machines, GPUs, and/or distributed parallel computing in heterogeneous clusters.

As of 2021, Python’s data science ecosystem includes impressive tools such as:

Project Jupyter

Two browser-based interfaces—Jupyter Notebook and JupyterLab—that allow users to run and document analytics code potentially running across the network on remote machines. Both are hybrid Python/JavaScript applications, supporting computing kernels written in different languages, all integrated via ZeroMQ—an asynchronous messaging library for distributed applications. The name *Jupyter* actually comes from Julia, Python, and R, the first three languages supported by the Notebook. The rich ecosystem built on top of the Jupyter tools include **Bokeh**, a powerful interactive visualization library that lets users navigate and interact with large datasets or continuously updated streaming data, thanks to the performance of modern JavaScript engines and browsers.

TensorFlow and PyTorch

These are the top two deep learning frameworks, according to **O’Reilly’s January 2021 report** on usage of their learning resources during 2020. Both projects are written in C++, and are able to leverage multiple cores, GPUs, and clusters. They support other languages as well, but Python is their main focus and is used by the majority of their users. TensorFlow was created and is used internally by Google; PyTorch by Facebook.

Dask

A parallel computing library that can farm out work to local processes or clusters of machines, “tested on some of the largest supercomputers in the world”—as their **home page** states. Dask offers APIs that closely emulate NumPy, pandas, and scikit-learn—the most popular libraries in data science and machine learning today. Dask can be used from JupyterLab or Jupyter Notebook, and leverages Bokeh not only for data visualization but also for an interactive dashboard showing the flow of data and computations across the processes/machines in near real time. Dask is so impressive that I recommend watching a video such as this **15-minute demo** in which Matthew Rocklin—a maintainer of the project—shows Dask crunching data on 64 cores distributed in 8 EC2 machines on AWS.

These are only some examples to illustrate how the data science community is creating solutions that leverage the best of Python and overcome the limitations of the CPython runtime.

Server-Side Web/Mobile Development

Python is widely used in web applications and for the backend APIs supporting mobile applications. How is it that Google, YouTube, Dropbox, Instagram, Quora, and Reddit—among others—managed to build Python server-side applications serving hundreds of millions of users 24x7? Again, the answer goes way beyond what Python provides “out of the box.”

Before we discuss tools to support Python at scale, I must quote an admonition from the Thoughtworks *Technology Radar*:

High performance envy/web scale envy

We see many teams run into trouble because they have chosen complex tools, frameworks or architectures because they “might need to scale.” Companies such as Twitter and Netflix need to support extreme loads and so need these architectures, but they also have extremely skilled development teams able to handle the complexity. Most situations do not require these kinds of engineering feats; teams should keep their *web scale envy* in check in favor of simpler solutions that still get the job done.²⁰

At *web scale*, the key is an architecture that allows horizontal scaling. At that point, all systems are distributed systems, and no single programming language is likely to be the right choice for every part of the solution.

Distributed systems is a field of academic research, but fortunately some practitioners have written accessible books anchored on solid research and practical experience. One of them is Martin Kleppmann, the author of *Designing Data-Intensive Applications* (O’Reilly).

Consider [Figure 19-3](#), the first of many architecture diagrams in Kleppmann’s book. Here are some components I’ve seen in Python engagements that I worked on or have firsthand knowledge of:

- Application caches:²¹ *memcached*, *Redis*, *Varnish*
- Relational databases: *PostgreSQL*, *MySQL*
- Document databases: *Apache CouchDB*, *MongoDB*
- Full-text indexes: *Elasticsearch*, *Apache Solr*
- Message queues: *RabbitMQ*, *Redis*

20 Source: Thoughtworks Technology Advisory Board, *Technology Radar—November 2015*.

21 Contrast application caches—used directly by your application code—with HTTP caches, which would be placed on the top edge of [Figure 19-3](#) to serve static assets like images, CSS, and JS files. Content Delivery Networks (CDNs) offer another type of HTTP cache, deployed in data centers closer to the end users of your application.

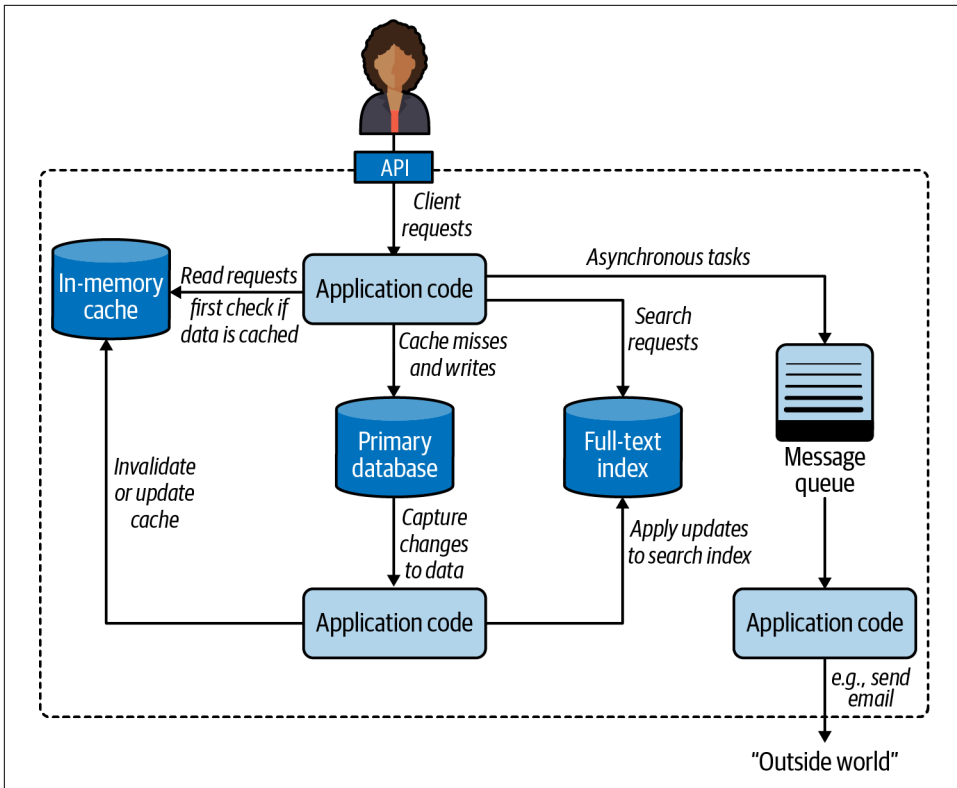


Figure 19-3. One possible architecture for a system combining several components.²²

There are other industrial-strength open source products in each of those categories. Major cloud providers also offer their own proprietary alternatives.

Kleppmann’s diagram is general and language independent—as is his book. For Python server-side applications, two specific components are often deployed:

- An application server to distribute the load among several instances of the Python application. The application server would appear near the top in [Figure 19-3](#), handling client requests before they reached the application code.
- A task queue built around the message queue on the righthand side of [Figure 19-3](#), providing a higher-level, easier-to-use API to distribute tasks to processes running on other machines.

²² Diagram adapted from Figure 1-1, *Designing Data-Intensive Applications* by Martin Kleppmann (O’Reilly).

The next two sections explore these components that are recommended best practices in Python server-side deployments.

WSGI Application Servers

WSGI—the **Web Server Gateway Interface**—is a standard API for a Python framework or application to receive requests from an HTTP server and send responses to it.²³ WSGI application servers manage one or more processes running your application, maximizing the use of the available CPUs.

Figure 19-4 illustrates a typical WSGI deployment.



If we wanted to merge the previous pair of diagrams, the content of the dashed rectangle in Figure 19-4 would replace the solid “Application code” rectangle at the top of Figure 19-3.

The best-known application servers in Python web projects are:

- *mod_wsgi*
- *uWSGI*²⁴
- *Gunicorn*
- *NGINX Unit*

For users of the Apache HTTP server, *mod_wsgi* is the best option. It’s as old as WSGI itself, but is actively maintained, and now provides a command-line launcher called *mod_wsgi-express* that makes it easier to configure and more suitable for use in Docker containers.

²³ Some speakers spell out the WSGI acronym, while others pronounce it as one word rhyming with “whisky.”

²⁴ *uWSGI* is spelled with a lowercase “u,” but that is pronounced as the Greek letter “μ,” so the whole name sounds like “micro-whisky” with a “g” instead of the “k.”

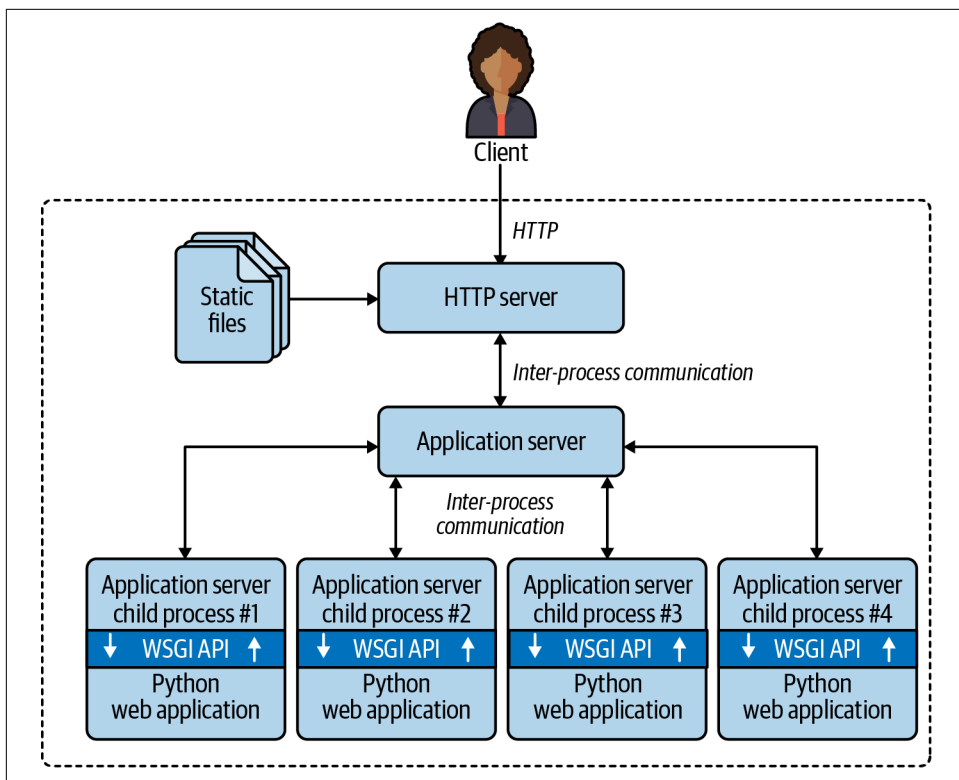


Figure 19-4. Clients connect to an HTTP server that delivers static files and routes other requests to the application server, which forks child processes to run the application code, leveraging multiple CPU cores. The WSGI API is the glue between the application server and the Python application code.

uWSGI and *Gunicorn* are the top choices in recent projects I know about. Both are often used with the *NGINX* HTTP server. *uWSGI* offers a lot of extra functionality, including an application cache, a task queue, cron-like periodic tasks, and many other features. On the flip side, *uWSGI* is much harder to configure properly than *Gunicorn*.²⁵

Released in 2018, *NGINX Unit* is a new product from the makers of the well-known *NGINX* HTTP server and reverse proxy.

²⁵ Bloomberg engineers Peter Sperl and Ben Green wrote “[Configuring uWSGI for Production Deployment](#)”, explaining how many of the default settings in *uWSGI* are not suitable for many common deployment scenarios. Sperl presented a summary of their recommendations at [EuroPython 2019](#). Highly recommended for users of *uWSGI*.

mod_wsgi and *Gunicorn* support Python web apps only, while *uWSGI* and *NGINX Unit* work with other languages as well. Please browse their docs to learn more.

The main point: all of these application servers can potentially use all CPU cores on the server by forking multiple Python processes to run traditional web apps written in good old sequential code in *Django*, *Flask*, *Pyramid*, etc. This explains why it's been possible to earn a living as a Python web developer without ever studying the threading, multiprocessing, or *asyncio* modules: the application server handles concurrency transparently.



ASGI—Asynchronous Server Gateway Interface

WSGI is a synchronous API. It doesn't support coroutines with *async/await*—the most efficient way to implement WebSockets or HTTP long polling in Python. The **ASGI specification** is a successor to WSGI, designed for asynchronous Python web frameworks such as *aiohttp*, *Sanic*, *FastAPI*, etc., as well as *Django* and *Flask*, which are gradually adding asynchronous functionality.

Now let's turn to another way of bypassing the GIL to achieve higher performance with server-side Python applications.

Distributed Task Queues

When the application server delivers a request to one of the Python processes running your code, your app needs to respond quickly: you want the process to be available to handle the next request as soon as possible. However, some requests demand actions that may take longer—for example, sending email or generating a PDF. That's the problem that distributed task queues are designed to solve.

Celery and *RQ* are the best known open source task queues with Python APIs. Cloud providers also offer their own proprietary task queues.

These products wrap a message queue and offer a high-level API for delegating tasks to workers, possibly running on different machines.



In the context of task queues, the words *producer* and *consumer* are used instead of traditional client/server terminology. For example, a *Django* view handler *produces* job requests, which are put in the queue to be *consumed* by one or more PDF rendering processes.

Quoting directly from *Celery*'s **FAQ**, here are some typical use cases:

- Running something in the background. For example, to finish the web request as soon as possible, then update the users page incrementally. This gives the user the

impression of good performance and “snappiness,” even though the real work might actually take some time.

- Running something after the web request has finished.
- Making sure something is done, by executing it asynchronously and using retries.
- Scheduling periodic work.

Besides solving these immediate problems, task queues support horizontal scalability. Producers and consumers are decoupled: a producer doesn’t call a consumer, it puts a request in a queue. Consumers don’t need to know anything about the producers (but the request may include information about the producer, if an acknowledgment is required). Crucially, you can easily add more workers to consume tasks as demand grows. That’s why *Celery* and *RQ* are called distributed task queues.

Recall that our simple *procs.py* (Example 19-13) used two queues: one for job requests, the other for collecting results. The distributed architecture of *Celery* and *RQ* uses a similar pattern. Both support using the *Redis* NoSQL database as a message queue and result storage. *Celery* also supports other message queues like *RabbitMQ* or *Amazon SQS*, as well other databases for result storage.

This wraps up our introduction to concurrency in Python. The next two chapters will continue this theme, focusing on the `concurrent.futures` and `asyncio` packages of the standard library.

Chapter Summary

After a bit of theory, this chapter presented the spinner scripts implemented in each of Python’s three native concurrency programming models:

- Threads, with the `threading` package
- Processes, with `multiprocessing`
- Asynchronous coroutines with `asyncio`

We then explored the real impact of the GIL with an experiment: changing the spinner examples to compute the primality of a large integer and observe the resulting behavior. This demonstrated graphically that CPU-intensive functions must be avoided in `asyncio`, as they block the event loop. The threaded version of the experiment worked—despite the GIL—because Python periodically interrupts threads, and the example used only two threads: one doing compute-intensive work, and the other driving the animation only 10 times per second. The `multiprocessing` variant worked around the GIL, starting a new process just for the animation, while the main process did the primality check.

The next example, computing several primes, highlighted the difference between multiprocessing and threading, proving that only processes allow Python to benefit from multicore CPUs. Python’s GIL makes threads worse than sequential code for heavy computations.

The GIL dominates discussions about concurrent and parallel computing in Python, but we should not overestimate its impact. That was the point of “[Python in the Multicore World](#)” on page 725. For example, the GIL doesn’t affect many use cases of Python in system administration. On the other hand, the data science and server-side development communities have worked around the GIL with industrial-strength solutions tailored to their specific needs. The last two sections mentioned two common elements to support Python server-side applications at scale: WSGI application servers and distributed task queues.

Further Reading

This chapter has an extensive reading list, so I split it into subsections.

Concurrency with Threads and Processes

The *concurrent.futures* library covered in [Chapter 20](#) uses threads, processes, locks, and queues under the hood, but you won’t see individual instances of them; they’re bundled and managed by the higher-level abstractions of a `ThreadPoolExecutor` and a `ProcessPoolExecutor`. If you want to learn more about the practice of concurrent programming with those low-level objects, “[An Intro to Threading in Python](#)” by Jim Anderson is a good first read. Doug Hellmann has a chapter titled “Concurrency with Processes, Threads, and Coroutines” on his [website](#) and book, *The Python 3 Standard Library by Example* (Addison-Wesley).

Brett Slatkin’s *Effective Python*, 2nd ed. (Addison-Wesley), David Beazley’s *Python Essential Reference*, 4th ed. (Addison-Wesley), and Martelli et al., *Python in a Nutshell*, 3rd ed. (O’Reilly) are other general Python references with significant coverage of threading and multiprocessing. The vast multiprocessing official documentation includes useful advice in its “[Programming guidelines](#)” section.

Jesse Noller and Richard Oudkerk contributed the multiprocessing package, introduced in [PEP 371—Addition of the multiprocessing package to the standard library](#). The official documentation for the package is a [93 KB .rst file](#)—that’s about 63 pages—making it one of the longest chapters in the Python standard library.

In *High Performance Python*, 2nd ed., (O’Reilly), authors Micha Gorelick and Ian Ozsvald include a chapter about multiprocessing with an example about checking for primes with a different strategy than our *procs.py* example. For each number, they split the range of possible factors—from 2 to $\sqrt{n}$ —into subranges, and make

each worker iterate over one of the subranges. Their divide-and-conquer approach is typical of scientific computing applications where the datasets are huge, and workstations (or clusters) have more CPU cores than users. On a server-side system handling requests from many users, it is simpler and more efficient to let each process work on one computation from start to finish—reducing the overhead of communication and coordination among processes. Besides multiprocessing, Gorelick and Ozsvald present many other ways of developing and deploying high-performance data science applications leveraging multiple cores, GPUs, clusters, profilers, and compilers like Cython and Numba. Their last chapter, “Lessons from the Field,” is a valuable collection of short case studies contributed by other practitioners of high-performance computing in Python.

Advanced Python Development by Matthew Wilkes (Apress), is a rare book that includes short examples to explain concepts, while also showing how to build a realistic application ready for production: a data aggregator, similar to DevOps monitoring systems or IoT data collectors for distributed sensors. Two chapters in *Advanced Python Development* cover concurrent programming with threading and asyncio.

Jan Palach’s *Parallel Programming with Python* (Packt, 2014) explains the core concepts behind concurrency and parallelism, covering Python’s standard library as well as *Celery*.

“The Truth About Threads” is the title of Chapter 2 in *Using Asyncio in Python* by Caleb Hattingh (O’Reilly).²⁶ The chapter covers the benefits and drawbacks of threading—with compelling quotes from several authoritative sources—making it clear that the fundamental challenges of threads have nothing to do with Python or the GIL. Quoting verbatim from page 14 of *Using Asyncio in Python*:

These themes repeat throughout:

- Threading makes code hard to reason about.
- Threading is an inefficient model for large-scale concurrency (thousands of concurrent tasks).

If you want to learn the hard way how difficult it is to reason about threads and locks—without risking your job—try the exercises in Allen Downey’s workbook, *The Little Book of Semaphores* (Green Tea Press). The exercises in Downey’s book range from easy to very hard to unsolvable, but even the easy ones are eye-opening.

²⁶ Caleb is one of the tech reviewers for this edition of *Fluent Python*.

The GIL

If you are intrigued about the GIL, remember we have no control over it from Python code, so the canonical reference is in the C-API documentation: *Thread State and the Global Interpreter Lock*. The *Python Library and Extension FAQ* answers: “*Can’t we get rid of the Global Interpreter Lock?*”. Also worth reading are posts by Guido van Rossum and Jesse Noller (contributor of the multiprocessing package), respectively: “*It isn’t Easy to Remove the GIL*” and “*Python Threads and the Global Interpreter Lock*”.

CPython Internals by Anthony Shaw (Real Python) explains the implementation of the CPython 3 interpreter at the C programming level. Shaw’s longest chapter is “Parallelism and Concurrency”: a deep dive into Python’s native support for threads and processes, including managing the GIL from extensions using the C/Python API.

Finally, David Beazley presented a detailed exploration in “*Understanding the Python GIL*”.²⁷ In slide #54 of the *presentation*, Beazley reports an increase in processing time for a particular benchmark with the new GIL algorithm introduced in Python 3.2. The issue is not significant with real workloads, according to *a comment* by Antoine Pitrou—who implemented the new GIL algorithm—in the bug report submitted by Beazley: *Python issue #7946*.

Concurrency Beyond the Standard Library

Fluent Python focuses on core language features and core parts of the standard library. *Full Stack Python* is a great complement to this book: it’s about Python’s ecosystem, with sections titled “Development Environments,” “Data,” “Web Development,” and “DevOps,” among others.

I’ve already mentioned two books that cover concurrency using the Python standard library that also include significant content on third-party libraries and tools: *High Performance Python*, 2nd ed. and *Parallel Programming with Python*. Francesco Pierfederici’s *Distributed Computing with Python* (Packt) covers the standard library and also the use of cloud providers and HPC (High-Performance Computing) clusters.

“*Python, Performance, and GPUs*” by Matthew Rocklin is “a status update for using GPU accelerators from Python,” posted in June 2019.

“Instagram currently features the world’s largest deployment of the *Django* web framework, which is written entirely in Python.” That’s the opening sentence of the blog post, “*Web Service Efficiency at Instagram with Python*”, written by Min Ni—a software engineer at Instagram. The post describes metrics and tools Instagram uses

²⁷ Thanks to Lucas Brunialti for sending me a link to this talk.

to optimize the efficiency of its Python codebase, as well as detect and diagnose performance regressions as it deploys its back end “30-50 times a day.”

Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices by Harry Percival and Bob Gregory (O’Reilly) presents architectural patterns for Python server-side applications. The authors also made the book freely available online at cosmicpython.com.

Two elegant and easy-to-use libraries for parallelizing tasks over processes are *lelo* by João S. O. Bueno and *python-parallelize* by Nat Pryce. The *lelo* package defines a `@parallel` decorator that you can apply to any function to magically make it unblocking: when you call the decorated function, its execution is started in another process. Nat Pryce’s *python-parallelize* package provides a `parallelize` generator that distributes the execution of a for loop over multiple CPUs. Both packages are built on the *multiprocessing* library.

Python core developer Eric Snow maintains a [Multicore Python](#) wiki, with notes about his and other people’s efforts to improve Python’s support for parallel execution. Snow is the author of [PEP 554—Multiple Interpreters in the Stdlib](#). If approved and implemented, PEP 554 lays the groundwork for future enhancements that may eventually allow Python to use multiple cores without the overheads of *multiprocessing*. One of the biggest blockers is the complex interaction between multiple active subinterpreters and extensions that assume a single interpreter.

Mark Shannon—also a Python maintainer—created a [useful table](#) comparing concurrent models in Python, referenced in a discussion about subinterpreters between him, Eric Snow, and other developers on the [python-dev](#) mailing list. In Shannon’s table, the “Ideal CSP” column refers to the theoretical [Communicating sequential processes](#) model proposed by Tony Hoare in 1978. Go also allows shared objects, violating an essential constraint of CSP: execution units should communicate through message passing through channels.

Stackless Python (a.k.a. *Stackless*) is a fork of CPython implementing microthreads, which are application-level lightweight threads—as opposed to OS threads. The massively multiplayer online game *EVE Online* was built on *Stackless*, and engineers employed by the game company CCP were [maintainers of Stackless](#) for a while. Some features of *Stackless* were reimplemented in the *Pypy* interpreter and the *greenlet* package, the core technology of the *gevent* networking library, which in turn is the foundation of the *Gunicorn* application server.

The actor model of concurrent programming is at the core of the highly scalable Erlang and Elixir languages, and is also the model of the Akka framework for Scala and Java. If you want to try out the actor model in Python, check out the *Thespian* and *Pykka* libraries.

My remaining recommendations have few or zero mentions of Python, but are nevertheless relevant to readers interested in the theme of this chapter.

Concurrency and Scalability Beyond Python

RabbitMQ in Action by Alvaro Videla and Jason J. W. Williams (Manning) is a very well-written introduction to *RabbitMQ* and the Advanced Message Queuing Protocol (AMQP) standard, with examples in Python, PHP, and Ruby. Regardless of the rest of your tech stack, and even if you plan to use *Celery* with *RabbitMQ* under the hood, I recommend this book for its coverage of concepts, motivation, and patterns for distributed message queues, as well as operating and tuning *RabbitMQ* at scale.

I learned a lot reading *Seven Concurrency Models in Seven Weeks*, by Paul Butcher (Pragmatic Bookshelf), with the eloquent subtitle *When Threads Unravel*. Chapter 1 of the book presents the core concepts and challenges of programming with threads and locks in Java.²⁸ The remaining six chapters of the book are devoted to what the author considers better alternatives for concurrent and parallel programming, as supported by different languages, tools, and libraries. The examples use Java, Clojure, Elixir, and C (for the chapter about parallel programming with the *OpenCL framework*). The CSP model is exemplified with Clojure code, although the Go language deserves credit for popularizing that approach. Elixir is the language of the examples illustrating the actor model. A freely available, alternative *bonus chapter* about actors uses Scala and the Akka framework. Unless you already know Scala, Elixir is a more accessible language to learn and experiment with the actor model and the Erlang/OTP distributed systems platform.

Unmesh Joshi of Thoughtworks has contributed several pages documenting “Patterns of Distributed Systems” to Martin Fowler’s *blog*. The *opening page* is a great introduction the topic, with links to individual patterns. Joshi is adding patterns incrementally, but what’s already there distills years of hard-earned experience in mission-critical systems.

Martin Kleppmann’s *Designing Data-Intensive Applications* (O’Reilly) is a rare book written by a practitioner with deep industry experience and advanced academic background. The author worked with large-scale data infrastructure at LinkedIn and two startups, before becoming a researcher of distributed systems at the University of Cambridge. Each chapter in Kleppmann’s book ends with an extensive list of references, including recent research results. The book also includes numerous illuminating diagrams and beautiful concept maps.

I was fortunate to be in the audience for Francesco Cesarini’s outstanding workshop on the architecture of reliable distributed systems at OSCON 2016: “Designing and

²⁸ Python’s `threading` and `concurrent.futures` APIs are heavily influenced by the Java standard library.

architecting for scalability with Erlang/OTP” ([video](#) at the O’Reilly Learning Platform). Despite the title, 9:35 into the video, Cesarini explains:

Very little of what I am going to say will be Erlang-specific [...]. The fact remains that Erlang will remove a lot of accidental difficulties to making systems which are resilient and which never fail, and are scalable. So it will be much easier if you do use Erlang, or a language running on the Erlang virtual machine.

That workshop was based on the last four chapters of *Designing for Scalability with Erlang/OTP* by Francesco Cesarini and Steve Vinoski (O’Reilly).

Programming distributed systems is challenging and exciting, but beware of *web-scale envy*. The **KISS principle** remains solid engineering advice.

Check out the paper “**Scalability! But at what COST?**” by Frank McSherry, Michael Isard, and Derek G. Murray. The authors identified parallel graph-processing systems presented in academic symposia that require hundreds of cores to outperform a “competent single-threaded implementation.” They also found systems that “underperform one thread for all of their reported configurations.”

Those findings remind me of a classic hacker quip:

My Perl script is faster than your Hadoop cluster.

Soapbox

To Manage Complexity, We Need Constraints

I learned to program on a TI-58 calculator. Its “language” was similar to assembly. At that level, all “variables” are globals, and you don’t have the luxury of structured control flow statements. You have conditional jumps: instructions that take the execution directly to an arbitrary location—ahead or behind the current spot—depending on the value of a CPU register or flag.

Basically you can do anything in assembly, and that’s the challenge: there are very few constraints to keep you from making mistakes, and to help maintainers understand the code when changes are needed.

The second language I learned was the unstructured BASIC that came with 8-bit computers—nothing like Visual Basic, which appeared much later. There were FOR, GOSUB, and RETURN statements, but still no concept of local variables. GOSUB did not support parameter passing; it was just a fancy GOTO that put a return line number in a stack so that RETURN had a target to jump to. Subroutines could help themselves to the global data, and put results there too. We had to improvise other forms of control flow with combinations of IF and GOTO—which, again, allowed you to jump to any line of the program.

After a few years of programming with jumps and global variables, I remember the struggle to rewire my brain for “structured programming” when I learned Pascal. Now I had to use control flow statements around blocks of code that have a single entry point. I couldn’t jump to any instruction I liked. Global variables were unavoidable in BASIC, but now they were taboo. I needed to rethink the flow of data and explicitly pass arguments to functions.

The next challenge for me was learning object-oriented programming. At its core, object-oriented programming is structured programming with more constraints and polymorphism. Information hiding forces yet another rethink of where data lives. I remember being frustrated more than once because I had to refactor my code so that a method I was writing could get information that was encapsulated in an object that my method could not reach.

Functional programming languages add other constraints, but immutability is the hardest to swallow after decades of imperative programming and object-oriented programming. After we get used to these constraints, we see them as blessings. They make reasoning about the code much easier.

Lack of constraints is the main problem with the threads-and-locks model of concurrent programming. When summarizing Chapter 1 of *Seven Concurrency Models in Seven Weeks*, Paul Butcher wrote:

The greatest weakness of the approach, however, is that threads-and-locks programming is *hard*. It may be easy for a language designer to add them to a language, but they provide us, the poor programmers, with very little help.

Some examples of unconstrained behavior in that model:

- Threads can share access to arbitrary, mutable data structures.
- The scheduler can interrupt a thread at almost any point, including in the middle of a simple operation like `a += 1`. Very few operations are atomic at the level of source code expressions.
- Locks are usually *advisory*. That’s a technical term meaning that you must remember to explicitly hold a lock before updating a shared data structure. If you forget to get the lock, nothing prevents your code from messing up the data while another thread dutifully holds the lock and is updating the same data.

In contrast, consider some constraints enforced by the actor model, in which the execution unit is called an *actor*:²⁹

²⁹ The Erlang community uses the term “process” for actors. In Erlang, each process is a function in its own loop, so they are very lightweight and it’s feasible to have millions of them active at once in a single machine —no relation to the heavyweight OS processes we’ve been talking about elsewhere in this chapter. So here we have examples of the two sins described by Prof. Simon: using different words to mean the same thing, and using one word to mean different things.

- An actor can have internal state, but cannot share state with other actors.
- Actors can only communicate by sending and receiving messages.
- Messages only hold copies of data, not references to mutable data.
- An actor only handles one message at a time. There is no concurrent execution inside a single actor.

Of course, you can adopt an *actor style* of coding in any language by following these rules. You can also use object-oriented programming idioms in C, and even structured programming patterns in assembly. But doing any of that requires a lot of agreement and discipline among everyone who touches the code.

Managing locks is unnecessary in the actor model, as implemented by Erlang and Elixir, where all data types are immutable.

Threads-and-locks are not going away. I just don't think dealing with such low-level entities is a good use of my time as I write applications—as opposed to kernel modules or databases.

I reserve the right to change my mind, always. But right now, I am convinced that the actor model is the most sensible, general-purpose concurrent programming model available. CSP (Communicating Sequential Processes) is also sensible, but its implementation in Go leaves out some constraints. The idea in CSP is that coroutines (or *goroutines* in Go) exchange data and synchronize using queues (called *channels* in Go). But Go also supports memory sharing and locks. I've seen a book about Go advocate the use of shared memory and locks instead of channels—in the name of performance. Old habits die hard.

Concurrent Executors

The people bashing threads are typically system programmers which have in mind use cases that the typical application programmer will never encounter in her life. [...] In 99% of the use cases an application programmer is likely to run into, the simple pattern of spawning a bunch of independent threads and collecting the results in a queue is everything one needs to know.

—Michele Simionato, Python deep thinker¹

This chapter focuses on the `concurrent.futures.Executor` classes that encapsulate the pattern of “spawning a bunch of independent threads and collecting the results in a queue,” described by Michele Simionato. The concurrent executors make this pattern almost trivial to use, not only with threads but also with processes—useful for compute-intensive tasks.

Here I also introduce the concept of *futures*—objects representing the asynchronous execution of an operation, similar to JavaScript promises. This primitive idea is the foundation not only of `concurrent.futures` but also of the `asyncio` package, the subject of [Chapter 21](#).

What’s New in This Chapter

I renamed the chapter from “Concurrency with Futures” to “Concurrent Executors” because the executors are the most important high-level feature covered here. Futures are low-level objects, focused on in [“Where Are the Futures?” on page 751](#), but mostly invisible in the rest of the chapter.

¹ From Michele Simionato’s post, [“Threads, processes and concurrency in Python: some thoughts”](#), summarized as “Removing the hype around the multicore (non) revolution and some (hopefully) sensible comment about threads and other forms of concurrency.”

All the HTTP client examples now use the new *HTTPX* library, which provides synchronous and asynchronous APIs.

The setup for the experiments in “Downloads with Progress Display and Error Handling” on page 762 is now simpler, thanks to the multithreaded server added to the *http.server* package in Python 3.7. Previously, the standard library only had the single-threaded *BaseHttpServer*, which was no good for experimenting with concurrent clients, so I had to resort to external tools in the first edition of this book.

“Launching Processes with *concurrent.futures*” on page 754 now demonstrates how an executor simplifies the code we saw in “Code for the Multicore Prime Checker” on page 719.

Finally, I moved most of the theory to the new Chapter 19, “Concurrency Models in Python”.

Concurrent Web Downloads

Concurrency is essential for efficient network I/O: instead of idly waiting for remote machines, the application should do something else until a response comes back.²

To demonstrate with code, I wrote three simple programs to download images of 20 country flags from the web. The first one, *flags.py*, runs sequentially: it only requests the next image when the previous one is downloaded and saved locally. The other two scripts make concurrent downloads: they request several images practically at the same time, and save them as they arrive. The *flags_threadpool.py* script uses the *concurrent.futures* package, while *flags_asyncio.py* uses *asyncio*.

Example 20-1 shows the result of running the three scripts, three times each. I also posted a 73s video on YouTube so you can watch them running while a macOS Finder window displays the flags as they are saved. The scripts are downloading images from *fluentpython.com*, which is behind a CDN, so you may see slower results in the first runs. The results in Example 20-1 were obtained after several runs, so the CDN cache was warm.

*Example 20-1. Three typical runs of the scripts *flags.py*, *flags_threadpool.py*, and *flags_asyncio.py**

```
$ python3 flags.py
BD BR CD CN DE EG ET FR ID IN IR JP MX NG PH PK RU TR US VN ❶
20 flags downloaded in 7.26s ❷
$ python3 flags.py
BD BR CD CN DE EG ET FR ID IN IR JP MX NG PH PK RU TR US VN
```

² Particularly if your cloud provider rents machines by the second, regardless of how busy the CPUs are.

```

20 flags downloaded in 7.20s
$ python3 flags.py
BD BR CD CN DE EG ET FR ID IN IR JP MX NG PH PK RU TR US VN
20 flags downloaded in 7.09s
$ python3 flags_threadpool.py
DE BD CN JP ID EG NG BR RU CD IR MX US PH FR PK VN IN ET TR
20 flags downloaded in 1.37s ❸
$ python3 flags_threadpool.py
EG BR FR IN BD JP DE RU PK PH CD MX ID US NG TR CN VN ET IR
20 flags downloaded in 1.60s
$ python3 flags_threadpool.py
BD DE EG CN ID RU IN VN ET MX FR CD NG US JP TR PK BR IR PH
20 flags downloaded in 1.22s
$ python3 flags_asyncio.py ❹
BD BR IN ID TR DE CN US IR PK PH FR RU NG VN ET MX EG JP CD
20 flags downloaded in 1.36s
$ python3 flags_asyncio.py
RU CN BR IN FR BD TR EG VN IR PH CD ET ID NG DE JP PK MX US
20 flags downloaded in 1.27s
$ python3 flags_asyncio.py
RU IN ID DE BR VN PK MX US IR ET EG NG BD FR CN JP PH CD TR ❺
20 flags downloaded in 1.42s

```

- ❶ The output for each run starts with the country codes of the flags as they are downloaded, and ends with a message stating the elapsed time.
- ❷ It took *flags.py* an average 7.18s to download 20 images.
- ❸ The average for *flags_threadpool.py* was 1.40s.
- ❹ For *flags_asyncio.py*, 1.35s was the average time.
- ❺ Note the order of the country codes: the downloads happened in a different order every time with the concurrent scripts.

The difference in performance between the concurrent scripts is not significant, but they are both more than five times faster than the sequential script—and this is just for the small task of downloading 20 files of a few kilobytes each. If you scale the task to hundreds of downloads, the concurrent scripts can outpace the sequential code by a factor of 20 or more.



While testing concurrent HTTP clients against public web servers, you may inadvertently launch a denial-of-service (DoS) attack, or be suspected of doing so. In the case of [Example 20-1](#), it's OK to do it because those scripts are hardcoded to make only 20 requests. We'll use Python's `http.server` package to run tests later in this chapter.

Now let's study the implementations of two of the scripts tested in [Example 20-1](#): *flags.py* and *flags_threadpool.py*. I will leave the third script, *flags_asyncio.py*, for [Chapter 21](#), but I wanted to demonstrate all three together to make two points:

1. Regardless of the concurrency constructs you use—threads or coroutines—you'll see vastly improved throughput over sequential code in network I/O operations, if you code it properly.
2. For HTTP clients that can control how many requests they make, there is no significant difference in performance between threads and coroutines.³

On to the code.

A Sequential Download Script

[Example 20-2](#) contains the implementation of *flags.py*, the first script we ran in [Example 20-1](#). It's not very interesting, but we'll reuse most of its code and settings to implement the concurrent scripts, so it deserves some attention.



For clarity, there is no error handling in [Example 20-2](#). We will deal with exceptions later, but here I want to focus on the basic structure of the code, to make it easier to contrast this script with the concurrent ones.

Example 20-2. flags.py: sequential download script; some functions will be reused by the other scripts

```
import time
from pathlib import Path
from typing import Callable

import httpx ❶

POP20_CC = ('CN IN US ID BR PK NG BD RU JP '
            'MX PH VN ET EG DE IR TR CD FR').split() ❷

BASE_URL = 'https://www.fluentpython.com/data/flags' ❸
DEST_DIR = Path('downloaded') ❹

def save_flag(img: bytes, filename: str) -> None: ❺
    (DEST_DIR / filename).write_bytes(img)
```

³ For servers that may be hit by many clients, there is a difference: coroutines scale better because they use much less memory than threads, and also reduce the cost of context switching, which I mentioned in “Thread-Based Nonsolution” on page 724.

```

def get_flag(cc: str) -> bytes: ❹
    url = f'{BASE_URL}/{cc}/{cc}.gif'.lower()
    resp = httpx.get(url, timeout=6.1, ❷
                     follow_redirects=True) ❸
    resp.raise_for_status() ❹
    return resp.content

def download_many(cc_list: list[str]) -> int: ❺
    for cc in sorted(cc_list): ❻
        image = get_flag(cc)
        save_flag(image, f'{cc}.gif')
        print(cc, end=' ', flush=True) ❼
    return len(cc_list)

def main(downloader: Callable[[list[str]], int]) -> None: ❶
    DEST_DIR.mkdir(exist_ok=True) ❷
    t0 = time.perf_counter() ❸
    count = downloader(POP20_CC)
    elapsed = time.perf_counter() - t0
    print(f'\n{count} downloads in {elapsed:.2f}s')

if __name__ == '__main__':
    main(download_many) ❽

```

- ❶ Import the `httpx` library. It's not part of the standard library, so by convention the import goes after the standard library modules and a blank line.
- ❷ List of the ISO 3166 country codes for the 20 most populous countries in order of decreasing population.
- ❸ The directory with the flag images.⁴
- ❹ Local directory where the images are saved.
- ❺ Save the `img` bytes to `filename` in the `DEST_DIR`.
- ❻ Given a country code, build the URL and download the image, returning the binary contents of the response.
- ❼ It's good practice to add a sensible timeout to network operations, to avoid blocking for several minutes for no good reason.

⁴ The images are originally from the [CIA World Factbook](#), a public-domain, US government publication. I copied them to my site to avoid the risk of launching a DOS attack on [cia.gov](#).

- ❶ By default, *HTTPX* does not follow redirects.⁵
- ❷ There's no error handling in this script, but this method raises an exception if the HTTP status is not in the 2XX range—highly recommended to avoid silent failures.
- ❸ `download_many` is the key function to compare with the concurrent implementations.
- ❹ Loop over the list of country codes in alphabetical order, to make it easy to see that the ordering is preserved in the output; return the number of country codes downloaded.
- ❺ Display one country code at a time in the same line so we can see progress as each download happens. The `end=' '` argument replaces the usual line break at the end of each line printed with a space character, so all country codes are displayed progressively in the same line. The `flush=True` argument is needed because, by default, Python output is line buffered, meaning that Python only displays printed characters after a line break.
- ❻ `main` must be called with the function that will make the downloads; that way, we can use `main` as a library function with other implementations of `download_many` in the `threadpool` and `asyncio` examples.
- ❼ Create `DEST_DIR` if needed; don't raise an error if the directory exists.
- ❽ Record and report the elapsed time after running the `download` function.
- ❾ Call `main` with the `download_many` function.



The *HTTPX* library is inspired by the Pythonic *requests* package, but is built on a more modern foundation. Crucially, *HTTPX* provides synchronous and asynchronous APIs, so we can use it in all HTTP client examples in this chapter and the next. Python's standard library provides the `urllib.request` module, but its API is synchronous only, and is not user friendly.

⁵ Setting `follow_redirects=True` is not needed for this example, but I wanted to highlight this important difference between *HTTPX* and *requests*. Also, setting `follow_redirects=True` in this example gives me flexibility to host the image files elsewhere in the future. I think the *HTTPX* default setting of `follow_redirects=False` is sensible because unexpected redirects can mask needless requests and complicate error diagnostics.

There's really nothing new to *flags.py*. It serves as a baseline for comparing the other scripts, and I used it as a library to avoid redundant code when implementing them. Now let's see a reimplementation using `concurrent.futures`.

Downloading with `concurrent.futures`

The main features of the `concurrent.futures` package are the `ThreadPoolExecutor` and `ProcessPoolExecutor` classes, which implement an API to submit callables for execution in different threads or processes, respectively. The classes transparently manage a pool of worker threads or processes, and queues to distribute jobs and collect results. But the interface is very high-level, and we don't need to know about any of those details for a simple use case like our flag downloads.

Example 20-3 shows the easiest way to implement the downloads concurrently, using the `ThreadPoolExecutor.map` method.

Example 20-3. `flags_threadpool.py`: threaded download script using `futures.ThreadPoolExecutor`

```
from concurrent import futures

from flags import save_flag, get_flag, main ❶

def download_one(cc: str): ❷
    image = get_flag(cc)
    save_flag(image, f'{cc}.gif')
    print(cc, end=' ', flush=True)
    return cc

def download_many(cc_list: list[str]) -> int:
    with futures.ThreadPoolExecutor() as executor: ❸
        res = executor.map(download_one, sorted(cc_list)) ❹

    return len(list(res)) ❺

if __name__ == '__main__':
    main(download_many) ❻
```

- ❶ Reuse some functions from the `flags` module (**Example 20-2**).
- ❷ Function to download a single image; this is what each worker will execute.
- ❸ Instantiate the `ThreadPoolExecutor` as a context manager; the executor's `__exit__` method will call `executor.shutdown(wait=True)`, which will block until all threads are done.

- ④ The `map` method is similar to the `map` built-in, except that the `download_one` function will be called concurrently from multiple threads; it returns a generator that you can iterate to retrieve the value returned by each function call—in this case, each call to `download_one` will return a country code.
- ⑤ Return the number of results obtained. If any of the threaded calls raises an exception, that exception is raised here when the implicit `next()` call inside the `list` constructor tries to retrieve the corresponding return value from the iterator returned by `executor.map`.
- ⑥ Call the `main` function from the `flags` module, passing the concurrent version of `download_many`.

Note that the `download_one` function from [Example 20-3](#) is essentially the body of the `for` loop in the `download_many` function from [Example 20-2](#). This is a common refactoring when writing concurrent code: turning the body of a sequential `for` loop into a function to be called concurrently.



[Example 20-3](#) is very short because I was able to reuse most functions from the sequential *flags.py* script. One of the best features of `concurrent.futures` is to make it simple to add concurrent execution on top of legacy sequential code.

The `ThreadPoolExecutor` constructor takes several arguments not shown, but the first and most important one is `max_workers`, setting the maximum number of worker threads to be executed. When `max_workers` is `None` (the default), `ThreadPoolExecutor` decides its value using the following expression—since Python 3.8:

```
max_workers = min(32, os.cpu_count() + 4)
```

The rationale is explained in the [ThreadPoolExecutor documentation](#):

This default value preserves at least 5 workers for I/O bound tasks. It utilizes at most 32 CPU cores for CPU bound tasks which release the GIL. And it avoids using very large resources implicitly on many-core machines.

`ThreadPoolExecutor` now reuses idle worker threads before starting `max_workers` worker threads too.

To conclude: the computed default for `max_workers` is sensible, and `ThreadPoolExecutor` avoids starting new workers unnecessarily. Understanding the logic behind `max_workers` may help you decide when and how to set it yourself.

The library is called *concurrency.futures*, yet there are no futures to be seen in [Example 20-3](#), so you may be wondering where they are. The next section explains.

Where Are the Futures?

Futures are core components of `concurrent.futures` and of `asyncio`, but as users of these libraries we sometimes don't see them. [Example 20-3](#) depends on futures behind the scenes, but the code I wrote does not touch them directly. This section is an overview of futures, with an example that shows them in action.

Since Python 3.4, there are two classes named `Future` in the standard library: `concurrent.futures.Future` and `asyncio.Future`. They serve the same purpose: an instance of either `Future` class represents a deferred computation that may or may not have completed. This is somewhat similar to the `Deferred` class in Twisted, the `Future` class in Tornado, and `Promise` in modern JavaScript.

Futures encapsulate pending operations so that we can put them in queues, check whether they are done, and retrieve results (or exceptions) when they become available.

An important thing to know about futures is that you and I should not create them: they are meant to be instantiated exclusively by the concurrency framework, be it `concurrent.futures` or `asyncio`. Here is why: a `Future` represents something that will eventually run, therefore it must be scheduled to run, and that's the job of the framework. In particular, `concurrent.futures.Future` instances are created only as the result of submitting a callable for execution with a `concurrent.futures.Executor` subclass. For example, the `Executor.submit()` method takes a callable, schedules it to run, and returns a `Future`.

Application code is not supposed to change the state of a future: the concurrency framework changes the state of a future when the computation it represents is done, and we can't control when that happens.

Both types of `Future` have a `.done()` method that is nonblocking and returns a Boolean that tells you whether the callable wrapped by that future has executed or not. However, instead of repeatedly asking whether a future is done, client code usually asks to be notified. That's why both `Future` classes have an `.add_done_callback()` method: you give it a callable, and the callable will be invoked with the future as the single argument when the future is done. Be aware that the callback callable will run in the same worker thread or process that ran the function wrapped in the future.

There is also a `.result()` method, which works the same in both classes when the future is done: it returns the result of the callable, or re-raises whatever exception might have been thrown when the callable was executed. However, when the future is not done, the behavior of the `result` method is very different between the two flavors of `Future`. In a `concurrent.futures.Future` instance, invoking `f.result()` will block the caller's thread until the result is ready. An optional `timeout` argument can

be passed, and if the future is not done in the specified time, the `result` method raises `TimeoutError`. The `asyncio.Future.result` method does not support timeout, and `await` is the preferred way to get the result of futures in `asyncio`—but `await` doesn't work with `concurrent.futures.Future` instances.

Several functions in both libraries return futures; others use them in their implementation in a way that is transparent to the user. An example of the latter is the `Executor.map` we saw in [Example 20-3](#): it returns an iterator in which `__next__` calls the `result` method of each future, so we get the results of the futures, and not the futures themselves.

To get a practical look at futures, we can rewrite [Example 20-3](#) to use the `concurrent.futures.as_completed` function, which takes an iterable of futures and returns an iterator that yields futures as they are done.

Using `futures.as_completed` requires changes to the `download_many` function only. The higher-level `executor.map` call is replaced by two `for` loops: one to create and schedule the futures, the other to retrieve their results. While we are at it, we'll add a few `print` calls to display each future before and after it's done. [Example 20-4](#) shows the code for a new `download_many` function. The code for `download_many` grew from 5 to 17 lines, but now we get to inspect the mysterious futures. The remaining functions are the same as in [Example 20-3](#).

Example 20-4. `flags_threadpool_futures.py`: replacing `executor.map` with `executor.submit` and `futures.as_completed` in the `download_many` function

```
def download_many(cc_list: list[str]) -> int:
    cc_list = cc_list[:5] ❶
    with futures.ThreadPoolExecutor(max_workers=3) as executor: ❷
        to_do: list[futures.Future] = []
        for cc in sorted(cc_list): ❸
            future = executor.submit(download_one, cc) ❹
            to_do.append(future) ❺
            print(f'Scheduled for {cc}: {future}') ❻

        for count, future in enumerate(futures.as_completed(to_do), 1): ❼
            res: str = future.result() ❽
            print(f'{future} result: {res!r}') ❾

    return count
```

- ❶ For this demonstration, use only the top five most populous countries.
- ❷ Set `max_workers` to 3 so we can see pending futures in the output.
- ❸ Iterate over country codes alphabetically, to make it clear that results will arrive out of order.
- ❹ `executor.submit` schedules the callable to be executed, and returns a future representing this pending operation.
- ❺ Store each future so we can later retrieve them with `as_completed`.
- ❻ Display a message with the country code and the respective future.
- ❼ `as_completed` yields futures as they are completed.
- ❽ Get the result of this future.
- ❾ Display the future and its result.

Note that the `future.result()` call will never block in this example because the future is coming out of `as_completed`. [Example 20-5](#) shows the output of one run of [Example 20-4](#).

Example 20-5. Output of `flags_threadpool_futures.py`

```
$ python3 flags_threadpool_futures.py
Scheduled for BR: <Future at 0x100791518 state=running> ❶
Scheduled for CN: <Future at 0x100791710 state=running>
Scheduled for ID: <Future at 0x100791a90 state=running>
Scheduled for IN: <Future at 0x101807080 state=pending> ❷
Scheduled for US: <Future at 0x101807128 state=pending>
CN <Future at 0x100791710 state=finished returned str> result: 'CN' ❸
BR ID <Future at 0x100791518 state=finished returned str> result: 'BR' ❹
<Future at 0x100791a90 state=finished returned str> result: 'ID'
IN <Future at 0x101807080 state=finished returned str> result: 'IN'
US <Future at 0x101807128 state=finished returned str> result: 'US'

5 downloads in 0.70s
```

- ❶ The futures are scheduled in alphabetical order; the `repr()` of a future shows its state: the first three are running, because there are three worker threads.
- ❷ The last two futures are pending, waiting for worker threads.

- ③ The first CN here is the output of `download_one` in a worker thread; the rest of the line is the output of `download_many`.
- ④ Here, two threads output codes before `download_many` in the main thread can display the result of the first thread.



I recommend experimenting with `flags_threadpool_futures.py`. If you run it several times, you'll see the order of the results varying. Increasing `max_workers` to 5 will increase the variation in the order of the results. Decreasing it to 1 will make this script run sequentially, and the order of the results will always be the order of the submit calls.

We saw two variants of the download script using `concurrent.futures`: one in [Example 20-3](#) with `ThreadPoolExecutor.map` and one in [Example 20-4](#) with `futures.as_completed`. If you are curious about the code for `flags_asyncio.py`, you may peek at [Example 21-3](#) in [Chapter 21](#), where it is explained.

Now let's take a brief look at a simple way to work around the GIL for CPU-bound jobs using `concurrent.futures`.

Launching Processes with `concurrent.futures`

The [concurrent.futures documentation page](#) is subtitled “Launching parallel tasks.” The package enables parallel computation on multicore machines because it supports distributing work among multiple Python processes using the `ProcessPoolExecutor` class.

Both `ProcessPoolExecutor` and `ThreadPoolExecutor` implement the `Executor` interface, so it's easy to switch from a thread-based to a process-based solution using `concurrent.futures`.

There is no advantage in using a `ProcessPoolExecutor` for the flags download example or any I/O-bound job. It's easy to verify this; just change these lines in [Example 20-3](#):

```
def download_many(cc_list: list[str]) -> int:
    with futures.ThreadPoolExecutor() as executor:
```

To this:

```
def download_many(cc_list: list[str]) -> int:
    with futures.ProcessPoolExecutor() as executor:
```

The constructor for `ProcessPoolExecutor` also has a `max_workers` parameter, which defaults to `None`. In that case, the executor limits the number of workers to the number returned by `os.cpu_count()`.

Processes use more memory and take longer to start than threads, so the real value of `ProcessPoolExecutor` is in CPU-intensive jobs. Let's go back to the primality test example of “A Homegrown Process Pool” on page 716, rewriting it with `concurrent.futures`.

Multicore Prime Checker Redux

In “Code for the Multicore Prime Checker” on page 719 we studied `procs.py`, a script that checked the primality of some large numbers using multiprocessing. In [Example 20-6](#) we solve the same problem in the `proc_pool.py` program using a `ProcessPoolExecutor`. From the first import to the `main()` call at the end, `procs.py` has 43 nonblank lines of code, and `proc_pool.py` has 31—28% shorter.

Example 20-6. `proc_pool.py`: `procs.py` rewritten with `ProcessPoolExecutor`

```
import sys
from concurrent import futures ❶
from time import perf_counter
from typing import NamedTuple

from primes import is_prime, NUMBERS

class PrimeResult(NamedTuple): ❷
    n: int
    flag: bool
    elapsed: float

def check(n: int) -> PrimeResult:
    t0 = perf_counter()
    res = is_prime(n)
    return PrimeResult(n, res, perf_counter() - t0)

def main() -> None:
    if len(sys.argv) < 2:
        workers = None ❸
    else:
        workers = int(sys.argv[1])

    executor = futures.ProcessPoolExecutor(workers) ❹
    actual_workers = executor._max_workers # type: ignore ❺

    print(f'Checking {len(NUMBERS)} numbers with {actual_workers} processes:')

    t0 = perf_counter()
```

```

numbers = sorted(NUMBERS, reverse=True) ❹
with executor: ❺
    for n, prime, elapsed in executor.map(check, numbers): ❸
        label = 'p' if prime else ' '
        print(f'{n:16} {label} {elapsed:9.6f}s')

time = perf_counter() - t0
print(f'Total time: {time:.2f}s')

if __name__ == '__main__':
    main()

```

- ❶ No need to import multiprocessing, SimpleQueue etc.; concurrent.futures hides all that.
- ❷ The PrimeResult tuple and the check function are the same as we saw in *procs.py*, but we don't need the queues and the worker function anymore.
- ❸ Instead of deciding ourselves how many workers to use if no command-line argument was given, we set workers to None and let the ProcessPoolExecutor decide.
- ❹ Here I build the ProcessPoolExecutor before the with block in ❺ so that I can display the actual number of workers in the next line.
- ❺ `_max_workers` is an undocumented instance attribute of a ProcessPoolExecutor. I decided to use it to show the number of workers when the workers variable is None. *Mypy* correctly complains when I access it, so I put the type: ignore comment to silence it.
- ❻ Sort the numbers to be checked in descending order. This will expose a difference in the behavior of *proc_pool.py* when compared with *procs.py*. See the explanation after this example.
- ❼ Use the executor as a context manager.
- ❽ The executor.map call returns the PrimeResult instances returned by check in the same order as the numbers arguments.

If you run [Example 20-6](#), you'll see the results appearing in strict descending order, as shown in [Example 20-7](#). In contrast, the ordering of the output of *procs.py* (shown in “[Process-Based Solution](#)” on page 718) is heavily influenced by the difficulty in checking whether each number is a prime. For example, *procs.py* shows the result for

7777777777777777 near the top, because it has a low divisor, 7, so `is_prime` quickly determines it's not a prime.

In contrast, 7777777536340681 is 88191709^2 , so `is_prime` will take much longer to determine that it's a composite number, and even longer to find out that 777777777777753 is prime—therefore both of these numbers appear near the end of the output of `procs.py`.

Running `proc_pool.py`, you'll observe not only the descending order of the results, but also that the program will appear to be stuck after showing the result for 9999999999999999.

Example 20-7. Output of `proc_pool.py`

```
$ ./proc_pool.py
Checking 20 numbers with 12 processes:
9999999999999999      0.000024s ❶
9999999999999917 P  9.500677s ❷
7777777777777777      0.000022s ❸
7777777777777753 P  8.976933s
7777777536340681      8.896149s
6666667141414921      8.537621s
6666666666666719 P  8.548641s
6666666666666666      0.000002s
5555555555555555      0.000017s
5555555555555503 P  8.214086s
5555553133149889      8.067247s
4444444488888889      7.546234s
4444444444444444      0.000002s
4444444444444423 P  7.622370s
3333335652092209      6.724649s
3333333333333333      0.000018s
3333333333333301 P  6.655039s
299593572317531 P  2.072723s
142702110479723 P  1.461840s
      2 P  0.000001s
Total time: 9.65s
```

- ❶ This line appears very quickly.
- ❷ This line takes more than 9.5s to show up.
- ❸ All the remaining lines appear almost immediately.

Here is why *proc_pool.py* behaves in that way:

- As mentioned before, `executor.map(check, numbers)` returns the result in the same order as the numbers are given.
- By default, *proc_pool.py* uses as many workers as there are CPUs—it's what `ProcessPoolExecutor` does when `max_workers` is `None`. That's 12 processes in this laptop.
- Because we are submitting numbers in descending order, the first is 9999999999999999; with 9 as a divisor, it returns quickly.
- The second number is 9999999999999917, the largest prime in the sample. This will take longer than all the others to check.
- Meanwhile, the remaining 11 processes will be checking other numbers, which are either primes or composites with large factors, or composites with very small factors.
- When the worker in charge of 9999999999999917 finally determines that's a prime, all the other processes have completed their last jobs, so the results appear immediately after.



Although the progress of *proc_pool.py* is not as visible as that of *procs.py*, the overall execution time is practically the same as depicted in [Figure 19-2](#), for the same number of workers and CPU cores.

Understanding how concurrent programs behave is not straightforward, so here's is a second experiment that may help you visualize the operation of `Executor.map`.

Experimenting with `Executor.map`

Let's investigate `Executor.map`, now using a `ThreadPoolExecutor` with three workers running five callables that output timestamped messages. The code is in [Example 20-8](#), the output in [Example 20-9](#).

Example 20-8. `demo_executor_map.py`: Simple demonstration of the `map` method of `ThreadPoolExecutor`

```
from time import sleep, strftime
from concurrent import futures

def display(*args): ❶
    print(strftime('%H:%M:%S'), end=' ')
    print(*args)
```

```

def loiter(n): ❷
    msg = '{}loiter({}): doing nothing for {}s...'
    display(msg.format('\t'*n, n, n))
    sleep(n)
    msg = '{}loiter({}): done.'
    display(msg.format('\t'*n, n))
    return n * 10 ❸

def main():
    display('Script starting.')
    executor = futures.ThreadPoolExecutor(max_workers=3) ❹
    results = executor.map(loiter, range(5)) ❺
    display('results:', results) ❻
    display('Waiting for individual results:')
    for i, result in enumerate(results): ❼
        display(f'result {i}: {result}')

if __name__ == '__main__':
    main()

```

- ❶ This function simply prints whatever arguments it gets, preceded by a timestamp in the format [HH:MM:SS].
- ❷ loiter does nothing except display a message when it starts, sleep for n seconds, then display a message when it ends; tabs are used to indent the messages according to the value of n.
- ❸ loiter returns `n * 10` so we can see how to collect results.
- ❹ Create a `ThreadPoolExecutor` with three threads.
- ❺ Submit five tasks to the executor. Since there are only three threads, only three of those tasks will start immediately: the calls `loiter(0)`, `loiter(1)`, and `loiter(2)`; this is a nonblocking call.
- ❻ Immediately display the results of invoking `executor.map`: it's a generator, as the output in [Example 20-9](#) shows.
- ❼ The `enumerate` call in the for loop will implicitly invoke `next(results)`, which in turn will invoke `_f.result()` on the (internal) `_f` future representing the first call, `loiter(0)`. The `result` method will block until the future is done, therefore each iteration in this loop will have to wait for the next result to be ready.

I encourage you to run [Example 20-8](#) and see the display being updated incrementally. While you're at it, play with the `max_workers` argument for the `ThreadPoolExecutor` and with the `range` function that produces the arguments for the `executor.map` call—or replace it with lists of handpicked values to create different delays.

[Example 20-9](#) shows a sample run of [Example 20-8](#).

Example 20-9. Sample run of `demo_executor_map.py` from [Example 20-8](#)

```
$ python3 demo_executor_map.py
[15:56:50] Script starting. ❶
[15:56:50] loiter(0): doing nothing for 0s... ❷
[15:56:50] loiter(0): done.
[15:56:50]     loiter(1): doing nothing for 1s... ❸
[15:56:50]     loiter(2): doing nothing for 2s...
[15:56:50] results: <generator object result_iterator at 0x106517168> ❹
[15:56:50]     loiter(3): doing nothing for 3s... ❺
[15:56:50] Waiting for individual results:
[15:56:50] result 0: 0 ❻
[15:56:51]     loiter(1): done. ❼
[15:56:51]     loiter(4): doing nothing for 4s...
[15:56:51] result 1: 10 ❽
[15:56:52]     loiter(2): done. ❾
[15:56:52] result 2: 20
[15:56:53]     loiter(3): done.
[15:56:53] result 3: 30
[15:56:55]     loiter(4): done. ❿
[15:56:55] result 4: 40
```

- ❶ This run started at 15:56:50.
- ❷ The first thread executes `loiter(0)`, so it will sleep for 0s and return even before the second thread has a chance to start, but YMMV.⁶
- ❸ `loiter(1)` and `loiter(2)` start immediately (because the thread pool has three workers, it can run three functions concurrently).
- ❹ This shows that the results returned by `executor.map` is a generator; nothing so far would block, regardless of the number of tasks and the `max_workers` setting.

⁶ Your mileage may vary: with threads, you never know the exact sequencing of events that should happen nearly at the same time; it's possible that, in another machine, you see `loiter(1)` starting before `loiter(0)` finishes, particularly because `sleep` always releases the GIL, so Python may switch to another thread even if you sleep for 0s.

- ⑤ Because `loiter(0)` is done, the first worker is now available to start the fourth thread for `loiter(3)`.
- ⑥ This is where execution may block, depending on the parameters given to the `loiter` calls: the `__next__` method of the `results` generator must wait until the first future is complete. In this case, it won't block because the call to `loiter(0)` finished before this loop started. Note that everything up to this point happened within the same second: 15:56:50.
- ⑦ `loiter(1)` is done one second later, at 15:56:51. The thread is freed to start `loiter(4)`.
- ⑧ The result of `loiter(1)` is shown: 10. Now the `for` loop will block waiting for the result of `loiter(2)`.
- ⑨ The pattern repeats: `loiter(2)` is done, its result is shown; same with `loiter(3)`.
- ⑩ There is a 2s delay until `loiter(4)` is done, because it started at 15:56:51 and did nothing for 4s.

The `Executor.map` function is easy to use, but often it's preferable to get the results as they are ready, regardless of the order they were submitted. To do that, we need a combination of the `Executor.submit` method and the `futures.as_completed` function, as we saw in [Example 20-4](#). We'll come back to this technique in [“Using `futures.as\_completed`” on page 769](#).



The combination of `executor.submit` and `futures.as_completed` is more flexible than `executor.map` because you can submit different callables and arguments, while `executor.map` is designed to run the same callable on the different arguments. In addition, the set of futures you pass to `futures.as_completed` may come from more than one executor—perhaps some were created by a `ThreadPoolExecutor` instance, while others are from a `ProcessPoolExecutor`.

In the next section, we will resume the flag download examples with new requirements that will force us to iterate over the results of `futures.as_completed` instead of using `executor.map`.

Downloads with Progress Display and Error Handling

As mentioned, the scripts in “Concurrent Web Downloads” on page 744 have no error handling to make them easier to read and to contrast the structure of the three approaches: sequential, threaded, and asynchronous.

In order to test the handling of a variety of error conditions, I created the `flags2` examples:

flags2_common.py

This module contains common functions and settings used by all `flags2` examples, including a `main` function, which takes care of command-line parsing, timing, and reporting results. That is really support code, not directly relevant to the subject of this chapter, so I will not list the source code here, but you can read it in the [fluentpython/example-code-2e](#) repository: [20-executors/getflags/flags2_common.py](#).

flags2_sequential.py

A sequential HTTP client with proper error handling and progress bar display. Its `download_one` function is also used by `flags2_threadpool.py`.

flags2_threadpool.py

Concurrent HTTP client based on `futures.ThreadPoolExecutor` to demonstrate error handling and integration of the progress bar.

flags2_asyncio.py

Same functionality as the previous example, but implemented with `asyncio` and `httpx`. This will be covered in “Enhancing the `asyncio` Downloader” on page 787, in Chapter 21.



Be Careful when Testing Concurrent Clients

When testing concurrent HTTP clients on public web servers, you may generate many requests per second, and that’s how denial-of-service (DoS) attacks are made. Carefully throttle your clients when hitting public servers. For testing, set up a local HTTP server. See “Setting Up Test Servers” on page 765 for instructions.

The most visible feature of the `flags2` examples is that they have an animated, text-mode progress bar implemented with the [tqdm package](#). I posted a [108s video on YouTube](#) to show the progress bar and contrast the speed of the three `flags2` scripts. In the video, I start with the sequential download, but I interrupt it after 32s because it was going to take more than 5 minutes to hit on 676 URLs and get 194 flags. I then run the threaded and `asyncio` scripts three times each, and every time they complete

the job in 6s or less (i.e., more than 60 times faster). [Figure 20-1](#) shows two screenshots: during and after running `flags2_threadpool.py`.

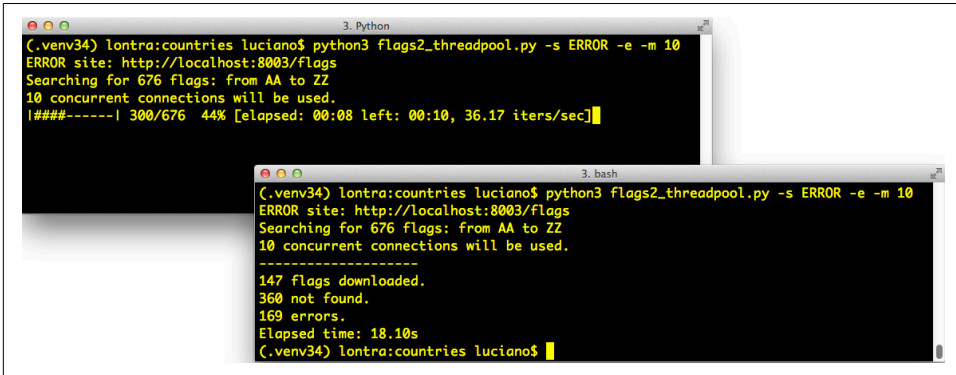


Figure 20-1. Top-left: `flags2_threadpool.py` running with live progress bar generated by `tqdm`; bottom-right: same terminal window after the script is finished.

The simplest `tqdm` example appears in an animated `.gif` in the project’s [README.md](#). If you type the following code in the Python console after installing the `tqdm` package, you’ll see an animated progress bar where the comment is:

```
>>> import time
>>> from tqdm import tqdm
>>> for i in tqdm(range(1000)):
...     time.sleep(.01)
...
>>> # -> progress bar will appear here <-
```

Besides the neat effect, the `tqdm` function is also interesting conceptually: it consumes any iterable and produces an iterator which, while it’s consumed, displays the progress bar and estimates the remaining time to complete all iterations. To compute that estimate, `tqdm` needs to get an iterable that has a `len`, or additionally receive the `total=` argument with the expected number of items. Integrating `tqdm` with our `flags2` examples provides an opportunity to look deeper into how the concurrent scripts actually work, by forcing us to use the `futures.as_completed` and the `asyncio.as_completed` functions so that `tqdm` can display progress as each future is completed.

The other feature of the `flags2` example is a command-line interface. All three scripts accept the same options, and you can see them by running any of the scripts with the `-h` option. [Example 20-10](#) shows the help text.

Example 20-10. Help screen for the scripts in the flags2 series

```
$ python3 flags2_threadpool.py -h
usage: flags2_threadpool.py [-h] [-a] [-e] [-l N] [-m CONCURRENT] [-s LABEL]
                           [-v]
                           [CC [CC ...]]
```

Download flags for country codes. Default: top 20 countries by population.

positional arguments:

CC country code or 1st letter (eg. B for BA...BZ)

optional arguments:

-h, --help show this help message and exit
-a, --all get all available flags (AD to ZW)
-e, --every get flags for every possible code (AA...ZZ)
-l N, --limit N limit to N first codes
-m CONCURRENT, --max_req CONCURRENT
maximum concurrent requests (default=30)
-s LABEL, --server LABEL
Server to hit; one of DELAY, ERROR, LOCAL, REMOTE
(default=LOCAL)
-v, --verbose output detailed progress info

All arguments are optional. But the `-s/--server` is essential for testing: it lets you choose which HTTP server and port will be used in the test. Pass one of these case-insensitive labels to determine where the script will look for the flags:

LOCAL

Use `http://localhost:8000/flags`; this is the default. You should configure a local HTTP server to answer at port 8000. See the following note for instructions.

REMOTE

Use `http://fluentpython.com/data/flags`; that is a public website owned by me, hosted on a shared server. Please do not pound it with too many concurrent requests. The *fluentpython.com* domain is handled by the [Cloudflare](#) CDN (Content Delivery Network) so you may notice that the first downloads are slower, but they get faster when the CDN cache warms up.

DELAY

Use `http://localhost:8001/flags`; a server delaying HTTP responses should be listening to port 8001. I wrote *slow_server.py* to make it easier to experiment. You'll find it in the *20-futures/getflags/* directory of the [Fluent Python code repository](#). See the following note for instructions.

ERROR

Use `http://localhost:8002/flags`; a server returning some HTTP errors should be listening on port 8002. Instructions are next.



Setting Up Test Servers

If you don't have a local HTTP server for testing, I wrote setup instructions using only Python ≥ 3.9 (no external libraries) in *20-executors/getflags/README.adoc* in the *fluentpython/example-code-2e* repository. In short, *README.adoc* describes how to use:

```
python3 -m http.server
```

The LOCAL server on port 8000

```
python3 slow_server.py
```

The DELAY server on port 8001, which adds a random delay of .5s to 5s before each response

```
python3 slow_server.py 8002 --error-rate .25
```

The ERROR server on port 8002, which in addition to the random delay, has a 25% chance of returning a “418 I’m a teapot” error response

By default, each *flags2*.py* script will fetch the flags of the 20 most populous countries from the LOCAL server (<http://localhost:8000/flags>) using a default number of concurrent connections, which varies from script to script. *Example 20-11* shows a sample run of the *flags2_sequential.py* script using all defaults. To run it, you need a local server, as explained in “Be Careful when Testing Concurrent Clients” on [page 762](#).

Example 20-11. Running flags2_sequential.py with all defaults: LOCAL site, top 20 flags, 1 concurrent connection

```
$ python3 flags2_sequential.py
LOCAL site: http://localhost:8000/flags
Searching for 20 flags: from BD to VN
1 concurrent connection will be used.
-----
20 flags downloaded.
Elapsed time: 0.10s
```

You can select which flags will be downloaded in several ways. *Example 20-12* shows how to download all flags with country codes starting with the letters A, B, or C.

Example 20-12. Run flags2_threadpool.py to fetch all flags with country codes prefixes A, B, or C from the DELAY server

```
$ python3 flags2_threadpool.py -s DELAY a b c
DELAY site: http://localhost:8001/flags
Searching for 78 flags: from AA to CZ
30 concurrent connections will be used.
```

```
-----  
43 flags downloaded.  
35 not found.  
Elapsed time: 1.72s
```

Regardless of how the country codes are selected, the number of flags to fetch can be limited with the `-l/--limit` option. **Example 20-13** demonstrates how to run exactly 100 requests, combining the `-a` option to get all flags with `-l 100`.

Example 20-13. Run `flags2_asyncio.py` to get 100 flags (`-al 100`) from the `ERROR` server, using 100 concurrent requests (`-m 100`)

```
$ python3 flags2_asyncio.py -s ERROR -al 100 -m 100  
ERROR site: http://localhost:8002/flags  
Searching for 100 flags: from AD to LK  
100 concurrent connections will be used.  
-----  
73 flags downloaded.  
27 errors.  
Elapsed time: 0.64s
```

That’s the user interface of the `flags2` examples. Let’s see how they are implemented.

Error Handling in the `flags2` Examples

The common strategy in all three examples to deal with HTTP errors is that 404 errors (not found) are handled by the function in charge of downloading a single file (`download_one`). Any other exception propagates to be handled by the `download_many` function or the supervisor coroutine—in the `asyncio` example.

Once more, we’ll start by studying the sequential code, which is easier to follow—and mostly reused by the thread pool script. **Example 20-14** shows the functions that perform the actual downloads in the `flags2_sequential.py` and `flags2_threadpool.py` scripts.

Example 20-14. `flags2_sequential.py`: basic functions in charge of downloading; both are reused in `flags2_threadpool.py`

```
from collections import Counter  
from http import HTTPStatus  
  
import httpx  
import tqdm # type: ignore ❶  
  
from flags2_common import main, save_flag, DownloadStatus ❷  
  
DEFAULT_CONCUR_REQ = 1  
MAX_CONCUR_REQ = 1
```

```

def get_flag(base_url: str, cc: str) -> bytes:
    url = f'{base_url}/{cc}/{cc}.gif'.lower()
    resp = httpx.get(url, timeout=3.1, follow_redirects=True)
    resp.raise_for_status() ❸
    return resp.content

def download_one(cc: str, base_url: str, verbose: bool = False) -> DownloadStatus:
    try:
        image = get_flag(base_url, cc)
    except httpx.HTTPStatusError as exc: ❹
        res = exc.response
        if res.status_code == HTTPStatus.NOT_FOUND:
            status = DownloadStatus.NOT_FOUND ❺
            msg = f'not found: {res.url}'
        else:
            raise ❻
    else:
        save_flag(image, f'{cc}.gif')
        status = DownloadStatus.OK
        msg = 'OK'

    if verbose: ❼
        print(cc, msg)

    return status

```

- ❶ Import the `tqdm` progress-bar display library, and tell MyPy to skip checking it.⁷
- ❷ Import a couple of functions and an Enum from the `flags2_common` module.
- ❸ Raises `HTTPStatusError` if the HTTP status code is not in `range(200, 300)`.
- ❹ `download_one` catches `HTTPStatusError` to handle HTTP code 404 specifically...
- ❺ ...by setting its local `status` to `DownloadStatus.NOT_FOUND`; `DownloadStatus` is an Enum imported from `flags2_common.py`.
- ❻ Any other `HTTPStatusError` exception is re-raised to propagate to the caller.
- ❼ If the `-v/--verbose` command-line option is set, the country code and status message are displayed; this is how you'll see progress in verbose mode.

⁷ As of September 2021, there are no type hints in the current release of `tqdm`. That's OK. The world will not end because of that. Thank Guido for optional typing!

Example 20-15 lists the sequential version of the `download_many` function. This code is straightforward, but it's worth studying to contrast with the concurrent versions coming up. Focus on how it reports progress, handles errors, and tallies downloads.

Example 20-15. `flags2_sequential.py`: the sequential implementation of `download_many`

```
def download_many(cc_list: list[str],
                  base_url: str,
                  verbose: bool,
                  _unused_concur_req: int) -> Counter[DownloadStatus]:
    counter: Counter[DownloadStatus] = Counter() ❶
    cc_iter = sorted(cc_list) ❷
    if not verbose:
        cc_iter = tqdm.tqdm(cc_iter) ❸
    for cc in cc_iter:
        try:
            status = download_one(cc, base_url, verbose) ❹
        except httpx.HTTPStatusError as exc: ❺
            error_msg = 'HTTP error {resp.status_code} - {resp.reason_phrase}'
            error_msg = error_msg.format(resp=exc.response)
        except httpx.RequestError as exc: ❻
            error_msg = f'{exc} {type(exc)}'.strip()
        except KeyboardInterrupt: ❼
            break
        else: ❽
            error_msg = ''

        if error_msg:
            status = DownloadStatus.ERROR ❾
            counter[status] += 1 ❿
            if verbose and error_msg: ⓫
                print(f'{cc} error: {error_msg}')

    return counter ⓫
```

- ❶ This Counter will tally the different download outcomes: `DownloadStatus.OK`, `DownloadStatus.NOT_FOUND`, or `DownloadStatus.ERROR`.
- ❷ `cc_iter` holds the list of the country codes received as arguments, ordered alphabetically.
- ❸ If not running in verbose mode, `cc_iter` is passed to `tqdm`, which returns an iterator yielding the items in `cc_iter` while also animating the progress bar.
- ❹ Make successive calls to `download_one`.

- ⑤ HTTP status code exceptions raised by `get_flag` and not handled by `download_one` are handled here.
- ⑥ Other network-related exceptions are handled here. Any other exception will abort the script, because the `flags2_common.main` function that calls `download_many` has no `try/except`.
- ⑦ Exit the loop if the user hits Ctrl-C.
- ⑧ If no exception escaped `download_one`, clear the error message.
- ⑨ If there was an error, set the local status accordingly.
- ⑩ Increment the counter for that status.
- ⑪ In verbose mode, display the error message for the current country code, if any.
- ⑫ Return counter so that `main` can display the numbers in the final report.

We'll now study the refactored thread pool example, *flags2_threadpool.py*.

Using `futures.as_completed`

In order to integrate the *tqdm* progress bar and handle errors on each request, the *flags2_threadpool.py* script uses `futures.ThreadPoolExecutor` with the `futures.as_completed` function we've already seen. [Example 20-16](#) is the full listing of *flags2_threadpool.py*. Only the `download_many` function is implemented; the other functions are reused from *flags2_common.py* and *flags2_sequential.py*.

Example 20-16. flags2_threadpool.py: full listing

```
from collections import Counter
from concurrent.futures import ThreadPoolExecutor, as_completed

import httpx
import tqdm # type: ignore

from flags2_common import main, DownloadStatus
from flags2_sequential import download_one ①

DEFAULT_CONCUR_REQ = 30 ②
MAX_CONCUR_REQ = 1000 ③

def download_many(cc_list: list[str],
                  base_url: str,
```

```

        verbose: bool,
        concur_req: int) -> Counter[DownloadStatus]:
counter: Counter[DownloadStatus] = Counter()
with ThreadPoolExecutor(max_workers=concur_req) as executor: ❷
    to_do_map = {} ❸
    for cc in sorted(cc_list): ❹
        future = executor.submit(download_one, cc,
                                base_url, verbose) ❺
        to_do_map[future] = cc ❻
    done_iter = as_completed(to_do_map) ❼
    if not verbose:
        done_iter = tqdm.tqdm(done_iter, total=len(cc_list)) ❽
    for future in done_iter: ❾
        try:
            status = future.result() ❿
        except httpx.HTTPStatusError as exc: ⓫
            error_msg = 'HTTP error {resp.status_code} - {resp.reason_phrase}'
            error_msg = error_msg.format(resp=exc.response)
        except httpx.RequestError as exc:
            error_msg = f'{exc} {type(exc)}'.strip()
        except KeyboardInterrupt:
            break
        else:
            error_msg = ''

        if error_msg:
            status = DownloadStatus.ERROR
            counter[status] += 1
        if verbose and error_msg:
            cc = to_do_map[future] ⓫
            print(f'{cc} error: {error_msg}')

    return counter

if __name__ == '__main__':
    main(download_many, DEFAULT_CONCUR_REQ, MAX_CONCUR_REQ)

```

- ❶ Reuse `download_one` from `flags2_sequential` (Example 20-14).
- ❷ If the `-m/--max_req` command-line option is not given, this will be the maximum number of concurrent requests, implemented as the size of the thread pool; the actual number may be smaller if the number of flags to download is smaller.
- ❸ `MAX_CONCUR_REQ` caps the maximum number of concurrent requests regardless of the number of flags to download or the `-m/--max_req` command-line option. It's a safety precaution to avoid launching too many threads with their significant memory overhead.

- ④ Create the executor with `max_workers` set to `concur_req`, computed by the `main` function as the smaller of: `MAX_CONCUR_REQ`, the length of `cc_list`, or the value of the `-m/--max_req` command-line option. This avoids creating more threads than necessary.
- ⑤ This dict will map each `Future` instance—representing one download—with the respective country code for error reporting.
- ⑥ Iterate over the list of country codes in alphabetical order. The order of the results will depend on the timing of the HTTP responses more than anything, but if the size of the thread pool (given by `concur_req`) is much smaller than `len(cc_list)`, you may notice the downloads batched alphabetically.
- ⑦ Each call to `executor.submit` schedules the execution of one callable and returns a `Future` instance. The first argument is the callable, the rest are the arguments it will receive.
- ⑧ Store the future and the country code in the dict.
- ⑨ `futures.as_completed` returns an iterator that yields futures as each task is done.
- ⑩ If not in verbose mode, wrap the result of `as_completed` with the `tqdm` function to display the progress bar; because `done_iter` has no `len`, we must tell `tqdm` what is the expected number of items as the `total=` argument, so `tqdm` can estimate the work remaining.
- ⑪ Iterate over the futures as they are completed.
- ⑫ Calling the `result` method on a future either returns the value returned by the callable, or raises whatever exception was caught when the callable was executed. This method may block waiting for a resolution, but not in this example because `as_completed` only returns futures that are done.
- ⑬ Handle the potential exceptions; the rest of this function is identical to the sequential `download_many` in [Example 20-15](#)), except for the next callout.
- ⑭ To provide context for the error message, retrieve the country code from the `to_do_map` using the current future as key. This was not necessary in the sequential version because we were iterating over the list of country codes, so we knew the current `cc`; here we are iterating over the futures.



Example 20-16 uses an idiom that is very useful with `futures.as_completed`: building a dict to map each future to other data that may be useful when the future is completed. Here the `to_do_map` maps each future to the country code assigned to it. This makes it easy to do follow-up processing with the result of the futures, despite the fact that they are produced out of order.

Python threads are well suited for I/O-intensive applications, and the `concurrent.futures` package makes it relatively simple to use for certain use cases. With `ProcessPoolExecutor`, you can also solve CPU-intensive problems on multiple cores—if the computations are “**embarrassingly parallel**”. This concludes our basic introduction to `concurrent.futures`.

Chapter Summary

We started the chapter by comparing two concurrent HTTP clients with a sequential one, demonstrating that the concurrent solutions show significant performance gains over the sequential script.

After studying the first example based on `concurrent.futures`, we took a closer look at future objects, either instances of `concurrent.futures.Future` or `asyncio.Future`, emphasizing what these classes have in common (their differences will be emphasized in **Chapter 21**). We saw how to create futures by calling `Executor.submit`, and iterate over completed futures with `concurrent.futures.as_completed`.

We then discussed the use of multiple processes with the `concurrent.futures.ProcessPoolExecutor` class, to go around the GIL and use multiple CPU cores to simplify the multicore prime checker we first saw in **Chapter 19**.

In the following section, we saw how the `concurrent.futures.ThreadPoolExecutor` works with a didactic example, launching tasks that did nothing for a few seconds, except for displaying their status with a timestamp.

Next we went back to the flag downloading examples. Enhancing them with a progress bar and proper error handling prompted further exploration of the `future.as_completed` generator function, showing a common pattern: storing futures in a dict to link further information to them when submitting, so that we can use that information when the future comes out of the `as_completed` iterator.

Further Reading

The `concurrent.futures` package was contributed by Brian Quinlan, who presented it in a great talk titled “**The Future Is Soon!**” at PyCon Australia 2010. Quinlan’s talk has no slides; he shows what the library does by typing code directly in the Python

console. As a motivating example, the presentation features a short video with XKCD cartoonist/programmer Randall Munroe making an unintended DoS attack on Google Maps to build a colored map of driving times around his city. The formal introduction to the library is [PEP 3148 - futures - execute computations asynchronously](#). In the PEP, Quinlan wrote that the `concurrent.futures` library was “heavily influenced by the Java `java.util.concurrent` package.”

For additional resources covering `concurrent.futures`, please see [Chapter 19](#). All the references that cover Python’s threading and multiprocessing in [“Concurrency with Threads and Processes” on page 734](#) also cover `concurrent.futures`.

Soapbox

Thread Avoidance

Concurrency: one of the most difficult topics in computer science (usually best avoided).

—David Beazley, Python instructor and mad scientist⁸

I agree with the apparently contradictory quotes by David Beazley and Michele Simionato at the start of this chapter.

I attended an undergraduate course about concurrency. All we did was [POSIX threads](#) programming. What I learned: I don’t want to manage threads and locks myself, for the same reason that I don’t want to manage memory allocation and deallocation. Those jobs are best carried out by the systems programmers who have the know-how, the inclination, and the time to get them right—hopefully. I am paid to develop applications, not operating systems. I don’t need all the fine-grained control of threads, locks, `malloc`, and `free`—see [“C dynamic memory allocation”](#).

That’s why I think the `concurrent.futures` package is interesting: it treats threads, processes, and queues as infrastructure at your service, not something you have to deal with directly. Of course, it’s designed with simple jobs in mind, the so-called embarrassingly parallel problems. But that’s a large slice of the concurrency problems we face when writing applications—as opposed to operating systems or database servers, as Simionato points out in that quote.

For “nonembarrassing” concurrency problems, threads and locks are not the answer either. Threads will never disappear at the OS level, but every programming language I’ve found exciting in the last several years provides higher-level, concurrency abstractions that are easier to use correctly, as the excellent [Seven Concurrency Models in Seven Weeks](#) book by Paul Butcher demonstrates. Go, Elixir, and Clojure are

⁸ Slide #9 from [“A Curious Course on Coroutines and Concurrency”](#) tutorial presented at PyCon 2009.

among them. Erlang—the implementation language of Elixir—is a prime example of a language designed from the ground up with concurrency in mind. Erlang doesn’t excite me for a simple reason: I find its syntax ugly. Python spoiled me that way.

José Valim, previously a Ruby on Rails core contributor, designed Elixir with a pleasant, modern syntax. Like Lisp and Clojure, Elixir implements syntactic macros. That’s a double-edged sword. Syntactic macros enable powerful DSLs, but the proliferation of sublanguages can lead to incompatible codebases and community fragmentation. Lisp drowned in a flood of macros, with each Lisp shop using its own arcane dialect. Standardizing around Common Lisp resulted in a bloated language. I hope José Valim can inspire the Elixir community to avoid a similar outcome. So far, it’s looking good. The **Ecto** database wrapper and query generator is a joy to use: a great example of using macros to create a flexible yet user-friendly DSL—Domain-Specific Language—for interacting with relational and nonrelational databases.

Like Elixir, Go is a modern language with fresh ideas. But, in some regards, it’s a conservative language, compared to Elixir. Go doesn’t have macros, and its syntax is simpler than Python’s. Go doesn’t support inheritance or operator overloading, and it offers fewer opportunities for metaprogramming than Python. These limitations are considered features. They lead to more predictable behavior and performance. That’s a big plus in the highly concurrent, mission-critical settings where Go aims to replace C++, Java, and Python.

While Elixir and Go are direct competitors in the high-concurrency space, their design philosophies appeal to different crowds. Both are likely to thrive. But in the history of programming languages, the conservative ones tend to attract more coders.

Asynchronous Programming

The problem with normal approaches to asynchronous programming is that they're all-or-nothing propositions. You rewrite all your code so none of it blocks or you're just wasting your time.

Alvaro Videla and Jason J. W. Williams, *RabbitMQ in Action*¹

This chapter addresses three major topics that are closely related:

- Python's `async def`, `await`, `async with`, and `async for` constructs
- Objects supporting those constructs: native coroutines and asynchronous variants of context managers, iterables, generators, and comprehensions
- *asyncio* and other asynchronous libraries

This chapter builds on the ideas of iterables and generators ([Chapter 17](#), in particular “[Classic Coroutines](#)” on page 641), context managers ([Chapter 18](#)), and general concepts of concurrent programming ([Chapter 19](#)).

We'll study concurrent HTTP clients similar to the ones we saw in [Chapter 20](#), rewritten with native coroutines and asynchronous context managers, using the same *HTTPX* library as before, but now through its asynchronous API. We'll also see how to avoid blocking the event loop by delegating slow operations to a thread or process executor.

After the HTTP client examples, we'll see two simple asynchronous server-side applications, one of them using the increasingly popular *FastAPI* framework. Then we'll cover other language constructs enabled by the `async/await` keywords:

¹ Videla & Williams, *RabbitMQ in Action* (Manning), Chapter 4, “Solving Problems with Rabbit: coding and patterns,” p. 61.

asynchronous generator functions, asynchronous comprehensions, and asynchronous generator expressions. To emphasize the fact that those language features are not tied to *asyncio*, we'll see one example rewritten to use *Curio*—the elegant and innovative asynchronous framework invented by David Beazley.

To wrap up the chapter, I wrote a brief section on the advantages and pitfalls of asynchronous programming.

That's a lot of ground to cover. We only have space for basic examples, but they will illustrate the most important features of each idea.



The *asyncio* documentation is much better after Yury Selivanov² reorganized it, separating the few functions useful to application developers from the low-level API for creators of packages like web frameworks and database drivers.

For book-length coverage of *asyncio*, I recommend *Using Asyncio in Python* by Caleb Hattingh (O'Reilly). Full disclosure: Caleb is one of the tech reviewers of this book.

What's New in This Chapter

When I wrote the first edition of *Fluent Python*, the *asyncio* library was provisional and the `async/await` keywords did not exist. Therefore, I had to update all examples in this chapter. I also created new examples: domain probing scripts, a *FastAPI* web service, and experiments with Python's new asynchronous console mode.

New sections cover language features that did not exist at the time, such as native coroutines, `async with`, `async for`, and the objects that support those constructs.

The ideas in “*How Async Works and How It Doesn't*” on page 825 reflect hard-earned lessons that I consider essential reading for anyone using asynchronous programming. They may save you a lot of trouble—whether you're using Python or Node.js.

Finally, I removed several paragraphs about `asyncio.Futures`, which is now considered part of the low-level *asyncio* APIs.

² Selivanov implemented `async/await` in Python, and wrote the related PEPs 492, 525, and 530.

A Few Definitions

At the start of “[Classic Coroutines](#)” on page 641, we saw that Python 3.5 and later offer three kinds of coroutines:

Native coroutine

A coroutine function defined with `async def`. You can delegate from a native coroutine to another native coroutine using the `await` keyword, similar to how classic coroutines use `yield from`. The `async def` statement always defines a native coroutine, even if the `await` keyword is not used in its body. The `await` keyword cannot be used outside of a native coroutine.³

Classic coroutine

A generator function that consumes data sent to it via `my_coro.send(data)` calls, and reads that data by using `yield` in an expression. Classic coroutines can delegate to other classic coroutines using `yield from`. Classic coroutines cannot be driven by `await`, and are no longer supported by *asyncio*.

Generator-based coroutine

A generator function decorated with `@types.coroutine`—introduced in Python 3.5. That decorator makes the generator compatible with the new `await` keyword.

In this chapter, we focus on native coroutines as well as *asynchronous generators*:

Asynchronous generator

A generator function defined with `async def` and using `yield` in its body. It returns an asynchronous generator object that provides `__anext__`, a coroutine method to retrieve the next item.



@asyncio.coroutine has No Future⁴

The `@asyncio.coroutine` decorator for classic coroutines and generator-based coroutines was deprecated in Python 3.8 and is scheduled for removal in Python 3.11, according to [Issue 43216](#). In contrast, `@types.coroutine` should remain, per [Issue 36921](#). It is no longer supported by *asyncio*, but is used in low-level code in the *Curio* and *Trio* asynchronous frameworks.

³ There is one exception to this rule: if you run Python with the `-m asyncio` option, you can use `await` directly at the `>>>` prompt to drive a native coroutine. This is explained in “[Experimenting with Python’s async console](#)” on page 812.

⁴ Sorry, I could not resist it.

An asyncio Example: Probing Domains

Imagine you are about to start a new blog on Python, and you plan to register a domain using a Python keyword and the *.DEV* suffix—for example: *AWAIT.DEV*. **Example 21-1** is a script using *asyncio* to check several domains concurrently. This is the output it produces:

```
$ python3 blogdom.py
with.dev
+ elif.dev
+ def.dev
  from.dev
  else.dev
  or.dev
  if.dev
  del.dev
+ as.dev
  none.dev
  pass.dev
  true.dev
+ in.dev
+ for.dev
+ is.dev
+ and.dev
+ try.dev
+ not.dev
```

Note that the domains appear unordered. If you run the script, you'll see them displayed one after the other, with varying delays. The + sign indicates your machine was able to resolve the domain via DNS. Otherwise, the domain did not resolve and may be available.⁵

In *blogdom.py*, the DNS probing is done via native coroutine objects. Because the asynchronous operations are interleaved, the time needed to check the 18 domains is much less than checking them sequentially. In fact, the total time is practically the same as the time for the single slowest DNS response, instead of the sum of the times of all responses.

Example 21-1 shows the code for *blogdom.py*.

Example 21-1. blogdom.py: search for domains for a Python blog

```
#!/usr/bin/env python3
import asyncio
import socket
```

⁵ true.dev is available for USD 360/year as I write this. I see that for.dev is registered, but has no DNS configured.

```

from keyword import kwlist

MAX_KEYWORD_LEN = 4 ❶

async def probe(domain: str) -> tuple[str, bool]: ❷
    loop = asyncio.get_running_loop() ❸
    try:
        await loop.getaddrinfo(domain, None) ❹
    except socket.gaierror:
        return (domain, False)
    return (domain, True)

async def main() -> None: ❺
    names = (kw for kw in kwlist if len(kw) <= MAX_KEYWORD_LEN) ❻
    domains = (f'{name}.dev'.lower() for name in names) ❼
    coros = [probe(domain) for domain in domains] ❽
    for coro in asyncio.as_completed(coros): ❾
        domain, found = await coro ❿
        mark = '+' if found else ' '
        print(f'{mark} {domain}')

if __name__ == '__main__':
    asyncio.run(main()) ⓫

```

- ❶ Set maximum length of keyword for domains, because shorter is better.
- ❷ `probe` returns a tuple with the domain name and a boolean; `True` means the domain resolved. Returning the domain name will make it easier to display the results.
- ❸ Get a reference to the `asyncio` event loop, so we can use it next.
- ❹ The `loop.getaddrinfo(...)` coroutine-method returns a **five-part tuple of parameters** to connect to the given address using a socket. In this example, we don't need the result. If we got it, the domain resolves; otherwise, it doesn't.
- ❺ `main` must be a coroutine, so that we can use `await` in it.
- ❻ Generator to yield Python keywords with length up to `MAX_KEYWORD_LEN`.
- ❼ Generator to yield domain names with the `.dev` suffix.
- ❽ Build a list of coroutine objects by invoking the `probe` coroutine with each domain argument.

- 9 `asyncio.as_completed` is a generator that yields coroutines that return the results of the coroutines passed to it in the order they are completed—not the order they were submitted. It’s similar to `futures.as_completed`, which we saw in [Chapter 20, Example 20-4](#).
- 10 At this point, we know the coroutine is done because that’s how `as_completed` works. Therefore, the `await` expression will not block but we need it to get the result from `coro`. If `coro` raised an unhandled exception, it would be re-raised here.
- 11 `asyncio.run` starts the event loop and returns only when the event loop exits. This is a common pattern for scripts that use `asyncio`: implement `main` as a coroutine, and drive it with `asyncio.run` inside the `if __name__ == '__main__':` block.



The `asyncio.get_running_loop` function was added in Python 3.7 for use inside coroutines, as shown in `probe`. If there’s no running loop, `asyncio.get_running_loop` raises `RuntimeError`. Its implementation is simpler and faster than `asyncio.get_event_loop`, which may start an event loop if necessary. Since Python 3.10, `asyncio.get_event_loop` is **deprecated** and will eventually become an alias to `asyncio.get_running_loop`.

Guido’s Trick to Read Asynchronous Code

There are a lot of new concepts to grasp in *asyncio*, but the overall logic of [Example 21-1](#) is easy to follow if you employ a trick suggested by Guido van Rossum himself: squint and pretend the `async` and `await` keywords are not there. If you do that, you’ll realize that coroutines read like plain old sequential functions.

For example, imagine that the body of this coroutine...

```
async def probe(domain: str) -> tuple[str, bool]:
    loop = asyncio.get_running_loop()
    try:
        await loop.getaddrinfo(domain, None)
    except socket.gaierror:
        return (domain, False)
    return (domain, True)
```

...works like the following function, except that it magically never blocks:

```
def probe(domain: str) -> tuple[str, bool]: # no async
    loop = asyncio.get_running_loop()
    try:
        loop.getaddrinfo(domain, None) # no await
```

```
except socket.gaierror:
    return (domain, False)
return (domain, True)
```

Using the syntax `await loop.getaddrinfo(...)` avoids blocking because `await` suspends the current coroutine object. For example, during the execution of the `probe('if.dev')` coroutine, a new coroutine object is created by `getaddrinfo('if.dev', None)`. Awaiting it starts the low-level `addrinfo` query and yields control back to the event loop, not to the `probe('if.dev')` coroutine, which is suspended. The event loop can then drive other pending coroutine objects, such as `probe('or.dev')`.

When the event loop gets a response for the `getaddrinfo('if.dev', None)` query, that specific coroutine object resumes and returns control back to the `probe('if.dev')`—which was suspended at `await`—and can now handle a possible exception and return the result tuple.

So far, we've only seen `asyncio.as_completed` and `await` applied to coroutines. But they handle any *awaitable* object. That concept is explained next.

New Concept: Awaitable

The `for` keyword works with *iterables*. The `await` keyword works with *awaitables*.

As an end user of *asyncio*, these are the awaitables you will see on a daily basis:

- A *native coroutine object*, which you get by calling a *native coroutine function*
- An `asyncio.Task`, which you usually get by passing a coroutine object to `asyncio.create_task()`

However, end-user code does not always need to `await` on a `Task`. We use `asyncio.create_task(one_coro())` to schedule `one_coro` for concurrent execution, without waiting for its return. That's what we did with the `spinner` coroutine in *spinner_async.py* (Example 19-4). If you don't expect to cancel the task or wait for it, there is no need to keep the `Task` object returned from `create_task`. Creating the task is enough to schedule the coroutine to run.

In contrast, we use `await other_coro()` to run `other_coro` right now and wait for its completion because we need its result before we can proceed. In *spinner_async.py*, the supervisor coroutine did `res = await slow()` to execute `slow` and get its result.

When implementing asynchronous libraries or contributing to *asyncio* itself, you may also deal with these lower-level awaitables:

- An object with an `__await__` method that returns an iterator; for example, an `asyncio.Future` instance (`asyncio.Task` is a subclass of `asyncio.Future`)
- Objects written in other languages using the Python/C API with a `tp_as_async.am_await` function, returning an iterator (similar to `__await__` method)

Existing codebases may also have one additional kind of awaitable: *generator-based coroutine objects*—which are in the process of being deprecated.



PEP 492 [states](#) that the `await` expression “uses the `yield from` implementation with an extra step of validating its argument” and “`await` only accepts an awaitable.” The PEP does not explain that implementation in detail, but refers to [PEP 380](#), which introduced `yield from`. I posted a detailed explanation in “[Classic Coroutines](#)”, section “[The Meaning of `yield from`](#)”, at [fluentpython.com](#).

Now let’s study the *asyncio* version of a script that downloads a fixed set of flag images.

Downloading with *asyncio* and HTTPX

The *flags_asyncio.py* script downloads a fixed set of 20 flags from [fluentpython.com](#). We first mentioned it in “[Concurrent Web Downloads](#)” on [page 744](#), but now we’ll study it in detail, applying the concepts we just saw.

As of Python 3.10, *asyncio* only supports TCP and UDP directly, and there are no asynchronous HTTP client or server packages in the standard library. I am using *HTTPX* in all the HTTP client examples.

We’ll explore *flags_asyncio.py* from the bottom up—that is, looking first at the functions that set up the action in [Example 21-2](#).



To make the code easier to read, *flags_asyncio.py* has no error handling. As we introduce `async/await`, it’s useful to focus on the “happy path” initially, to understand how regular functions and coroutines are arranged in a program. Starting with “[Enhancing the *asyncio* Downloader](#)” on [page 787](#), the examples include error handling and more features.

The *flags.py* examples from this chapter and [Chapter 20](#) share code and data, so I put them together in the *example-code-2e/20-executors/getflags* directory.

Example 21-2. *flags_asyncio.py*: startup functions

```
def download_many(cc_list: list[str]) -> int: ❶
    return asyncio.run(supervisor(cc_list)) ❷

async def supervisor(cc_list: list[str]) -> int: ❸
    async with AsyncClient() as client:
        to_do = [download_one(client, cc)
                  for cc in sorted(cc_list)] ❹
        res = await asyncio.gather(*to_do) ❺

    return len(res) ❻

if __name__ == '__main__':
    main(download_many)
```

- ❶ This needs to be a plain function—not a coroutine—so it can be passed to and called by the `main` function from the *flags.py* module (Example 20-2).
- ❷ Execute the event loop driving the `supervisor(cc_list)` coroutine object until it returns. This will block while the event loop runs. The result of this line is whatever `supervisor` returns.
- ❸ Asynchronous HTTP client operations in `httpx` are methods of `AsyncClient`, which is also an asynchronous context manager: a context manager with asynchronous setup and teardown methods (more about this in “Asynchronous Context Managers” on page 786).
- ❹ Build a list of coroutine objects by calling the `download_one` coroutine once for each flag to be retrieved.
- ❺ Wait for the `asyncio.gather` coroutine, which accepts one or more awaitable arguments and waits for all of them to complete, returning a list of results for the given awaitables in the order they were submitted.
- ❻ `supervisor` returns the length of the list returned by `asyncio.gather`.

Now let’s review the top of *flags_asyncio.py* (Example 21-3). I reorganized the coroutines so we can read them in the order they are started by the event loop.

Example 21-3. *flags_asyncio.py*: imports and download functions

```
import asyncio

from httpx import AsyncClient ❶

from flags import BASE_URL, save_flag, main ❷
```

```

async def download_one(client: AsyncClient, cc: str): ❸
    image = await get_flag(client, cc)
    save_flag(image, f'{cc}.gif')
    print(cc, end=' ', flush=True)
    return cc

async def get_flag(client: AsyncClient, cc: str) -> bytes: ❹
    url = f'{BASE_URL}/{cc}/{cc}.gif'.lower()
    resp = await client.get(url, timeout=6.1,
                            follow_redirects=True) ❺
    return resp.read() ❻

```

- ❶ httpx must be installed—it’s not in the standard library.
- ❷ Reuse code from *flags.py* (Example 20-2).
- ❸ `download_one` must be a native coroutine, so it can `await` on `get_flag`—which does the HTTP request. Then it displays the code of the downloaded flag, and saves the image.
- ❹ `get_flag` needs to receive the `AsyncClient` to make the request.
- ❺ The `get` method of an `httpx.AsyncClient` instance returns a `ClientResponse` object that is also an asynchronous context manager.
- ❻ Network I/O operations are implemented as coroutine methods, so they are driven asynchronously by the `asyncio` event loop.



For better performance, the `save_flag` call inside `get_flag` should be asynchronous, to avoid blocking the event loop. However, *asyncio* does not provide an asynchronous filesystem API at this time—as Node.js does.

“Using `asyncio.as_completed` and a Thread” on page 788 will show how to delegate `save_flag` to a thread.

Your code delegates to the `httpx` coroutines explicitly through `await` or implicitly through the special methods of the asynchronous context managers, such as `AsyncClient` and `ClientResponse`—as we’ll see in “Asynchronous Context Managers” on page 786.

The Secret of Native Coroutines: Humble Generators

A key difference between the classic coroutine examples we saw in “Classic Coroutines” on page 641 and *flags_asyncio.py* is that there are no visible `.send()` calls or

yield expressions in the latter. Your code sits between the *asyncio* library and the asynchronous libraries you are using, such as *HTTPX*. This is illustrated in Figure 21-1.

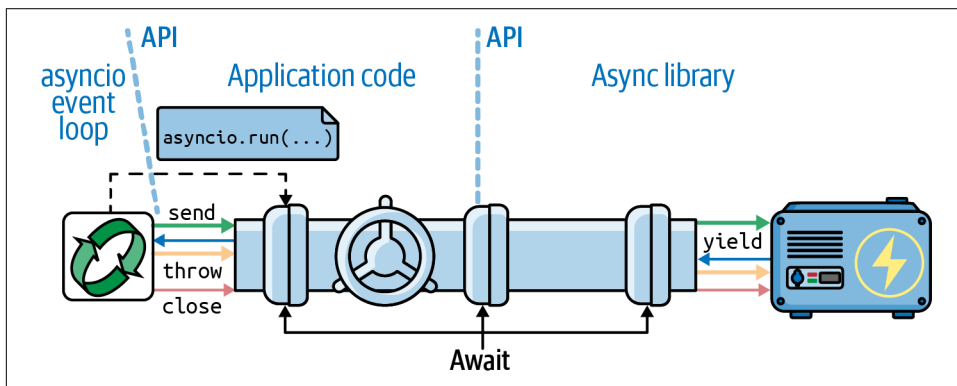


Figure 21-1. In an asynchronous program, a user’s function starts the event loop, scheduling an initial coroutine with `asyncio.run`. Each user’s coroutine drives the next with an `await` expression, forming a channel that enables communication between a library like *HTTPX* and the event loop.

Under the hood, the `asyncio` event loop makes the `.send` calls that drive your coroutines, and your coroutines `await` on other coroutines, including library coroutines. As mentioned, `await` borrows most of its implementation from `yield from`, which also makes `.send` calls to drive coroutines.

The `await` chain eventually reaches a low-level awaitable, which returns a generator that the event loop can drive in response to events such as timers or network I/O. The low-level awaitables and generators at the end of these `await` chains are implemented deep into the libraries, are not part of their APIs, and may be Python/C extensions.

Using functions like `asyncio.gather` and `asyncio.create_task`, you can start multiple concurrent `await` channels, enabling concurrent execution of multiple I/O operations driven by a single event loop, in a single thread.

The All-or-Nothing Problem

Note that in Example 21-3, I could not reuse the `get_flag` function from *flags.py* (Example 20-2). I had to rewrite it as a coroutine to use the asynchronous API of *HTTPX*. For peak performance with *asyncio*, we must replace every function that does I/O with an asynchronous version that is activated with `await` or `asyncio.create_task`, so that control is given back to the event loop while the function waits for I/O. If you can’t rewrite a blocking function as a coroutine, you should run it in a separate thread or process, as we’ll see in “Delegating Tasks to Executors” on page 797.

That’s why I chose the epigraph for this chapter, which includes this advice: “You rewrite all your code so none of it blocks or you’re just wasting your time.”

For the same reason, I could not reuse the `download_one` function from *flags_thread-pool.py* (Example 20-3) either. The code in Example 21-3 drives `get_flag` with `await`, so `download_one` must also be a coroutine. For each request, a `download_one` coroutine object is created in supervisor, and they are all driven by the `asyncio.gather` coroutine.

Now let’s study the `async with` statement that appeared in supervisor (Example 21-2) and `get_flag` (Example 21-3).

Asynchronous Context Managers

In “Context Managers and with Blocks” on page 658, we saw how an object can be used to run code before and after the body of a `with` block, if its class provides the `__enter__` and `__exit__` methods.

Now, consider Example 21-4, from the *asyncpg* *asyncio*-compatible PostgreSQL driver [documentation on transactions](#).

Example 21-4. Sample code from the documentation of the asyncpg PostgreSQL driver

```
tr = connection.transaction()
await tr.start()
try:
    await connection.execute("INSERT INTO mytable VALUES (1, 2, 3)")
except:
    await tr.rollback()
    raise
else:
    await tr.commit()
```

A database transaction is a natural fit for the context manager protocol: the transaction has to be started, data is changed with `connection.execute`, and then a rollback or commit must happen, depending on the outcome of the changes.

In an asynchronous driver like *asyncpg*, the setup and wrap-up need to be coroutines so that other operations can happen concurrently. However, the implementation of the classic `with` statement doesn’t support coroutines doing the work of `__enter__` or `__exit__`.

That’s why [PEP 492—Coroutines with `async` and `await` syntax](#) introduced the `async with` statement, which works with asynchronous context managers: objects implementing the `__aenter__` and `__aexit__` methods as coroutines.

With `async with`, [Example 21-4](#) can be written like this other snippet from the *asyncpg* documentation:

```
async with connection.transaction():
    await connection.execute("INSERT INTO mytable VALUES (1, 2, 3)")
```

In the *asyncpg.Transaction* class, the `__aenter__` coroutine method does `await self.start()`, and the `__aexit__` coroutine awaits on private `__rollback` or `__commit` coroutine methods, depending on whether an exception occurred or not. Using coroutines to implement *Transaction* as an asynchronous context manager allows *asyncpg* to handle many transactions concurrently.



Caleb Hattingh on *asyncpg*

Another really great thing about *asyncpg* is that it also works around PostgreSQL’s lack of high-concurrency support (it uses one server-side process per connection) by implementing a connection pool for internal connections to Postgres itself.

This means you don’t need additional tools like *pgbouncer* as explained in the *asyncpg* [documentation](#).⁶

Back to *flags_asyncio.py*, the *AsyncClient* class of *httpx* is an asynchronous context manager, so it can use awaitables in its `__aenter__` and `__aexit__` special coroutine methods.



“Asynchronous generators as context managers” on page 817 shows how to use Python’s `contextlib` to create an asynchronous context manager without having to write a class. That explanation comes later in this chapter because of a prerequisite topic: “Asynchronous Generator Functions” on page 812.

We’ll now enhance the *asyncio* flag download example with a progress bar, which will lead us to explore a bit more of the *asyncio* API.

Enhancing the *asyncio* Downloader

Recall from “Downloads with Progress Display and Error Handling” on page 762 that the `flags2` set of examples share the same command-line interface, and they display a progress bar while the downloads are happening. They also include error handling.

⁶ This tip is quoted verbatim from a comment by tech reviewer Caleb Hattingh. Thanks, Caleb!

Fortunately, there is an `asyncio` equivalent of the `as_completed` generator function we used in the thread pool example with the progress bar (Example 20-16).

Example 21-6 shows the top of the `flags2_asyncio.py` script where the `get_flag` and `download_one` coroutines are defined. Example 21-7 lists the rest of the source, with `supervisor` and `download_many`. This script is longer than `flags_asyncio.py` because of error handling.

Example 21-6. `flags2_asyncio.py`: top portion of the script; remaining code is in Example 21-7

```
import asyncio
from collections import Counter
from http import HTTPStatus
from pathlib import Path

import httpx
import tqdm # type: ignore

from flags2_common import main, DownloadStatus, save_flag

# low concurrency default to avoid errors from remote site,
# such as 503 - Service Temporarily Unavailable
DEFAULT_CONCUR_REQ = 5
MAX_CONCUR_REQ = 1000

async def get_flag(client: httpx.AsyncClient, ❶
                  base_url: str,
                  cc: str) -> bytes:
    url = f'{base_url}/{cc}/{cc}.gif'.lower()
    resp = await client.get(url, timeout=3.1, follow_redirects=True) ❷
    resp.raise_for_status()
    return resp.content

async def download_one(client: httpx.AsyncClient,
                      cc: str,
                      base_url: str,
                      semaphore: asyncio.Semaphore,
                      verbose: bool) -> DownloadStatus:
    try:
        async with semaphore: ❸
            image = await get_flag(client, base_url, cc)
    except httpx.HTTPStatusError as exc: ❹
        res = exc.response
        if res.status_code == HTTPStatus.NOT_FOUND:
            status = DownloadStatus.NOT_FOUND
            msg = f'not found: {res.url}'
        else:
            raise
    else:
        return
```

```

        await asyncio.to_thread(save_flag, image, f'{cc}.gif') ❸
        status = DownloadStatus.OK
        msg = 'OK'
    if verbose and msg:
        print(cc, msg)
    return status

```

- ❶ `get_flag` is very similar to the sequential version in [Example 20-14](#). First difference: it requires the `client` parameter.
- ❷ Second and third differences: `.get` is an `AsyncClient` method, and it's a coroutine, so we need to `await` it.
- ❸ Use the semaphore as an asynchronous context manager so that the program as a whole is not blocked; only this coroutine is suspended when the semaphore counter is zero. More about this in [“Python’s Semaphores” on page 791](#).
- ❹ The error handling logic is the same as in `download_one`, from [Example 20-14](#).
- ❺ Saving the image is an I/O operation. To avoid blocking the event loop, run `save_flag` in a thread.

All network I/O is done with coroutines in *asyncio*, but not file I/O. However, file I/O is also “blocking”—in the sense that reading/writing files takes **thousands of times longer** than reading/writing to RAM. If you’re using [Network-Attached Storage](#), it may even involve network I/O under the covers.

Since Python 3.9, the `asyncio.to_thread` coroutine makes it easy to delegate file I/O to a thread pool provided by *asyncio*. If you need to support Python 3.7 or 3.8, [“Delegating Tasks to Executors” on page 797](#) shows how to add a couple of lines to do it. But first, let’s finish our study of the HTTP client code.

Throttling Requests with a Semaphore

Network clients like the ones we are studying should be *throttled* (i.e., limited) to avoid pounding the server with too many concurrent requests.

A *semaphore* is a synchronization primitive, more flexible than a lock. A semaphore can be held by multiple coroutines, with a configurable maximum number. This makes it ideal to throttle the number of active concurrent coroutines. [“Python’s Semaphores” on page 791](#) has more information.

In *flags2_threadpool.py* ([Example 20-16](#)), the throttling was done by instantiating the `ThreadPoolExecutor` with the required `max_workers` argument set to `concur_req` in the `download_many` function. In *flags2_asyncio.py*, an `asyncio.Semaphore` is created

by the supervisor function (shown in [Example 21-7](#)) and passed as the semaphore argument to `download_one` in [Example 21-6](#).

Python's Semaphores

Computer scientist Edsger W. Dijkstra invented the **semaphore** in the early 1960s. It's a simple idea, but it's so flexible that most other synchronization objects—such as locks and barriers—can be built on top of semaphores. There are three Semaphore classes in Python's standard library: one in `threading`, another in `multiprocessing`, and a third one in `asyncio`. Here we'll describe the latter.

An `asyncio.Semaphore` has an internal counter that is decremented whenever we await on the `.acquire()` coroutine method, and incremented when we call the `.release()` method—which is not a coroutine because it never blocks. The initial value of the counter is set when the Semaphore is instantiated:

```
semaphore = asyncio.Semaphore(concur_req)
```

Awaiting on `.acquire()` causes no delay when the counter is greater than zero, but if the counter is zero, `.acquire()` suspends the awaiting coroutine until some other coroutine calls `.release()` on the same Semaphore, thus incrementing the counter. Instead of using those methods directly, it's safer to use the semaphore as an asynchronous context manager, as I did in [Example 21-6](#), function `download_one`:

```
async with semaphore:
    image = await get_flag(client, base_url, cc)
```

The Semaphore `__aenter__` coroutine method awaits for `.acquire()`, and its `__aexit__` coroutine method calls `.release()`. That snippet guarantees that no more than `concur_req` instances of `get_flags` coroutines will be active at any time.

Each of the Semaphore classes in the standard library has a `BoundedSemaphore` subclass that enforces an additional constraint: the internal counter can never become larger than the initial value when there are more `.release()` than `.acquire()` operations.⁷

Now let's take a look at the rest of the script in [Example 21-7](#).

Example 21-7. `flags2_asyncio.py`: script continued from [Example 21-6](#)

```
async def supervisor(cc_list: list[str],
                    base_url: str,
```

⁷ Thanks to Guto Maia who noted that the concept of a semaphore was not explained when he read the first edition draft for this chapter.

```

        verbose: bool,
        concur_req: int) -> Counter[DownloadStatus]: ❶
counter: Counter[DownloadStatus] = Counter()
semaphore = asyncio.Semaphore(concur_req) ❷
async with httpx.AsyncClient() as client:
    to_do = [download_one(client, cc, base_url, semaphore, verbose)
              for cc in sorted(cc_list)] ❸
    to_do_iter = asyncio.as_completed(to_do) ❹
    if not verbose:
        to_do_iter = tqdm.tqdm(to_do_iter, total=len(cc_list)) ❺
    error: httpx.HTTPError | None = None ❻
    for coro in to_do_iter: ❼
        try:
            status = await coro ❽
        except httpx.HTTPStatusError as exc:
            error_msg = 'HTTP error {resp.status_code} - {resp.reason_phrase}'
            error_msg = error_msg.format(resp=exc.response)
            error = exc ❾
        except httpx.RequestError as exc:
            error_msg = f'{exc} {type(exc)}'.strip()
            error = exc ❿
        except KeyboardInterrupt:
            break

    if error:
        status = DownloadStatus.ERROR ⓫
        if verbose:
            url = str(error.request.url) ⓬
            cc = Path(url).stem.upper() ⓭
            print(f'{cc} error: {error_msg}')
        counter[status] += 1

    return counter

def download_many(cc_list: list[str],
                  base_url: str,
                  verbose: bool,
                  concur_req: int) -> Counter[DownloadStatus]:
    coro = supervisor(cc_list, base_url, verbose, concur_req)
    counts = asyncio.run(coro) ⓮

    return counts

if __name__ == '__main__':
    main(download_many, DEFAULT_CONCUR_REQ, MAX_CONCUR_REQ)

```

- ❶ supervisor takes the same arguments as the `download_many` function, but it cannot be invoked directly from `main` because it's a coroutine and not a plain function like `download_many`.

- ❷ Create an `asyncio.Semaphore` that will not allow more than `concur_req` active coroutines among those using this semaphore. The value of `concur_req` is computed by the `main` function from *flags2_common.py*, based on command-line options and constants set in each example.
- ❸ Create a list of coroutine objects, one per call to the `download_one` coroutine.
- ❹ Get an iterator that will return coroutine objects as they are done. I did not place this call to `as_completed` directly in the `for` loop below because I may need to wrap it with the `tqdm` iterator for the progress bar, depending on the user's choice for verbosity.
- ❺ Wrap the `as_completed` iterator with the `tqdm` generator function to display progress.
- ❻ Declare and initialize `error` with `None`; this variable will be used to hold an exception beyond the `try/except` statement, if one is raised.
- ❼ Iterate over the completed coroutine objects; this loop is similar to the one in `download_many` in [Example 20-16](#).
- ❽ `await` on the coroutine to get its result. This will not block because `as_completed` only produces coroutines that are done.
- ❾ This assignment is necessary because the `exc` variable scope is limited to this `except` clause, but I need to preserve its value for later.
- ❿ Same as before.
- ⓫ If there was an error, set the status.
- ⓬ In verbose mode, extract the URL from the exception that was raised...
- ⓭ ...and extract the name of the file to display the country code next.
- ⓮ `download_many` instantiates the supervisor coroutine object and passes it to the event loop with `asyncio.run`, collecting the counter supervisor returns when the event loop ends.

In [Example 21-7](#), we could not use the mapping of futures to country codes we saw in [Example 20-16](#), because the awaitables returned by `asyncio.as_completed` are the same awaitables we pass into the `as_completed` call. Internally, the *asyncio*

machinery may replace the awaitables we provide with others that will, in the end, produce the same results.⁸



Because I could not use the awaitables as keys to retrieve the country code from a `dict` in case of failure, I had to extract the country code from the exception. To do that, I kept the exception in the error variable to retrieve outside of the `try/except` statement. Python is not a block-scoped language: statements such as loops and `try/except` don't create a local scope in the blocks they manage. But if an `except` clause binds an exception to a variable, like the `exc` variables we just saw—that binding only exists within the block inside that particular `except` clause.

This wraps up the discussion of an *asyncio* example functionally equivalent to the *flags2_threadpool.py* we saw earlier.

The next example demonstrates the simple pattern of executing one asynchronous task after another using coroutines. This deserves our attention because anyone with previous experience with JavaScript knows that running one asynchronous function after the other was the reason for the nested coding pattern known as *pyramid of doom*. The `await` keyword makes that curse go away. That's why `await` is now part of Python and JavaScript.

Making Multiple Requests for Each Download

Suppose you want to save each country flag with the name of the country and the country code, instead of just the country code. Now you need to make two HTTP requests per flag: one to get the flag image itself, the other to get the *metadata.json* file in the same directory as the image—that's where the name of the country is recorded.

Coordinating multiple requests in the same task is easy in the threaded script: just make one request then the other, blocking the thread twice, and keeping both pieces of data (country code and name) in local variables, ready to use when saving the files. If you needed to do the same in an asynchronous script with callbacks, you needed nested functions so that the country code and name were available in their closures until you could save the file, because each callback runs in a different local scope. The `await` keyword provides relief from that, allowing you to drive the asynchronous requests one after the other, sharing the local scope of the driving coroutine.

⁸ A detailed discussion about this can be found in a thread I started in the python-tulip group, titled “Which other futures may come out of `asyncio.as_completed?`”. Guido responds, and gives insight on the implementation of `as_completed`, as well as the close relationship between futures and coroutines in *asyncio*.



If you are doing asynchronous application programming in modern Python with lots of callbacks, you are probably applying old patterns that don't make sense in modern Python. That is justified if you are writing a library that interfaces with legacy or low-level code that does not support coroutines. Anyway, the StackOverflow Q&A, “[What is the use case for `future.add\_done\_callback\(\)`?](#)” explains why callbacks are needed in low-level code, but are not very useful in Python application-level code these days.

The third variation of the `asyncio` flag downloading script has a few changes:

`get_country`

This new coroutine fetches the `metadata.json` file for the country code, and gets the name of the country from it.

`download_one`

This coroutine now uses `await` to delegate to `get_flag` and the new `get_country` coroutine, using the result of the latter to build the name of the file to save.

Let's start with the code for `get_country` (Example 21-8). Note that it is very similar to `get_flag` from Example 21-6.

Example 21-8. `flags3_asyncio.py`: `get_country` coroutine

```
async def get_country(client: httpx.AsyncClient,
                    base_url: str,
                    cc: str) -> str: ❶
    url = f'{base_url}/{cc}/metadata.json'.lower()
    resp = await client.get(url, timeout=3.1, follow_redirects=True)
    resp.raise_for_status()
    metadata = resp.json() ❷
    return metadata['country'] ❸
```

- ❶ This coroutine returns a string with the country name—if all goes well.
- ❷ `metadata` will get a Python dict built from the JSON contents of the response.
- ❸ Return the country name.

Now let's see the modified `download_one` in Example 21-9, which has only a few lines changed from the same coroutine in Example 21-6.

Example 21-9. *flags3_asyncio.py*: *download_one* coroutine

```
async def download_one(client: httpx.AsyncClient,
                        cc: str,
                        base_url: str,
                        semaphore: asyncio.Semaphore,
                        verbose: bool) -> DownloadStatus:
    try:
        async with semaphore: ❶
            image = await get_flag(client, base_url, cc)
        async with semaphore: ❷
            country = await get_country(client, base_url, cc)
    except httpx.HTTPStatusError as exc:
        res = exc.response
        if res.status_code == HTTPStatus.NOT_FOUND:
            status = DownloadStatus.NOT_FOUND
            msg = f'not found: {res.url}'
        else:
            raise
    else:
        filename = country.replace(' ', '_') ❸
        await asyncio.to_thread(save_flag, image, f'{filename}.gif')
        status = DownloadStatus.OK
        msg = 'OK'
    if verbose and msg:
        print(cc, msg)
    return status
```

- ❶ Hold the semaphore to await for `get_flag`...
- ❷ ...and again for `get_country`.
- ❸ Use the country name to create a filename. As a command-line user, I don't like to see spaces in filenames.

Much better than nested callbacks!

I put the calls to `get_flag` and `get_country` in separate `with` blocks controlled by the semaphore because it's good practice to hold semaphores and locks for the shortest possible time.

I could schedule both `get_flag` and `get_country` in parallel using `asyncio.gather`, but if `get_flag` raises an exception, there is no image to save, so it's pointless to run `get_country`. But there are cases where it makes sense to use `asyncio.gather` to hit several APIs at the same time instead of waiting for one response before making the next request.

In *flags3_asyncio.py*, the `await` syntax appears six times, and `async with` three times. Hopefully, you should be getting the hang of asynchronous programming in Python.

One challenge is to know when you have to use `await` and when you can't use it. The answer in principle is easy: you `await` coroutines and other awaitables, such as `asyncio.Task` instances. But some APIs are tricky, mixing coroutines and plain functions in seemingly arbitrary ways, like the `StreamWriter` class we'll use in [Example 21-14](#).

[Example 21-9](#) wrapped up the *flags* set of examples. Now let's discuss the use of thread or process executors in asynchronous programming.

Delegating Tasks to Executors

One important advantage of Node.js over Python for asynchronous programming is the Node.js standard library, which provides `async` APIs for all I/O—not just for network I/O. In Python, if you're not careful, file I/O can seriously degrade the performance of asynchronous applications, because reading and writing to storage in the main thread blocks the event loop.

In the `download_one` coroutine of [Example 21-6](#), I used this line to save the downloaded image to disk:

```
await asyncio.to_thread(save_flag, image, f'{cc}.gif')
```

As mentioned before, the `asyncio.to_thread` was added in Python 3.9. If you need to support 3.7 or 3.8, then replace that single line with the lines in [Example 21-10](#).

Example 21-10. Lines to use instead of `await asyncio.to_thread`

```
loop = asyncio.get_running_loop()           ❶
loop.run_in_executor(None, save_flag,       ❷
                        image, f'{cc}.gif')  ❸
```

- ❶ Get a reference to the event loop.
- ❷ The first argument is the executor to use; passing `None` selects the default `ThreadPoolExecutor` that is always available in the `asyncio` event loop.
- ❸ You can pass positional arguments to the function to run, but if you need to pass keyword arguments, then you need to resort to `functool.partial`, as described in the [run_in_executor documentation](#).

The newer `asyncio.to_thread` function is easier to use and more flexible, as it also accepts keyword arguments.

The implementation of `asyncio` itself uses `run_in_executor` under the hood in a few places. For example, the `loop.getaddrinfo(...)` coroutine we saw in [Example 21-1](#) is implemented by calling the `getaddrinfo` function from the `socket` module—

which is a blocking function that may take seconds to return, as it depends on DNS resolution.

A common pattern in asynchronous APIs is to wrap blocking calls that are implementation details in coroutines using `run_in_executor` internally. That way, you provide a consistent interface of coroutines to be driven with `await`, and hide the threads you need to use for pragmatic reasons. The **Motor** asynchronous driver for MongoDB has an API compatible with `async/await` that is really a façade around a threaded core that talks to the database server. A. Jesse Jiryu Davis, the lead developer of Motor, explains his reasoning in “[Response to ‘Asynchronous Python and Databases’](#)”. Spoiler: Davis discovered that a thread pool was more performant in the particular use case of a database driver—despite the myth that asynchronous approaches are always faster than threads for network I/O.

The main reason to pass an explicit `Executor` to `loop.run_in_executor` is to employ a `ProcessPoolExecutor` if the function to execute is CPU intensive, so that it runs in a different Python process, avoiding contention for the GIL. Because of the high start-up cost, it would be better to start the `ProcessPoolExecutor` in the supervisor, and pass it to the coroutines that need to use it.

Caleb Hattingh—the author of *Using Asyncio in Python* (O’ Reilly)—is one of the tech reviewers of this book and suggested I add the following warning about executors and *asyncio*.



Caleb’s Warning about `run_in_executors`

Using `run_in_executor` can produce hard-to-debug problems since cancellation doesn’t work the way one might expect. Coroutines that use executors give merely the pretense of cancellation: the underlying thread (if it’s a `ThreadPoolExecutor`) has no cancellation mechanism. For example, a long-lived thread that is created inside a `run_in_executor` call may prevent your *asyncio* program from shutting down cleanly: `asyncio.run` will wait for the executor to fully shut down before returning, and it will wait forever if the executor jobs don’t stop somehow on their own. My greybeard inclination is to want that function to be named `run_in_executor_uncancellable`.

We’ll now go from client scripts to writing servers with *asyncio*.

Writing asyncio Servers

The classic toy example of a TCP server is an **echo server**. We'll build slightly more interesting toys: server-side Unicode character search utilities, first using HTTP with *FastAPI*, then using plain TCP with *asyncio* only.

These servers let users query for Unicode characters based on words in their standard names from the `unicodedata` module we discussed in “[The Unicode Database](#)” on [page 150](#). [Figure 21-2](#) shows a session with `web_mojifinder.py`, the first server we'll build.

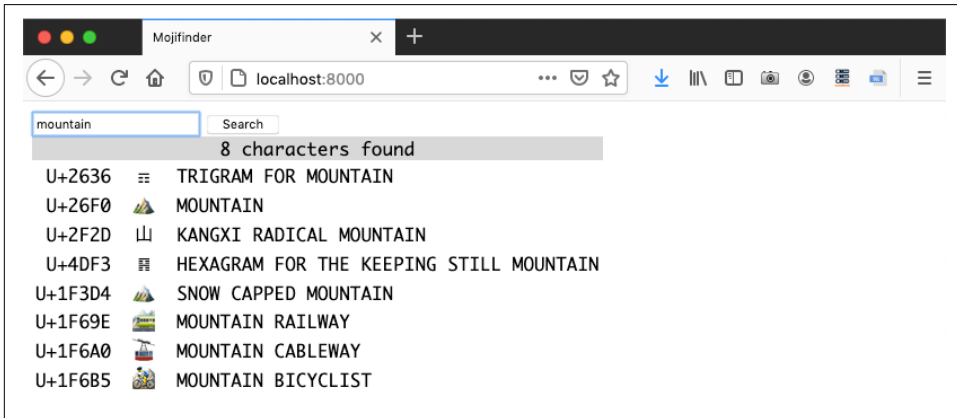


Figure 21-2. Browser window displaying search results for “mountain” from the `web_mojifinder.py` service.

The Unicode search logic in these examples is in the `InvertedIndex` class in the `char-index.py` module in the *Fluent Python code repository*. There’s nothing concurrent in that small module, so I’ll only give a brief overview in the optional box that follows. You can skip to the HTTP server implementation in “[A FastAPI Web Service](#)” on [page 800](#).

Meet the Inverted Index

An inverted index usually maps words to documents in which they occur. In the *mojifinder* examples, each “document” is one Unicode character. The `charindex.InvertedIndex` class indexes each word that appears in each character name in the Unicode database, and creates an inverted index stored in a `defaultdict`. For example, to index character U+0037—DIGIT SEVEN—the `InvertedIndex` initializer appends the character '7' to the entries under the keys 'DIGIT' and 'SEVEN'. After indexing the Unicode 13.0.0 data bundled with Python 3.9.1, 'DIGIT' maps to 868 characters, and 'SEVEN' maps to 143, including U+1F556—CLOCK FACE SEVEN

OCLOCK and U+2790—DINGBAT NEGATIVE CIRCLED SANS-SERIF DIGIT SEVEN (which appears in many code listings in this book).

See [Figure 21-3](#) for a demonstration using the entries for 'CAT' and 'FACE'.⁹

```
>>> from charindex import InvertedIndex
>>> idx.entries['CAT']
{'😺', '😸', '찰', '☒', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '🐈', '@', '😺', '😺'}
>>> len(idx.entries['FACE'])
171
>>> idx.entries['FACE'] & idx.entries['CAT']
{'😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺'}
>>> idx.search('cat face')
{'😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺', '😺'}
```

Figure 21-3. Python console exploring InvertedIndex attribute entries and search method.

The `InvertedIndex.search` method breaks the query into words, and returns the intersection of the entries for each word. That’s why searching for “face” finds 171 results, “cat” finds 14, but “cat face” only 10.

That’s the beautiful idea behind an inverted index: a fundamental building block in information retrieval—the theory behind search engines. See the English Wikipedia article [“Inverted Index”](#) to learn more.

A FastAPI Web Service

I wrote the next example—`web_mojifinder.py`—using *FastAPI*: one of the Python ASGI Web frameworks mentioned in [“ASGI—Asynchronous Server Gateway Interface” on page 732](#). [Figure 21-2](#) is a screenshot of the frontend. It’s a super simple SPA (Single Page Application): after the initial HTML download, the UI is updated by client-side JavaScript communicating with the server.

FastAPI is designed to implement backends for SPA and mobile apps, which mostly consist of web API end points returning JSON responses instead of server-rendered HTML. *FastAPI* leverages decorators, type hints, and code introspection to eliminate a lot of the boilerplate code for web APIs, and also automatically publishes interactive OpenAPI—a.k.a. *Swagger*—documentation for the APIs we create. [Figure 21-4](#) shows the autogenerated `/docs` page for `web_mojifinder.py`.

⁹ The boxed question mark in the screen shot is not a defect of the book or ebook you are reading. It’s the U+101EC—PHAISTOS DISC SIGN CAT character, which is missing from the font in the terminal I used. The *Phaistos disc* is an ancient artifact inscribed with pictograms, discovered in the island of Crete.

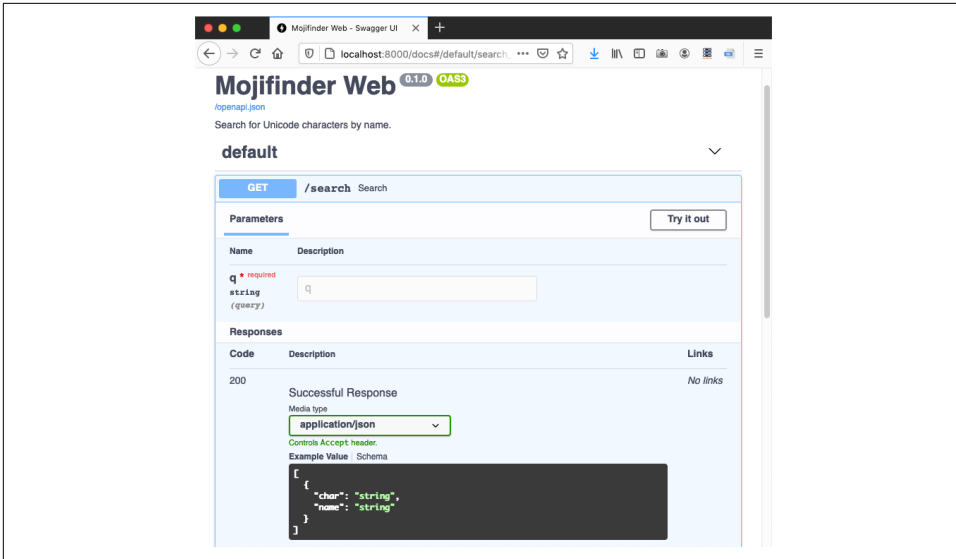


Figure 21-4. Autogenerated OpenAPI schema for the `/search` endpoint.

Example 21-11 is the code for `web_mojifinder.py`, but that’s just the backend code. When you hit the root URL `/`, the server sends the `form.html` file, which has 81 lines of code, including 54 lines of JavaScript to communicate with the server and fill a table with the results. If you’re interested in reading plain framework-less JavaScript, please find `21-async/mojifinder/static/form.html` in the [Fluent Python code repository](#).

To run `web_mojifinder.py`, you need to install two packages and their dependencies: `FastAPI` and `uvicorn`.¹⁰ This is the command to run **Example 21-11** with `uvicorn` in development mode:

```
$ uvicorn web_mojifinder:app --reload
```

The parameters are:

`web_mojifinder:app`

The package name, a colon, and the name of the ASGI application defined in it—`app` is the conventional name.

`--reload`

Make `uvicorn` monitor changes to application source files and automatically reload them. Useful only during development.

Now let’s study the source code for `web_mojifinder.py`.

¹⁰ Instead of `uvicorn`, you may use another ASGI server, such as `hypercorn` or `Daphne`. See the official ASGI documentation [page about implementations](#) for more information.

Example 21-11. *web_mojifinder.py*: complete source

```
from pathlib import Path
from unicodedata import name

from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from pydantic import BaseModel

from charindex import InvertedIndex

STATIC_PATH = Path(__file__).parent.absolute() / 'static' ❶

app = FastAPI( ❷
    title='Mojifinder Web',
    description='Search for Unicode characters by name.',
)

class CharName(BaseModel): ❸
    char: str
    name: str

def init(app): ❹
    app.state.index = InvertedIndex()
    app.state.form = (STATIC_PATH / 'form.html').read_text()

init(app) ❺

@app.get('/search', response_model=list[CharName]) ❻
async def search(q: str): ❼
    chars = sorted(app.state.index.search(q))
    return ({'char': c, 'name': name(c)} for c in chars) ❽

@app.get('/', response_class=HTMLResponse, include_in_schema=False)
def form(): ❾
    return app.state.form

# no main function ❿
```

- ❶ Unrelated to the theme of this chapter, but worth noting: the elegant use of the overloaded / operator by `pathlib`.¹¹
- ❷ This line defines the ASGI app. It could be as simple as `app = FastAPI()`. The parameters shown are metadata for the autogenerated documentation.

¹¹ Thanks to tech reviewer Miroslav Šedivý for highlighting good places to use `pathlib` in code examples.

- ③ A *pydantic* schema for a JSON response with `char` and `name` fields.¹²
- ④ Build the index and load the static HTML form, attaching both to the `app.state` for later use.
- ⑤ Run `init` when this module is loaded by the ASGI server.
- ⑥ Route for the `/search` endpoint; `response_model` uses that `CharName` *pydantic* model to describe the response format.
- ⑦ *FastAPI* assumes that any parameters that appear in the function or coroutine signature that are not in the route path will be passed in the HTTP query string, e.g., `/search?q=cat`. Since `q` has no default, *FastAPI* will return a 422 (Unprocessable Entity) status if `q` is missing from the query string.
- ⑧ Returning an iterable of dicts compatible with the `response_model` schema allows *FastAPI* to build the JSON response according to the `response_model` in the `@app.get` decorator.
- ⑨ Regular functions (i.e., non-async) can also be used to produce responses.
- ⑩ This module has no main function. It is loaded and driven by the ASGI server—*uvicorn* in this example.

Example 21-11 has no direct calls to `asyncio`. *FastAPI* is built on the *Starlette* ASGI toolkit, which in turn uses `asyncio`.

Also note that the body of `search` doesn't use `await`, `async with`, or `async for`, therefore it could be a plain function. I defined `search` as a coroutine just to show that *FastAPI* knows how to handle it. In a real app, most endpoints will query databases or hit other remote servers, so it is a critical advantage of *FastAPI*—and ASGI frameworks in general—to support coroutines that can take advantage of asynchronous libraries for network I/O.

¹² As mentioned in [Chapter 8](#), *pydantic* enforces type hints at runtime, for data validation.



The `init` and `form` functions I wrote to load and serve the static HTML form are a hack to make the example short and easy to run. The recommended best practice is to have a proxy/load-balancer in front of the ASGI server to handle all static assets, and also use a CDN (Content Delivery Network) when possible. One such proxy/load-balancer is *Traefik*, a self-described “edge router” that “receives requests on behalf of your system and finds out which components are responsible for handling them.” *FastAPI* has **project generation** scripts that prepare your code to do that.

The typing enthusiast may have noticed that there are no return type hints in `search` and `form`. Instead, *FastAPI* relies on the `response_model=` keyword argument in the route decorators. The “**Response Model**” page in the *FastAPI* documentation explains:

The response model is declared in this parameter instead of as a function return type annotation, because the path function may not actually return that response model but rather return a dict, database object or some other model, and then use the `response_model` to perform the field limiting and serialization.

For example, in `search`, I returned a generator of dict items, not a list of `CharName` objects, but that’s good enough for *FastAPI* and *pydantic* to validate my data and build the appropriate JSON response compatible with `response_model=list[CharName]`.

We’ll now focus on the `tcp_mojifinder.py` script that is answering the queries in **Figure 21-5**.

An asyncio TCP Server

The `tcp_mojifinder.py` program uses plain TCP to communicate with a client like Telnet or Netcat, so I could write it using `asyncio` without external dependencies—and without reinventing HTTP. **Figure 21-5** shows text-based UI.

```

luciano$ telnet localhost 2323
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
?> fire
U+2632 ☶      TRIGRAM FOR FIRE
U+2EA3 𐄃      CJK RADICAL FIRE
U+2F55 火      KANGXI RADICAL FIRE
U+322B 𤇀      PARENTHEZIZED IDEOGRAPH FIRE
U+328B 𤇁      CIRCLED IDEOGRAPH FIRE
U+4DDD 𠩺      HEXAGRAM FOR THE CLINGING FIRE
U+1F525 🔥      FIRE
U+1F692 🚒      FIRE ENGINE
U+1F6F1 🔥      ONCOMING FIRE ENGINE
U+1F702 🔥      ALCHEMICAL SYMBOL FOR FIRE
U+1F9EF 🧯      FIRE EXTINGUISHER
11 found

```

Figure 21-5. Telnet session with the `tcp_mojifinder.py` server: querying for “fire.”

This program is twice as long as `web_mojifinder.py`, so I split the presentation into three parts: [Example 21-12](#), [Example 21-14](#), and [Example 21-15](#). The top of `tcp_mojifinder.py`—including the `import` statements—is in [Example 21-14](#), but I will start by describing the supervisor coroutine and the `main` function that drives the program.

Example 21-12. `tcp_mojifinder.py`: a simple TCP server; continues in [Example 21-14](#)

```

async def supervisor(index: InvertedIndex, host: str, port: int) -> None:
    server = await asyncio.start_server(
        funtools.partial(finder, index),
        host, port)

    socket_list = cast(tuple[TransportSocket, ...], server.sockets)
    addr = socket_list[0].getsockname()
    print(f'Serving on {addr}. Hit CTRL-C to stop.')
    await server.serve_forever()

def main(host: str = '127.0.0.1', port_arg: str = '2323'):
    port = int(port_arg)
    print('Building index.')
    index = InvertedIndex()
    try:
        asyncio.run(supervisor(index, host, port))
    except KeyboardInterrupt:
        print('\nServer shut down.')

if __name__ == '__main__':
    main(*sys.argv[1:])

```

- ❶ This `await` quickly gets an instance of `asyncio.Server`, a TCP socket server. By default, `start_server` creates and starts the server, so it's ready to receive connections.
- ❷ The first argument to `start_server` is `client_connected_cb`, a callback to run when a new client connection starts. The callback can be a function or a coroutine, but it must accept exactly two arguments: an `asyncio.StreamReader` and an `asyncio.StreamWriter`. However, my `finder` coroutine also needs to get an index, so I used `functools.partial` to bind that parameter and obtain a callable that takes the reader and writer. Adapting user functions to callback APIs is the most common use case for `functools.partial`.
- ❸ `host` and `port` are the second and third arguments to `start_server`. See the full signature in the [asyncio documentation](#).
- ❹ This cast is needed because *typeshed* has an outdated type hint for the `sockets` property of the `Server` class—as of May 2021. See [Issue #5535 on typeshed](#).¹³
- ❺ Display the address and port of the first socket of the server.
- ❻ Although `start_server` already started the server as a concurrent task, I need to `await` on the `server_forever` method so that my supervisor is suspended here. Without this line, `supervisor` would return immediately, ending the loop started with `asyncio.run(supervisor(...))`, and exiting the program. The [documentation for `Server.serve\_forever`](#) says: “This method can be called if the server is already accepting connections.”
- ❼ Build the inverted index.¹⁴
- ❽ Start the event loop running supervisor.
- ❾ Catch the `KeyboardInterrupt` to avoid a distracting traceback when I stop the server with Ctrl-C on the terminal running it.

You may find it easier to understand how control flows in `tcp_mojifinder.py` if you study the output it generates on the server console, listed in [Example 21-13](#).

¹³ Issue #5535 is closed as of October 2021, but Mypy did not have a new release since then, so the error persists.

¹⁴ Tech reviewer Leonardo Rochael pointed out that building the index could be delegated to another thread using `loop.run_with_executor()` in the `supervisor` coroutine, so the server would be ready to take requests immediately while the index is built. That's true, but querying the index is the only thing this server does, so it would not be a big win in this example.

Example 21-13. tcp_mojifinder.py: this is the server side of the session depicted in Figure 21-5

```
$ python3 tcp_mojifinder.py
Building index. ❶
Serving on ('127.0.0.1', 2323). Hit Ctrl-C to stop. ❷
  From ('127.0.0.1', 58192): 'cat face' ❸
    To ('127.0.0.1', 58192): 10 results.
  From ('127.0.0.1', 58192): 'fire' ❹
    To ('127.0.0.1', 58192): 11 results.
  From ('127.0.0.1', 58192): '\x00' ❺
Close ('127.0.0.1', 58192). ❻
^C ❼
Server shut down. ❽
$
```

- ❶ Output by `main`. Before the next line appears, I see a 0.6s delay on my machine while the index is built.
- ❷ Output by `supervisor`.
- ❸ First iteration of a `while` loop in `finder`. The TCP/IP stack assigned port 58192 to my Telnet client. If you connect several clients to the server, you'll see their various ports in the output.
- ❹ Second iteration of the `while` loop in `finder`.
- ❺ I hit Ctrl-C on the client terminal; the `while` loop in `finder` exits.
- ❻ The `finder` coroutine displays this message then exits. Meanwhile the server is still running, ready to service another client.
- ❼ I hit Ctrl-C on the server terminal; `server.serve_forever` is cancelled, ending `supervisor` and the event loop.
- ❽ Output by `main`.

After `main` builds the index and starts the event loop, `supervisor` quickly displays the `Serving on...` message and is suspended at the `await server.serve_forever()` line. At that point, control flows into the event loop and stays there, occasionally coming back to the `finder` coroutine, which yields control back to the event loop whenever it needs to wait for the network to send or receive data.

While the event loop is alive, a new instance of the `finder` coroutine will be started for each client that connects to the server. In this way, many clients can be handled

concurrently by this simple server. This continues until a KeyboardInterrupt occurs on the server or its process is killed by the OS.

Now let's see the top of `tcp_mojifinder.py`, with the `finder` coroutine.

Example 21-14. tcp_mojifinder.py: continued from Example 21-12

```
import asyncio
import functools
import sys
from asyncio.trsock import TransportSocket
from typing import cast

from charindex import InvertedIndex, format_results ❶

CRLF = b'\r\n'
PROMPT = b'?> '

async def finder(index: InvertedIndex, ❷
                  reader: asyncio.StreamReader,
                  writer: asyncio.StreamWriter) -> None:
    client = writer.get_extra_info('peername') ❸
    while True: ❹
        writer.write(PROMPT) # can't await! ❺
        await writer.drain() # must await! ❻
        data = await reader.readline() ❼
        if not data: ❽
            break
        try:
            query = data.decode().strip() ❾
        except UnicodeDecodeError: ❿
            query = '\x00'
        print(f' From {client}: {query!r}') ⓫
        if query:
            if ord(query[:1]) < 32: ⓫
                break
            results = await search(query, index, writer) ⓫
            print(f' To {client}: {results} results.') ⓫

    writer.close() ⓫
    await writer.wait_closed() ⓫
    print(f'Close {client}.') ⓫
```

- ❶ `format_results` is useful to display the results of `InvertedIndex.search` in a text-based UI such as the command line or a Telnet session.
- ❷ To pass `finder` to `asyncio.start_server`, I wrapped it with `functools.partial`, because the server expects a coroutine or function that takes only the reader and writer arguments.
- ❸ Get the remote client address to which the socket is connected.
- ❹ This loop handles a dialog that lasts until a control character is received from the client.
- ❺ The `StreamWriter.write` method is not a coroutine, just a plain function; this line sends the `?>` prompt.
- ❻ `StreamWriter.drain` flushes the writer buffer; it is a coroutine, so it must be driven with `await`.
- ❼ `StreamWriter.readline` is a coroutine that returns bytes.
- ❽ If no bytes were received, the client closed the connection, so exit the loop.
- ❾ Decode the bytes to `str`, using the default UTF-8 encoding.
- ❿ A `UnicodeDecodeError` may happen when the user hits Ctrl-C and the Telnet client sends control bytes; if that happens, replace the query with a null character, for simplicity.
- ⓫ Log the query to the server console.
- ⓬ Exit the loop if a control or null character was received.
- ⓭ Do the actual search; code is presented next.
- ⓮ Log the response to the server console.
- ⓯ Close the `StreamWriter`.
- ⓰ Wait for the `StreamWriter` to close. This is recommended in the [.close\(\) method documentation](#).
- ⓱ Log the end of this client's session to the server console.

The last piece of this example is the search coroutine, shown in [Example 21-15](#).

Example 21-15. *tcp_mojifinder.py*: search coroutine

```
async def search(query: str, ❶
                    index: InvertedIndex,
                    writer: asyncio.StreamWriter) -> int:
    chars = index.search(query) ❷
    lines = (line.encode() + CRLF for line ❸
              in format_results(chars))
    writer.writelines(lines) ❹
    await writer.drain() ❺
    status_line = f'{"-" * 66} {len(chars)} found' ❻
    writer.write(status_line.encode() + CRLF)
    await writer.drain()
    return len(chars)
```

- ❶ search must be a coroutine because it writes to a `StreamWriter` and must use its `.drain()` coroutine method.
- ❷ Query the inverted index.
- ❸ This generator expression will yield byte strings encoded in UTF-8 with the Unicode codepoint, the actual character, its name, and a CRLF sequence—e.g., `b'U+0039\t9\tDIGIT NINE\r\n'`.
- ❹ Send the lines. Surprisingly, `writer.writelines` is not a coroutine.
- ❺ But `writer.drain()` is a coroutine. Don't forget the `await`!
- ❻ Build a status line, then send it.

Note that all network I/O in *tcp_mojifinder.py* is in bytes; we need to decode the bytes received from the network, and encode strings before sending them out. In Python 3, the default encoding is UTF-8, and that's what I used implicitly in all `encode` and `decode` calls in this example.



Note that some of the I/O methods are coroutines and must be driven with `await`, while others are simple functions. For example, `StreamWriter.write` is a plain function, because it writes to a buffer. On the other hand, `StreamWriter.drain`—which flushes the buffer and performs the network I/O—is a coroutine, as is `StreamReader.readline`—but not `StreamWriter.writelines`! While I was writing the first edition of this book, the `asyncio` API docs were improved by **clearly labeling coroutines as such**.

The `tcp_mojifinder.py` code leverages the high-level `asyncio Streams API` that provides a ready-to-use server so you only need to implement a handler function, which can be a plain callback or a coroutine. There is also a lower-level `Transports and Protocols API`, inspired by the transport and protocols abstractions in the *Twisted* framework. Refer to the `asyncio` documentation for more information, including `TCP and UDP echo servers and clients` implemented with that lower-level API.

Our next topic is `async for` and the objects that make it work.

Asynchronous Iteration and Asynchronous Iterables

We saw in “`Asynchronous Context Managers`” on page 786 how `async with` works with objects implementing the `__aenter__` and `__aexit__` methods returning awaitables—usually in the form of coroutine objects.

Similarly, `async for` works with *asynchronous iterables*: objects that implement `__aiter__`. However, `__aiter__` must be a regular method—not a coroutine method—and it must return an *asynchronous iterator*.

An asynchronous iterator provides an `__anext__` coroutine method that returns an awaitable—often a coroutine object. They are also expected to implement `__aiter__`, which usually returns `self`. This mirrors the important distinction of iterables and iterators we discussed in “`Don’t Make the Iterable an Iterator for Itself`” on page 605.

The *aiopg* asynchronous PostgreSQL driver [documentation](#) has an example that illustrates the use of `async for` to iterate over the rows of a database cursor:

```
async def go():
    pool = await aiopg.create_pool(dsn)
    async with pool.acquire() as conn:
        async with conn.cursor() as cur:
            await cur.execute("SELECT 1")
            ret = []
            async for row in cur:
                ret.append(row)
            assert ret == [(1,)]
```

In this example the query will return a single row, but in a realistic scenario you may have thousands of rows in response to a `SELECT` query. For large responses, the cursor will not be loaded with all the rows in a single batch. Therefore it is important that `async for row in cur:` does not block the event loop while the cursor may be waiting for additional rows. By implementing the cursor as an asynchronous iterator, *aiopg* may yield to the event loop at each `__anext__` call, and resume later when more rows arrive from PostgreSQL.

Asynchronous Generator Functions

You can implement an asynchronous iterator by writing a class with `__anext__` and `__aiter__`, but there is a simpler way: write a function declared with `async def` and use `yield` in its body. This parallels how generator functions simplify the classic `Iterator` pattern.

Let's study a simple example using `async for` and implementing an asynchronous generator. In [Example 21-1](#) we saw *blogdom.py*, a script that probed domain names. Now suppose we find other uses for the probe coroutine we defined there, and decide to put it into a new module—*domainlib.py*—together with a new `multi_probe` asynchronous generator that takes a list of domain names and yields results as they are probed.

We'll look at the implementation of *domainlib.py* soon, but first let's see how it is used with Python's new asynchronous console.

Experimenting with Python's `async` console

Since [Python 3.8](#), you can run the interpreter with the `-m asyncio` command-line option to get an “async REPL”: a Python console that imports `asyncio`, provides a running event loop, and accepts `await`, `async for`, and `async with` at the top-level prompt—which otherwise are syntax errors when used outside of native coroutines.¹⁵

To experiment with *domainlib.py*, go to the `21-async/domains/asyncio/` directory in your local copy of the [Fluent Python code repository](#). Then run:

```
$ python -m asyncio
```

You'll see the console start, similar to this:

```
asyncio REPL 3.9.1 (v3.9.1:1e5d33e9b9, Dec 7 2020, 12:10:52)
[Clang 6.0 (clang-600.0.57)] on darwin
Use "await" directly instead of "asyncio.run()".
Type "help", "copyright", "credits" or "license" for more information.
>>> import asyncio
>>>
```

Note how the header says you can use `await` instead of `asyncio.run()`—to drive coroutines and other awaitables. Also: I did not type `import asyncio`. The `asyncio` module is automatically imported and that line makes that fact clear to the user.

¹⁵ This is great for experimentation, like the Node.js console. Thanks to Yuri Selivanov for yet another excellent contribution to asynchronous Python.

Now let's import *domainlib.py* and play with its two coroutines: *probe* and *multi_probe* (Example 21-16).

Example 21-16. Experimenting with domainlib.py after running python3 -m asyncio

```
>>> await asyncio.sleep(3, 'Rise and shine!') ❶
'Rise and shine!'
>>> from domainlib import *
>>> await probe('python.org') ❷
Result(domain='python.org', found=True) ❸
>>> names = 'python.org rust-lang.org golang.org no-lang.invalid'.split() ❹
>>> async for result in multi_probe(names): ❺
...     print(*result, sep='\t')
...
golang.org      True ❻
no-lang.invalid False
python.org      True
rust-lang.org   True
>>>
```

- ❶ Try a simple `await` to see the asynchronous console in action. Tip: `asyncio.sleep()` takes an optional second argument that is returned when you await it.
- ❷ Drive the *probe* coroutine.
- ❸ The *domainlib* version of *probe* returns a *Result* named tuple.
- ❹ Make a list of domains. The `.invalid` top-level domain is reserved for testing. DNS queries for such domains always get an `NXDOMAIN` response from DNS servers, meaning “that domain does not exist.”¹⁶
- ❺ Iterate with `async for` over the *multi_probe* asynchronous generator to display the results.
- ❻ Note that the results are not in the order the domains were given to *multi_probe*. They appear as each DNS response comes back.

Example 21-16 shows that *multi_probe* is an asynchronous generator because it is compatible with `async for`. Now let's do a few more experiments, continuing from that example with Example 21-17.

¹⁶ See RFC 6761—Special-Use Domain Names.

Example 21-17. More experiments, continuing from [Example 21-16](#)

```
>>> probe('python.org') ❶
<coroutine object probe at 0x10e313740>
>>> multi_probe(names) ❷
<async_generator object multi_probe at 0x10e246b80>
>>> for r in multi_probe(names): ❸
...     print(r)
...
Traceback (most recent call last):
...
TypeError: 'async_generator' object is not iterable
```

- ❶ Calling a native coroutine gives you a coroutine object.
- ❷ Calling an asynchronous generator gives you an `async_generator` object.
- ❸ We can't use a regular `for` loop with asynchronous generators because they implement `__aiter__` instead of `__iter__`.

Asynchronous generators are driven by `async for`, which can be a block statement (as seen in [Example 21-16](#)), and it also appears in asynchronous comprehensions, which we'll cover soon.

Implementing an asynchronous generator

Now let's study the code for *domainlib.py*, with the `multi_probe` asynchronous generator ([Example 21-18](#)).

Example 21-18. *domainlib.py*: functions for probing domains

```
import asyncio
import socket
from collections.abc import Iterable, AsyncIterator
from typing import NamedTuple, Optional

class Result(NamedTuple): ❶
    domain: str
    found: bool

OptionalLoop = Optional[asyncio.AbstractEventLoop] ❷

async def probe(domain: str, loop: OptionalLoop = None) -> Result: ❸
    if loop is None:
        loop = asyncio.get_running_loop()
    try:
```

```

        await loop.getaddrinfo(domain, None)
    except socket.gaierror:
        return Result(domain, False)
    return Result(domain, True)

```

```

async def multi_probe(domains: Iterable[str]) -> AsyncIterator[Result]: ❹
    loop = asyncio.get_running_loop()
    coros = [probe(domain, loop) for domain in domains] ❺
    for coro in asyncio.as_completed(coros): ❻
        result = await coro ❼
        yield result ❽

```

- ❶ NamedTuple makes the result from probe easier to read and debug.
- ❷ This type alias is to avoid making the next line too long for a book listing.
- ❸ probe now gets an optional loop argument, to avoid repeated calls to get_running_loop when this coroutine is driven by multi_probe.
- ❹ An asynchronous generator function produces an asynchronous generator object, which can be annotated as AsyncIterator[SomeType].
- ❺ Build list of probe coroutine objects, each with a different domain.
- ❻ This is not async for because asyncio.as_completed is a classic generator.
- ❼ Await on the coroutine object to retrieve the result.
- ❽ Yield result. This line makes multi_probe an asynchronous generator.



The for loop in [Example 21-18](#) could be more concise:

```

for coro in asyncio.as_completed(coros):
    yield await coro

```

Python parses that as `yield (await coro)`, so it works.

I thought it could be confusing to use that shortcut in the first asynchronous generator example in the book, so I split it into two lines.

Given *domainlib.py*, we can demonstrate the use of the `multi_probe` asynchronous generator in *domaincheck.py*: a script that takes a domain suffix and searches for domains made from short Python keywords.

Here is a sample output of *domaincheck.py*:

```

$ ./domaincheck.py net
FOUND          NOT FOUND
=====
in.net
del.net
true.net
for.net
is.net

                none.net

try.net

                from.net

and.net
or.net
else.net
with.net
if.net
as.net

                elif.net
                pass.net
                not.net
                def.net

```

Thanks to *domainlib*, the code for *domaincheck.py* is straightforward, as seen in [Example 21-19](#).

Example 21-19. domaincheck.py: utility for probing domains using domainlib

```

#!/usr/bin/env python3
import asyncio
import sys
from keyword import kwlist

from domainlib import multi_probe

async def main(tld: str) -> None:
    tld = tld.strip('.')
    names = (kw for kw in kwlist if len(kw) <= 4) ❶
    domains = (f'{name}.{tld}'.lower() for name in names) ❷
    print('FOUND\t\tNOT FOUND') ❸
    print('=====\t\t=====')
    async for domain, found in multi_probe(domains): ❹
        indent = ' ' if found else '\t\t' ❺
        print(f'{indent}{domain}')

if __name__ == '__main__':
    if len(sys.argv) == 2:
        asyncio.run(main(sys.argv[1])) ❻
    else:
        print('Please provide a TLD.', f'Example: {sys.argv[0]} COM.BR')

```

- ❶ Generate keywords with length up to 4.
- ❷ Generate domain names with the given suffix as TLD.
- ❸ Format a header for the tabular output.
- ❹ Asynchronously iterate over `multi_probe(domains)`.
- ❺ Set indent to zero or two tabs to put the result in the proper column.
- ❻ Run the `main` coroutine with the given command-line argument.

Generators have one extra use unrelated to iteration: they can be made into context managers. This also applies to asynchronous generators.

Asynchronous generators as context managers

Writing our own asynchronous context managers is not a frequent programming task, but if you need to write one, consider using the `@asynccontextmanager` decorator added to the `contextlib` module in Python 3.7. That’s very similar to the `@contextmanager` decorator we studied in “Using `@contextmanager`” on page 664.

An interesting example combining `@asynccontextmanager` with `loop.run_in_executor` appears in Caleb Hattingh’s book *Using Asyncio in Python*. Example 21-20 is Caleb’s code—with a single change and added callouts.

Example 21-20. Example using `@asynccontextmanager` and `loop.run_in_executor`

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def web_page(url): ❶
    loop = asyncio.get_running_loop() ❷
    data = await loop.run_in_executor( ❸
        None, download_webpage, url)
    yield data ❹
    await loop.run_in_executor(None, update_stats, url) ❺

async with web_page('google.com') as data: ❻
    process(data)
```

- ❶ The decorated function must be an asynchronous generator.
- ❷ Minor update to Caleb’s code: use the lightweight `get_running_loop` instead of `get_event_loop`.

- ❸ Suppose `download_webpage` is a blocking function using the *requests* library; we run it in a separate thread to avoid blocking the event loop.
- ❹ All lines before this `yield` expression will become the `__aenter__` coroutine-method of the asynchronous context manager built by the decorator. The value of `data` will be bound to the `data` variable after the `as` clause in the `async with` statement below.
- ❺ Lines after the `yield` will become the `__aexit__` coroutine method. Here, another blocking call is delegated to the thread executor.
- ❻ Use `web_page` with `async with`.

This is very similar to the sequential `@contextmanager` decorator. Please see “[Using @contextmanager](#)” on page 664 for more details, including error handling at the `yield` line. For another example of `@asynccontextmanager`, see the [contextlib documentation](#).

Now let’s wrap up our coverage of asynchronous generator functions by contrasting them with native coroutines.

Asynchronous generators versus native coroutines

Here are some key similarities and differences between a native coroutine and an asynchronous generator function:

- Both are declared with `async def`.
- An asynchronous generator always has a `yield` expression in its body—that’s what makes it a generator. A native coroutine never contains `yield`.
- A native coroutine may return some value other than `None`. An asynchronous generator can only use empty `return` statements.
- Native coroutines are awaitable: they can be driven by `await` expressions or passed to one of the many `asyncio` functions that take awaitable arguments, such as `create_task`. Asynchronous generators are not awaitable. They are asynchronous iterables, driven by `async for` or by asynchronous comprehensions.

Time to talk about asynchronous comprehensions.

Async Comprehensions and Async Generator Expressions

[PEP 530—Asynchronous Comprehensions](#) introduced the use of `async for` and `await` in the syntax of comprehensions and generator expressions, starting with Python 3.6.

The only construct defined by PEP 530 that can appear outside an `async def` body is an asynchronous generator expression.

Defining and using an asynchronous generator expression

Given the `multi_probe` asynchronous generator from [Example 21-18](#), we could write another asynchronous generator returning only the names of the domains found. Here is how—again using the asynchronous console launched with `-m asyncio`:

```
>>> from domainlib import multi_probe
>>> names = 'python.org rust-lang.org golang.org no-lang.invalid'.split()
>>> gen_found = (name async for name, found in multi_probe(names) if found) ❶
>>> gen_found
<async_generator object <genexpr> at 0x10a8f9700> ❷
>>> async for name in gen_found: ❸
...     print(name)
...
golang.org
python.org
rust-lang.org
```

- ❶ The use of `async for` makes this an asynchronous generator expression. It can be defined anywhere in a Python module.
- ❷ The asynchronous generator expression builds an `async_generator` object—exactly the same type of object returned by an asynchronous generator function like `multi_probe`.
- ❸ The asynchronous generator object is driven by the `async for` statement, which in turn can only appear inside an `async def` body or in the magic asynchronous console I used in this example.

To summarize: an asynchronous generator expression can be defined anywhere in your program, but it can only be consumed inside a native coroutine or asynchronous generator function.

The remaining constructs introduced by PEP 530 can only be defined and used inside native coroutines or asynchronous generator functions.

Asynchronous comprehensions

Yury Selivanov—the author of PEP 530—justifies the need for asynchronous comprehensions with three short code snippets reproduced next.

We can all agree that we should be able to rewrite this code:

```
result = []
async for i in aiter():
```

```
if i % 2:
    result.append(i)
```

like this:

```
result = [i async for i in aiter() if i % 2]
```

In addition, given a native coroutine fun, we should be able to write this:

```
result = [await fun() for fun in funcs]
```



Using `await` in a list comprehension is similar to using `asyncio.gather`. But `gather` gives you more control over exception handling, thanks to its optional `return_exceptions` argument. Caleb Hattingh recommends always setting `return_exceptions=True` (the default is `False`). Please see the [asyncio.gather documentation](#) for more.

Back to the magic asynchronous console:

```
>>> names = 'python.org rust-lang.org golang.org no-lang.invalid'.split()
>>> names = sorted(names)
>>> coros = [probe(name) for name in names]
>>> await asyncio.gather(*coros)
[Result(domain='golang.org', found=True),
 Result(domain='no-lang.invalid', found=False),
 Result(domain='python.org', found=True),
 Result(domain='rust-lang.org', found=True)]
>>> [await probe(name) for name in names]
[Result(domain='golang.org', found=True),
 Result(domain='no-lang.invalid', found=False),
 Result(domain='python.org', found=True),
 Result(domain='rust-lang.org', found=True)]
>>>
```

Note that I sorted the list of names to show that the results come out in the order they were submitted, in both cases.

PEP 530 allows the use of `async for` and `await` in list comprehensions as well as in dict and set comprehensions. For example, here is a dict comprehension to store the results of `multi_probe` in the asynchronous console:

```
>>> {name: found async for name, found in multi_probe(names)}
{'golang.org': True, 'python.org': True, 'no-lang.invalid': False,
 'rust-lang.org': True}
```

We can use the `await` keyword in the expression before the `for` or `async for` clause, and also in the expression after the `if` clause. Here is a set comprehension in the asynchronous console, collecting only the domains that were found:

```
>>> {name for name in names if (await probe(name)).found}
{'rust-lang.org', 'python.org', 'golang.org'}
```

I had to put extra parentheses around the `await` expression due to the higher precedence of the `__getattr__` operator . (dot).

Again, all of these comprehensions can only appear inside an `async def` body or in the enchanted asynchronous console.

Now let's talk about a very important feature of the `async` statements, `async` expressions, and the objects they create. Those constructs are often used with *asyncio* but, they are actually library independent.

async Beyond asyncio: Curio

Python's `async/await` language constructs are not tied to any specific event loop or library.¹⁷ Thanks to the extensible API provided by special methods, anyone sufficiently motivated can write their own asynchronous runtime environment and framework to drive native coroutines, asynchronous generators, etc.

That's what David Beazley did in his *Curio* project. He was interested in rethinking how these new language features could be used in a framework built from scratch. Recall that *asyncio* was released in Python 3.4, and it used `yield from` instead of `await`, so its API could not leverage asynchronous context managers, asynchronous iterators, and everything else that the `async/await` keywords made possible. As a result, *Curio* has a cleaner API and a simpler implementation, compared to *asyncio*.

Example 21-21 shows the *blogdom.py* script (**Example 21-1**) rewritten to use *Curio*.

Example 21-21. blogdom.py: Example 21-1, now using Curio

```
#!/usr/bin/env python3
from curio import run, TaskGroup
import curio.socket as socket
from keyword import kwlist

MAX_KEYWORD_LEN = 4

async def probe(domain: str) -> tuple[str, bool]: ❶
    try:
        await socket.getaddrinfo(domain, None) ❷
    except socket.gaierror:
        return (domain, False)
    return (domain, True)
```

¹⁷ That's in contrast with JavaScript, where `async/await` is hardwired to the built-in event loop and runtime environment, i.e., a browser, Node.js, or Deno.

```

async def main() -> None:
    names = (kw for kw in kwlist if len(kw) <= MAX_KEYWORD_LEN)
    domains = (f'{name}.dev'.lower() for name in names)
    async with TaskGroup() as group: ❸
        for domain in domains:
            await group.spawn(probe, domain) ❹
        async for task in group: ❺
            domain, found = task.result
            mark = '+' if found else ' '
            print(f'{mark} {domain}')

if __name__ == '__main__':
    run(main()) ❻

```

- ❶ probe doesn't need to get the event loop, because...
- ❷ ...getaddrinfo is a top-level function of `curio.socket`, not a method of a loop object—as it is in `asyncio`.
- ❸ A `TaskGroup` is a core concept in *Curio*, to monitor and control several coroutines, and to make sure they are all executed and cleaned up.
- ❹ `TaskGroup.spawn` is how you start a coroutine, managed by a specific `TaskGroup` instance. The coroutine is wrapped by a `Task`.
- ❺ Iterating with `async for` over a `TaskGroup` yields `Task` instances as each is completed. This corresponds to the line in [Example 21-1](#) using `for ... as_completed(...):`.
- ❻ *Curio* pioneered this sensible way to start an asynchronous program in Python.

To expand on the last point: if you look at the `asyncio` code examples for the first edition of *Fluent Python*, you'll see lines like these, repeated over and over:

```

loop = asyncio.get_event_loop()
loop.run_until_complete(main())
loop.close()

```

A *Curio* `TaskGroup` is an asynchronous context manager that replaces several ad hoc APIs and coding patterns in `asyncio`. We just saw how iterating over a `TaskGroup` makes the `asyncio.as_completed(...)` function unnecessary. Another example: instead of a special gather function, this snippet from the “[Task Groups](#)” docs collects the results of all tasks in the group:

```

async with TaskGroup(wait=all) as g:
    await g.spawn(coro1)
    await g.spawn(coro2)

```

```
await g.spawn(coro3)
print('Results:', g.results)
```

Task groups support *structured concurrency*: a form of concurrent programming that constrains all the activity of a group of asynchronous tasks to a single entry and exit point. This is analogous to structured programming, which eschewed the GOTO command and introduced block statements to limit the entry and exit points of loops and subroutines. When used as an asynchronous context manager, a TaskGroup ensures that all tasks spawned inside are completed or cancelled, and any exceptions raised, upon exiting the enclosed block.



Structured concurrency will probably be adopted by *asyncio* in upcoming Python releases. A strong indication appears in [PEP 654—Exception Groups and `except\*`](#), which was [approved for Python 3.11](#). The “Motivation” section mentions *Trio*’s “nurseries,” their name for task groups: “Implementing a better task spawning API in *asyncio*, inspired by Trio nurseries, was the main motivation for this PEP.”

Another important feature of *Curio* is better support for programming with coroutines and threads in the same codebase—a necessity in most nontrivial asynchronous programs. Starting a thread with `await spawn_thread(func, ...)` returns an `AsyncThread` object with a Task-like interface. Threads can call coroutines thanks to a special `AWAIT(coro)` function—named in all caps because `await` is now a keyword.

Curio also provides a `UniversalQueue` that can be used to coordinate the work among threads, *Curio* coroutines, and *asyncio* coroutines. That’s right, *Curio* has features that allow it to run in a thread along with *asyncio* in another thread, in the same process, communicating via `UniversalQueue` and `UniversalEvent`. The API for these “universal” classes is the same inside and outside of coroutines, but in a coroutine, you need to prefix calls with `await`.

As I write this in October 2021, *HTTPX* is the first HTTP client library [compatible with *Curio*](#), but I don’t know of any asynchronous database libraries that support it yet. In the *Curio* repository there is an impressive set of [network programming examples](#), including one using *Websocket*, and another implementing the [RFC 8305—Happy Eyeballs](#) concurrent algorithm for connecting to IPv6 endpoints with fast fallback to IPv4 if needed.

The design of *Curio* has been influential. The *Trio* framework started by Nathaniel J. Smith was heavily inspired by *Curio*. *Curio* may also have prompted Python contributors to improve the usability of the *asyncio* API. For example, in its earliest releases, *asyncio* users very often had to get and pass around a loop object because some essential functions were either loop methods or required a loop argument. In recent

versions of Python, direct access to the loop is not needed as often, and in fact several functions that accepted an optional loop are now deprecating that argument.

Type annotations for asynchronous types are our next topic.

Type Hinting Asynchronous Objects

The return type of a native coroutine describes what you get when you await on that coroutine, which is the type of the object that appears in the return statements in the body of the native coroutine function.¹⁸

This chapter provided many examples of annotated native coroutines, including probe from [Example 21-21](#):

```
async def probe(domain: str) -> tuple[str, bool]:
    try:
        await socket.getaddrinfo(domain, None)
    except socket.gaierror:
        return (domain, False)
    return (domain, True)
```

If you need to annotate a parameter that takes a coroutine object, then the generic type is:

```
class typing.Coroutine(Awaitable[V_co], Generic[T_co, T_contra, V_co]):
    ...
```

That type, and the following types were introduced in Python 3.5 and 3.6 to annotate asynchronous objects:

```
class typing.AsyncContextManager(Generic[T_co]):
    ...
class typing.AsyncIterable(Generic[T_co]):
    ...
class typing.AsyncIterator(AsyncIterable[T_co]):
    ...
class typing.AsyncGenerator(AsyncIterator[T_co], Generic[T_co, T_contra]):
    ...
class typing.Awaitable(Generic[T_co]):
    ...
```

With Python ≥ 3.9 , use the `collections.abc` equivalents of these.

I want to highlight three aspects of those generic types.

First: they are all covariant on the first type parameter, which is the type of the items yielded from these objects. Recall rule #1 of “[Variance rules of thumb](#)” on page 551:

¹⁸ This differs from the annotations of classic coroutines, as discussed in “[Generic Type Hints for Classic Coroutines](#)” on page 650.

If a formal type parameter defines a type for data that comes out of the object, it can be covariant.

Second: `AsyncGenerator` and `Coroutine` are contravariant on the second to last parameter. That's the type of the argument of the low-level `.send()` method that the event loop calls to drive asynchronous generators and coroutines. As such, it is an "input" type. Therefore, it can be contravariant, per Variance Rule of Thumb #2:

If a formal type parameter defines a type for data that goes into the object after its initial construction, it can be contravariant.

Third: `AsyncGenerator` has no return type, in contrast with `typing.Generator`, which we saw in [“Generic Type Hints for Classic Coroutines” on page 650](#). Returning a value by raising `StopIteration(value)` was one of the hacks that enabled generators to operate as coroutines and support `yield from`, as we saw in [“Classic Coroutines” on page 641](#). There is no such overlap among the asynchronous objects: `AsyncGenerator` objects don't return values, and are completely separate from native coroutine objects, which are annotated with `typing.Coroutine`.

Finally, let's briefly discuss the advantages and challenges of asynchronous programming.

How Async Works and How It Doesn't

The sections closing this chapter discuss high-level ideas around asynchronous programming, regardless of the language or library you are using.

Let's begin by explaining the #1 reason why asynchronous programming is appealing, followed by a popular myth, and how to deal with it.

Running Circles Around Blocking Calls

Ryan Dahl, the inventor of Node.js, introduces the philosophy of his project by saying “We're doing I/O completely wrong.”¹⁹ He defines a *blocking function* as one that does file or network I/O, and argues that we can't treat them as we treat nonblocking functions. To explain why, he presents the numbers in the second column of [Table 21-1](#).

¹⁹ Video: [“Introduction to Node.js”](#) at 4:55.

Table 21-1. Modern computer latency for reading data from different devices; third column shows proportional times in a scale easier to understand for us slow humans

Device	CPU cycles	Proportional “human” scale
L1 cache	3	3 seconds
L2 cache	14	14 seconds
RAM	250	250 seconds
disk	41,000,000	1.3 years
network	240,000,000	7.6 years

To make sense of [Table 21-1](#), bear in mind that modern CPUs with GHz clocks run billions of cycles per second. Let’s say that a CPU runs exactly 1 billion cycles per second. That CPU can make more than 333 million L1 cache reads in 1 second, or 4 (four!) network reads in the same time. The third column of [Table 21-1](#) puts those numbers in perspective by multiplying the second column by a constant factor. So, in an alternate universe, if one read from L1 cache took 3 seconds, then a network read would take 7.6 years!

[Table 21-1](#) explains why a disciplined approach to asynchronous programming can lead to high-performance servers. The challenge is achieving that discipline. The first step is to recognize that “I/O bound system” is a fantasy.

The Myth of I/O-Bound Systems

A commonly repeated meme is that asynchronous programming is good for “I/O bound systems.” I learned the hard way that there are no “I/O-bound systems.” You may have I/O-bound *functions*. Perhaps the vast majority of the functions in your system are I/O bound; i.e., they spend more time waiting for I/O than crunching data. While waiting, they cede control to the event loop, which can then drive some other pending task. But inevitably, any nontrivial system will have some parts that are CPU bound. Even trivial systems reveal that, under stress. In [“Soapbox” on page 829](#), I tell the story of two asynchronous programs that struggled with CPU-bound functions slowing down the event loop with severe impact on performance.

Given that any nontrivial system will have CPU-bound functions, dealing with them is the key to success in asynchronous programming.

Avoiding CPU-Bound Traps

If you’re using Python at scale, you should have some automated tests designed specifically to detect performance regressions as soon as they appear. This is critically important with asynchronous code, but also relevant to threaded Python code—because of the GIL. If you wait until the slowdown starts bothering the development team, it’s too late. The fix will probably require some major makeover.

Here are some options for when you identify a CPU-hogging bottleneck:

- Delegate the task to a Python process pool.
- Delegate the task to an external task queue.
- Rewrite the relevant code in Cython, C, Rust, or some other language that compiles to machine code and interfaces with the Python/C API, preferably releasing the GIL.
- Decide that you can afford the performance hit and do nothing—but record the decision to make it easier to revert to it later.

The external task queue should be chosen and integrated as soon as possible at the start of the project, so that nobody in the team hesitates to use it when needed.

The last option—do nothing—falls in the category of **technical debt**.

Concurrent programming is a fascinating topic, and I would like to write a lot more about it. But it is not the main focus of this book, and this is already one of the longest chapters, so let's wrap it up.

Chapter Summary

The problem with normal approaches to asynchronous programming is that they're all-or-nothing propositions. You rewrite all your code so none of it blocks or you're just wasting your time.

—Alvaro Videla and Jason J. W. Williams, *RabbitMQ in Action*

I chose that epigraph for this chapter for two reasons. At a high level, it reminds us to avoid blocking the event loop by delegating slow tasks to a different processing unit, from a simple thread all the way to a distributed task queue. At a lower level, it is also a warning: once you write your first `async def`, your program is inevitably going to have more and more `async def`, `await`, `async with`, and `async for`. And using non-asynchronous libraries suddenly becomes a challenge.

After the simple *spinner* examples in [Chapter 19](#), here our main focus was asynchronous programming with native coroutines, starting with the *blogdom.py* DNS probing example, followed by the concept of *awaitables*. While reading the source code of *flags_asyncio.py*, we found the first example of an *asynchronous context manager*.

The more advanced variations of the flag downloading program introduced two powerful functions: the `asyncio.as_completed` generator and the `loop.run_in_executor` coroutine. We also saw the concept and application of a semaphore to limit the number of concurrent downloads—as expected from well-behaved HTTP clients.

Server-side asynchronous programming was presented through the *mojifinder* examples: a *FastAPI* web service and *tcp_mojifinder.py*—the latter using just *asyncio* and the TCP protocol.

Asynchronous iteration and asynchronous iterables were the next major topic, with sections on `async for`, Python’s *async* console, asynchronous generators, asynchronous generator expressions, and asynchronous comprehensions.

The last example in the chapter was *blogdom.py* rewritten with the *Curio* framework, to demonstrate how Python’s asynchronous features are not tied to the *asyncio* package. *Curio* also showcases the concept of *structured concurrency*, which may have an industry-wide impact, bringing more clarity to concurrent code.

Finally, the sections under “[How Async Works and How It Doesn’t](#)” on page 825 discuss the main appeal of asynchronous programming, the misconception of “I/O-bound systems,” and dealing with the inevitable CPU-bound parts of your program.

Further Reading

David Beazley’s PyOhio 2016 keynote “[Fear and Awaiting in Async](#)” is a fantastic, live-coded introduction to the potential of the language features made possible by Yury Selivanov’s contribution of the `async/await` keywords in Python 3.5. At one point, Beazley complains that `await` can’t be used in list comprehensions, but that was fixed by Selivanov in [PEP 530—Asynchronous Comprehensions](#), implemented in Python 3.6 later in that same year. Apart from that, everything else in Beazley’s keynote is timeless, as he demonstrates how the asynchronous objects we saw in this chapter work, without the help of any framework—just a simple run function using `.send(None)` to drive coroutines. Only at the very end Beazley shows *Curio*, which he started that year as an experiment to see how far can you go doing asynchronous programming without a foundation of callbacks or futures, just coroutines. As it turns out, you can go very far—as demonstrated by the evolution of *Curio* and the later creation of *Trio* by Nathaniel J. Smith. *Curio*’s documentation has [links](#) to more talks by Beazley on the subject.

Besides starting *Trio*, Nathaniel J. Smith wrote two deep blog posts that I highly recommend: “[Some thoughts on asynchronous API design in a post-async/await world](#)”, contrasting the design of *Curio* with that of *asyncio*, and “[Notes on structured concurrency, or: Go statement considered harmful](#)”, about structured concurrency. Smith also gave a long and informative answer to the question: “[What is the core difference between asyncio and trio?](#)” on StackOverflow.

To learn more about the *asyncio* package, I’ve mentioned the best written resources I know at the start of this chapter: the [official documentation](#) after the outstanding [overhaul](#) started by Yury Selivanov in 2018, and Caleb Hattingh’s book *Using Asyncio in Python* (O’Reilly). In the official documentation, make sure to read “[Developing](#)

with *asyncio*”: documenting the *asyncio* debug mode, and also discussing common mistakes and traps and how to avoid them.

For a very accessible, 30-minute introduction to asynchronous programming in general and also *asyncio*, watch Miguel Grinberg’s “*Asynchronous Python for the Complete Beginner*”, presented at PyCon 2017. Another great introduction is “*Demystifying Python’s Async and Await Keywords*”, presented by Michael Kennedy, where among other things I learned about the *unsync* library that provides a decorator to delegate the execution of coroutines, I/O-bound functions, and CPU-bound functions to *asyncio*, threading, or multiprocessing as needed.

At EuroPython 2019, Lynn Root—a global leader of *PyLadies*—presented the excellent “*Advanced asyncio: Solving Real-world Production Problems*”, informed by her experience using Python as a staff engineer at Spotify.

In 2020, Łukasz Langa recorded a series of great videos about *asyncio*, starting with “*Learn Python’s AsyncIO #1—The Async Ecosystem*”. Langa also made the super cool video “*AsyncIO + Music*” for PyCon 2020 that not only shows *asyncio* applied in a very concrete event-oriented domain, but also explains it from the ground up.

Another area dominated by event-oriented programming is embedded systems. That’s why Damien George added support for *async/await* in his *MicroPython* interpreter for microcontrollers. At PyCon Australia 2018, Matt Trentini demonstrated the *uasyncio* library, a subset of *asyncio* that is part of MicroPython’s standard library.

For higher-level thinking about async programming in Python, read the blog post “*Python async frameworks—Beyond developer tribalism*” by Tom Christie.

Finally, I recommend “*What Color Is Your Function?*” by Bob Nystrom, discussing the incompatible execution models of plain functions versus async functions—a.k.a. coroutines—in JavaScript, Python, C#, and other languages. Spoiler alert: Nystrom’s conclusion is that the language that got this right is Go, where all functions are the same color. I like that about Go. But I also think Nathaniel J. Smith has a point when he wrote “*Go statement considered harmful*”. Nothing is perfect, and concurrent programming is always hard.

Soapbox

How a Slow Function Almost Spoiled the uvloop Benchmarks

In 2016, Yuri Selivanov released *uvloop*, “a fast, drop-in replacement of the built-in *asyncio* event loop.” The benchmarks presented in Selivanov’s [blog post](#) announcing the library in 2016 are very impressive. He wrote: “it is at least 2x faster than nodejs, gevent, as well as any other Python asynchronous framework. The performance of uvloop-based *asyncio* is close to that of Go programs.”

However, the post reveals that *uvloop* is able to match the performance of Go under two conditions:

1. Go is configured to use a single thread. That makes the Go runtime behave similarly to *asyncio*: concurrency is achieved via multiple coroutines driven by an event loop, all in the same thread.²⁰
2. The Python 3.5 code uses *httptools* in addition to *uvloop* itself.

Selivanov explains that he wrote *httptools* after benchmarking *uvloop* with *aiohttp*—one of the first full-featured HTTP libraries built on *asyncio*:

However, the performance bottleneck in *aiohttp* turned out to be its HTTP parser, which is so slow, that it matters very little how fast the underlying I/O library is. To make things more interesting, we created a Python binding for *http-parser* (Node.js HTTP parser C library, originally developed for *NGINX*). The library is called *httptools*, and is available on Github and PyPI.

Now think about that: Selivanov’s HTTP performance tests consisted of a simple echo server written in the different languages/libraries, pounded by the *wrk* benchmarking tool. Most developers would consider a simple echo server an “I/O-bound system,” right? But it turned out that parsing HTTP headers is CPU bound, and it had a slow Python implementation in *aiohttp* in when Selivanov did the benchmarks in 2016. Whenever a function written in Python was parsing headers, the event loop was blocked. The impact was so significant that Selivanov went to the extra trouble of writing *httptools*. Without optimizing the CPU-bound code, the performance gains of a faster event loop were lost.

Death by a Thousand Cuts

Instead of a simple echo server, imagine a complex and evolving Python system with tens of thousands of lines of asynchronous code, interfacing with many external libraries. Years ago I was asked to help diagnose performance problems in a system like that. It was written in Python 2.7 with the *Twisted* framework—a solid library and in many ways a precursor to *asyncio* itself.

Python was used to build a façade for the web UI, integrating functionality provided by preexisting libraries and command-line tools written in other languages—but not designed for concurrent execution.

²⁰ Using a single thread was the default setting until Go 1.5 was released. Years before, Go had already earned a well-deserved reputation for enabling highly concurrent networked systems. One more evidence that concurrency doesn’t require multiple threads or CPU cores.

The project was ambitious; it had been in development for more than a year already, but it was not in production yet.²¹ Over time, the developers noticed that the performance of the whole system was decreasing, and they were having a hard time finding the bottlenecks.

What was happening: with each added feature, more CPU-bound code was slowing down *Twisted*'s event loop. Python's role as a glue language meant there was a lot of data parsing and conversion between formats. There wasn't a single bottleneck: the problem was spread over countless little functions added over months of development. Fixing that would require rethinking the architecture of the system, rewriting a lot of code, probably leveraging a task queue, and perhaps using microservices or custom libraries written in languages better suited for CPU-intensive concurrent processing. The stakeholders were not prepared to make that additional investment, and the project was cancelled shortly afterwards.

When I told this story to Glyph Lefkowitz—founder the *Twisted* project—he said that one of his priorities at the start of an asynchronous programming project is to decide which tools he will use to farm out the CPU-intensive tasks. This conversation with Glyph was the inspiration for “[Avoiding CPU-Bound Traps](#)” on page 826.

²¹ Regardless of technical choices, this was probably the biggest mistake in this project: the stakeholders did not go for an MVP approach—delivering a Minimum Viable Product as soon as possible, and then adding features at a steady pace.